

Một Pipeline Đánh Giá An Toàn Rò Rỉ (Leakage-Safe) cho Bài Toán Source Separation Âm Thanh Tim/Phổi trên HLS-CMDS

[Author Name]
TEEP Internship

2026-08-17, revised 2026-09-13

Abstract

Báo cáo này mô tả một pipeline phần mềm được xây dựng để đánh giá bài toán blind source separation cho âm thanh tim và phổi trên tập dữ liệu HLS-CMDS [1]. Pipeline này gồm: (1) một lớp nạp dữ liệu và kiểm tra tính toàn vẹn (data-loading and integrity-validation layer), bao gồm một audit additive-triplet theo từng hàng cho thấy chỉ có 36 trong số 145 hàng của `Mix.csv` có bản ghi hỗn hợp (mixed recording) thực sự liên quan về mặt âm học đến các nguồn tim/phổi được gán tên của nó — 109 hàng còn lại thất bại ngay cả trước khi xét đến vấn đề rò rỉ (leakage); (2) một cách chia cross-validation an toàn rò rỉ (leakage-safe), ở cấp độ triplet, được xây dựng từ một audit content-hash trên toàn bộ dataset cho thấy một phần ba số bản ghi “isolated” (ground truth riêng lẻ) là bản sao y hệt (literal duplicates) của các bản ghi đã được dùng bên trong các bản mix; (3) một module metric BSS Eval (SDR/SIR/SAR) bọc quanh `mir_eval`, được xác thực (validated) cả bằng cách đối chiếu trực tiếp với `mir_eval` lẫn bằng các bài kiểm tra tổng hợp có nghiệm dạng đóng (closed-form synthetic tests); (4) một evaluation harness k-fold gắn kết cách chia (split) và các metric lại với nhau, với phép tổng hợp (aggregation) ở cấp độ hàng theo từng fold, giữa các fold, và gộp chung (pooled); (5) một tập mixture tổng hợp (synthetic) mang tính tổ hợp (`synthetic_mix.py`) thay thế cho tập 36 hàng additive gốc (native-additive) làm nền đánh giá (evaluation substrate) chính cho một phép sweep SDR-theo-độ-khó liên tục — cách chia triplet/leak-group ở mục (2) được chứng minh sẽ suy biến (collapse) thành một thành phần liên thông (connected component) duy nhất một khi mọi file tim được ghép tổ hợp với mọi file phổi, do đó đơn vị chia (split unit) cho nền dữ liệu này phải chuyển từ cấp độ hàng mixture lên cấp độ từng bản ghi riêng lẻ; và (6) sáu baseline phân tách (separation baselines) được chạy xuyên suốt qua harness — lọc thông dải (bandpass filtering) cố định, một bản tái hiện an toàn rò rỉ (leakage-safe reproduction) của một phương pháp supervised-NMF đã công bố, một ablation NMF không giám sát (unsupervised) theo từng mixture, multi-stage Singular Spectrum Analysis (MSSA), một bản tái hiện của một phương pháp dựa trên EVMD đã công bố, và Conv-TasNet-lite, baseline neural đầu tiên được huấn luyện end-to-end. Chúng tôi báo cáo các đặc điểm chất lượng âm thanh (audio-quality) của tập dữ liệu, cấu trúc của các vấn đề tái sử dụng bản ghi (recording-reuse) và tính hợp lệ của mixture (mixture-validity) mà cách chia và audit additive-triplet được xây dựng để giải quyết, cùng kết quả SDR/SIR/SAR 5-fold của cả sáu baseline trên toàn bộ tập hàng gốc, tập con 36 hàng hợp lệ gốc (native-valid), và nền tổng hợp 1500 hàng, và đặt công trình này trong bối cảnh sáu bài báo liên quan đã được rà soát, không bài nào trong số đó kết hợp cả việc báo cáo chất lượng phân tách, so sánh phân loại isolated-so-với-separated, và một cách chia an toàn rò rỉ trên tập dữ liệu này.

Xây dựng trên pipeline chất lượng phân tách đó, bản sửa đổi này bổ sung thêm (7) một bộ phân loại âm tim (heart-sound classifier) ở hạ nguồn (downstream), sử dụng đặc trưng MFCC và một SVM, được đánh giá trên đúng các fold leak-group giống hệt, với một quyết định gộp lớp (class-grouping decision) tái hiện đúng sơ đồ bốn lớp của chính Yaqub et al., để kết quả có thể so sánh trực tiếp với kết quả gọi mở 89%→41% của họ; (8) một sơ đồ suy giảm có kiểm soát (controlled degradation scheme) nội suy liên tục giữa đầu ra thực của một phương pháp phân tách và ground truth isolated, cho phép đo độ chính xác phân loại như một hàm liên tục theo chất lượng phân tách (SDR) thay vì chỉ tại hai điểm đầu-cuối cố định, cùng với sweep được sinh ra từ đó trên cả sáu baseline; (9) một giao thức chia sẻ trọng số (weight-sharing, “Condition B”) để phân loại âm thanh đã phân tách bằng đúng bộ phân loại đã huấn luyện dùng cho âm thanh isolated, và đường cong accuracy-vs-SDR cùng định nghĩa knee point tương ứng — ngưỡng SDR mà dưới đó việc phân tách không còn tốt hơn việc không phân tách gì cả; (10) một bộ phân loại thứ hai, khác biệt về kiến trúc (log-Mel spectrogram + CNN), dùng làm kiểm tra tính vững (robustness check) xem knee point có phải là một thuộc tính của chất lượng phân tách hay chỉ là hiện tượng riêng của một kiến trúc; và (11) một phân tích chi phí tính toán (computational-cost analysis; số phép multiply-accumulate và độ trễ (latency) đo thực tế trên desktop theo một giao thức thống nhất) đặt chất lượng phân tách của mỗi baseline cạnh chi phí tính toán của nó trên một mặt phẳng Pareto. Sweep sáu baseline, đường cong knee-point thu được và đường cong của kiến trúc thứ hai nay đã hoàn thành và được báo cáo đầy đủ (Sections 8.7–8.9); một bản trước đó báo cáo chúng như kết quả từng phần, và kiểm tra trần oracle-mask mà bản đó liệt kê là không chính thức nay đã là một script được commit, chạy lại được (Section 12).

Nguồn gốc (provenance) của tập dữ liệu đã được xác minh độc lập sau khi pipeline này được xây dựng (Section 3.1): bản sao làm việc giống hệt từng byte (byte-identical) với bản phát hành Mendeley, do đó tần số lấy mẫu 4000 Hz và 109/145 hàng mixture không đạt phép kiểm tra additivity theo tham chiếu được gán tên là các thuộc tính của HLS-CMDS như đã công bố, chứ không phải của một bản phân phối lại đã bị hạ tần số lấy mẫu (downsampled redistribution).

Contents

1	Giới thiệu	4
2	Kiến Thức Nền Tảng và Cơ Sở Lý Thuyết	5
2.1	Bài toán: blind source separation	5
2.2	Làm sao biết một phép phân tách đã thành công? BSS Eval (SDR/SIR/SAR)	6
2.3	Matrix factorization cho âm thanh: Non-negative Matrix Factorization (NMF)	7
2.4	Một phép phân rã khác: Singular Spectrum Analysis (SSA)	8
2.5	Phân rã dựa trên tối ưu hóa: Variational Mode Decomposition (VMD)	9
2.6	Deep learning cho bài toán phân tách: convolution, dilation, và Conv-TasNet	9
2.7	Đánh giá machine learning một cách công bằng: cross-validation và data leakage	10
2.8	Các metric phân loại: accuracy, macro-F1, và confusion matrix	11
2.9	Chi phí tính toán: MAC, latency, và Pareto front	11

3	Tập Dữ Liệu	12
3.1	Nguồn gốc tập dữ liệu: được xác minh đối chiếu với bản phát hành . . .	12
3.2	Audit chất lượng âm thanh	13
3.2.1	Phân tích theo lớp và theo vị trí	13
4	Tái Sử Dụng Bản Ghi và Vấn Đề Rò Rỉ (Leakage)	15
4.1	Audit content-hash	15
4.2	Leak group	15
5	Tính Hợp Lệ của Mixture: Audit Additive-Triplet	16
6	Phương Pháp Luận	18
6.1	Các metric BSS Eval	18
6.1.1	Xác thực (Validation)	18
6.2	Evaluation harness	19
6.3	Tập mixture tổng hợp (synthetic) và cách chia an toàn rò rỉ riêng của nó	20
6.4	Baseline 1: lọc thông dải (bandpass filtering)	21
6.5	Baseline 2: supervised NMF	21
6.6	Baseline 3: standard NMF (dictionary ablation)	22
6.7	Baseline 4: multi-stage Singular Spectrum Analysis (MSSA)	22
6.8	Baseline 5: EVMD (Enhanced Variational Mode Decomposition)	23
6.9	Baseline 6: Conv-TasNet-lite	23
7	Kết Quả	24
7.1	Toàn bộ tập 145 hàng	24
7.2	Tập con 36 hàng additive-valid	25
7.3	Sáu phương pháp, ba nền dữ liệu	26
8	Phân Loại Hạ Nguồn: Việc Phân Tách Có Làm Giảm Độ Chính Xác Không?	28
8.1	Tại sao chỉ một kiến trúc classifier, một cách có chủ đích, lúc đầu	28
8.2	Quyết định gộp lớp (class-grouping)	29
8.3	Kiến trúc 1: thống kê tóm tắt MFCC + SVM kernel RBF	29
8.4	Phân tích chế độ thất bại: confusion matrix	30
8.5	Một cách có kiểm soát để sweep chất lượng phân tách: sơ đồ suy giảm (degradation)	30
8.6	Sinh ra phép sweep SDR	32
8.7	Condition B: phân loại âm thanh đã phân tách, và quyết định weight-sharing	33
8.8	Đường cong accuracy-vs-SDR và knee point	35
8.9	Kiểm tra tính vững: một classifier thứ hai, khác biệt, có thấy cùng một knee hay không?	37
9	Chi Phí Tính Toán	41
9.1	MAC theo từng phương pháp	41
9.2	Độ trễ (latency) trên desktop theo một giao thức thống nhất	42
9.3	Mật phẳng SDR-so-với-compute	44

10Triển Khai Nhúng: Baseline 1 trên Vi Điều Khiển	45
10.1Điểm khác biệt có chủ đích duy nhất: lọc nhân quả (causal filtering) . . .	45
10.2Xác minh một bản port không có triển khai tham chiếu trên mục tiêu . .	46
10.3Kết quả trên silicon	47
10.4Một phép đo trông hợp lý nhưng sai $18\times$	47
10.5Một ngưỡng dung sai được hiệu chuẩn trên sai tín hiệu	48
10.6Server so với vi điều khiển, trên khối lượng việc y hệt	49
10.7Năng lượng, nơi câu trả lời chưa được biết	51
10.8Tại sao năm baseline còn lại không chạy được ở đây	51
11Công Trình Liên Quan	53
11.1Yaqub et al.: kết quả gợi mở, không phải một bài báo phân loại không liên quan	55
11.2Các lưu ý về metric từ tài liệu BSS Eval	55
12Thảo Luận	56
13Hạn Chế và Công Việc Tương Lai	57

1 Giới thiệu

Âm thanh tim và phổi theo quy ước thường được ghi âm và phân loại một cách riêng lẻ (isolation), nhưng trên thực tế một ống nghe (stethoscope) lâm sàng lại thu cả hai cùng lúc. HLS-CMDS [1], theo hiểu biết của chúng tôi, là tập dữ liệu công bố đầu tiên cung cấp các bản ghi cardiorespiratory isolated và mixed tương ứng khớp nhau từ cùng những nguồn gốc, điều này cho phép đặt ra một câu hỏi mà các tập dữ liệu chỉ có isolated hoặc chỉ có mixed dùng ở nơi khác trong tài liệu không thể trả lời trực tiếp: *việc phân tách âm tim khỏi âm phổi, trước khi phân loại nó là bình thường hay bất thường, tốn kém bao nhiêu về độ chính xác so với việc phân loại trực tiếp trên ground truth isolated?*

Câu hỏi đó không mang tính tu từ. Yaqub et al. [6] đã đo chính xác chi phí này trên chính tập dữ liệu của dự án này và phát hiện nó rất nghiêm trọng: một bộ phân loại PCG được huấn luyện lại trên HLS-CMDS đạt 89.0% độ chính xác trên các bản ghi isolated sạch, nhưng sụp xuống còn 41.0% khi được đánh giá trên âm tim được phân tách bằng tính toán (computationally separated) từ các bản ghi mixed của HLS-CMDS bằng một bộ lọc thông dải (bandpass filter) tiêu chuẩn (Experiment 4 của họ; xem Section 11.1). Khoảng cách 89%→41% đó chính là kết quả gợi mở (motivating result) cho toàn bộ dự án này: các metric chất lượng phân tách của báo cáo này (SDR/SIR/SAR, Section 6.1) là thứ cho phép giải thích khoảng cách đó một cách có cơ chế (mechanistically) và truy vết nó liên tục theo chất lượng phân tách, thay vì chỉ đo tại hai điểm đầu-cuối như Yaqub et al. đã báo cáo.

Section 2 trình bày một phần giới thiệu từ đầu, ở mức độ sinh viên (student-level), cho mọi ý tưởng kỹ thuật mà phần còn lại của báo cáo này phụ thuộc vào (blind source separation, BSS Eval, NMF, SSA, VMD, kiến trúc convolutional của Conv-TasNet, cross-validation và data leakage, các metric phân loại, và đo chi phí tính toán) để các phần sau có thể giải thích *tại sao* một lựa chọn cụ thể được đưa ra mà không phải suy diễn lại khái niệm nền tảng mỗi lần. Section 3 mô tả tập dữ liệu, audit chất lượng

âm thanh được chạy trên nó, và một vấn đề về nguồn gốc (provenance) được phát hiện sau khi pipeline dưới đây đã được xây dựng. Section 4 trình bày phát hiện về tái sử dụng bản ghi (recording-reuse) làm động lực cho một cách chia train/evaluation không tầm thường. Section 5 trình bày một vấn đề dữ liệu thứ hai, độc lập: phần lớn các hàng mixture hóa ra không hề liên quan về mặt âm học đến các nguồn được gán tên của chúng. Section 6 mô tả các thành phần phần mềm được xây dựng để hỗ trợ hướng nghiên cứu này: cách chia (split), các metric chất lượng phân tách cùng việc xác thực chúng, harness cross-validation, và cả sáu baseline phân tách. Section 7 báo cáo kết quả trên toàn bộ tập hàng gốc, tập con 36 hàng hợp lệ, và nền tổng hợp 1500 hàng. Section 8 xây dựng trên pipeline chất lượng phân tách đó với một bộ phân loại hạ nguồn (downstream), sơ đồ suy giảm có kiểm soát và sweep SDR mà câu hỏi nghiên cứu C2 của chính dự án này cần, đường cong accuracy-vs-SDR và knee point thu được, và một kiểm tra tính vững bằng kiến trúc thứ hai trên knee point đó. Section 9 đo chi phí tính toán của từng phương pháp (MAC và độ trễ trên desktop) và đặt chất lượng phân tách cạnh chi phí tính toán trên cùng một mặt phẳng. Section 11 khảo sát các công trình liên quan. Section 12 thảo luận về kết quả và ý nghĩa của chúng, và Section 13 tổng kết các công việc còn mở.

Toàn bộ phần mềm được mô tả ở đây nằm trong một repository duy nhất, được tổ chức như sau: `src/load_dataset.py` (nạp/kiểm tra tính hợp lệ, bao gồm audit additive-triplet ở Section 5), `src/split.py` (các fold an toàn rò rỉ), `src/metrics.py` (BSS Eval), `src/eval_harness.py` (cross-validation), `src/baselines.py` (cả hai baseline phân tách), `src/statistics/audio_quality.py` (audit tập dữ liệu), và `src/visualization/` (các script vẽ biểu đồ), cùng với `src/test_metrics.py`, `src/test_split.py`, và `src/test_load_dataset.py` là các bộ test đi kèm (tổng cộng 31 test, tất cả đều pass tại thời điểm viết báo cáo).

2 Kiến Thức Nền Tảng và Cơ Sở Lý Thuyết

Phần này được viết cho một độc giả biết lập trình tổng quát và thống kê cơ bản nhưng chưa nhất thiết từng học về xử lý tín hiệu âm thanh (audio signal processing), matrix factorization, hay phương pháp luận đánh giá machine learning trước đây. Mọi phần sau đều dựa trên một trong những ý tưởng được giải thích ở đây, và tham chiếu ngược lại phần này bằng tên thay vì suy diễn lại ý tưởng đó tại chỗ. Nếu một phần sau nói “vì SAR năm bắt điều này” hay “đây sẽ là data leakage,” thì lời giải thích cho lý do điều đó quan trọng nằm ở đây, không được lặp lại mỗi lần.

2.1 Bài toán: blind source separation

Một ống nghe (stethoscope) đặt trên ngực thu âm tim và âm phổi cùng một lúc, được cộng dồn về mặt vật lý thành một dạng sóng (waveform) duy nhất bởi không khí và mô mà chúng truyền qua trước khi đến microphone. Gọi waveform âm tim thực là $h(t)$, waveform âm phổi thực là $l(t)$, và bản mix được ghi lại là $m(t)$. Mô hình vật lý đơn giản nhất cho những gì đã xảy ra là một phép trộn tuyến tính (linear mix):

$$m(t) = a \cdot (h(t) + l(t)) + n(t), \quad (1)$$

trong đó a là một hệ số khuếch đại ghi âm tổng thể và $n(t)$ là bất kỳ nhiễu bổ sung nào. *Source separation* có nghĩa là khôi phục các ước lượng $\hat{h}(t)$ và $\hat{l}(t)$ chỉ từ $m(t)$. Nó

được gọi là *blind* khi phương pháp phân tách không được cho biết h và l riêng lẻ nghe như thế nào đối với bản ghi cụ thể này — nó chỉ nhận được $m(t)$, cộng với nhiễu nhất là một số kiến thức tổng quát về việc âm tim và âm phổi thường có dạng như thế nào (học được từ các bản ghi khác, tách biệt).

Đây là một bài toán vô định (underdetermined) về bản chất: Equation 1 là một phương trình với hai hàm số chưa biết ở vế phải, do đó có vô số cặp $(\hat{h}, \hat{l})$ cộng lại đúng bằng m quan sát được. Lý do duy nhất khiến việc phân tách khả thi là vì âm tim và âm phổi không phải ngẫu nhiên — chúng khác nhau một cách có hệ thống ở vị trí năng lượng của chúng nằm ở đâu, cho dù điều đó được mô tả theo tần số (Sections 6.4, 6.7, 6.8), trong một “từ điển” (dictionary) phổ đã học được (Sections 6.5, 6.6), hay trong các mẫu hình mà một mạng neural tự khám phá ra từ các ví dụ (Section 6.9). Mọi phương pháp phân tách trong báo cáo này, bên dưới phần toán học riêng của nó, đều là một cách khác nhau để khai thác sự khác biệt có hệ thống đó nhằm phá vỡ bài toán vô định.

2.2 Làm sao biết một phép phân tách đã thành công? BSS Eval (SDR/SIR/SAR)

Một khi một phương pháp tạo ra một ước lượng $\hat{s}_j$ cho nguồn j (tim hoặc phổi), ý tưởng đầu tiên tự nhiên là so sánh nó với nguồn thực s_j bằng thứ gì đó như mean-squared error. Điều đó hóa ra là một thước đo kém đối với âm thanh, vì nó không phân biệt được lý do $\hat{s}_j$ khác với s_j — một độ lệch thời gian gần như không nghe thấy được và một tín hiệu hoàn toàn sai có thể tạo ra cùng một squared error. Khung BSS Eval của Vincent et al. [2] (Section 11.2) khắc phục điều này bằng cách phân rã sai số thành ba phần với các nguyên nhân khác nhau:

$$\hat{s}_j = s_{\text{target}} + e_{\text{interf}} + e_{\text{artif}}, \quad (2)$$

trong đó s_{target} là phiên bản lọc bất biến theo thời gian (time-invariant-filtered) tốt nhất của nguồn thực s_j có thể được trích xuất từ ước lượng (do đó một thay đổi gain vô hại hay một độ trễ ngắn, cố định sẽ không bị phạt), e_{interf} là phần sai số của ước lượng được giải thích bởi việc các nguồn khác rò rỉ vào (ví dụ năng lượng âm phổi xuất hiện trong ước lượng âm tim), và e_{artif} là phần còn lại — méo dạng (distortion) mà bản thân thuật toán phân tách gây ra và không thể giải thích được bằng nhiễu còn sót lại (leftover interference) (filter ringing, sai số tái tạo NMF, ảo giác của mạng neural). Ba tỉ số tiếp theo trực tiếp từ đó, mỗi tỉ số trả lời một câu hỏi khác nhau, luôn được biểu diễn theo decibel (thang logarit cơ số 10, $10 \log_{10}(\cdot)$), được chọn vì tỉ số năng lượng âm thanh trải dài qua nhiều bậc độ lớn và cảm nhận độ to của con người tự nó xấp xỉ logarit — một tỉ số tuyến tính sẽ khiến các trường hợp lớn, dễ phân tách và các trường hợp nhỏ, khó phân tách trông giống nhau một cách đánh lừa trên đồ thị):

$$\text{SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}} + e_{\text{artif}}\|^2} \quad \text{“ước lượng tốt đến mức nào, nói chung?”} \quad (3)$$

$$\text{SIR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}}\|^2} \quad \text{“bao nhiêu phần của nguồn kia đã rò rỉ vào?”} \quad (4)$$

$$\text{SAR} = 10 \log_{10} \frac{\|s_{\text{target}} + e_{\text{interf}}\|^2}{\|e_{\text{artif}}\|^2} \quad \text{“bản thân phương pháp đã làm méo tín hiệu bao nhiêu?”} \quad (5)$$

Tại sao báo cáo cả ba thay vì chỉ SDR. Một phương pháp có thể đạt điểm tốt ở chỉ số này nhưng kém ở chỉ số khác, và khoảng cách đó tự nó mang tính chẩn đoán (diagnostic). Chẳng hạn, một bộ lọc thông dải (bandpass filter) mạnh tay chặn nguồn còn lại hiệu quả (SIR tốt) nhưng cũng gây ringing và phá hủy các họa âm (harmonics) của chính mục tiêu trong quá trình đó (SAR kém); một phương pháp gần như không làm gì cả lại đạt điểm theo chiều ngược lại. SDR gộp cả hai hiệu ứng vào một con số, làm nó tiện lợi nhưng cũng có nghĩa là một SDR tốt có thể che giấu một SAR kém mà một bộ phân loại hạ nguồn (downstream classifier) có thể thực sự quan tâm (luận điểm ở Section 11.2 từ Vincent et al. rằng một bộ phân loại có thể vẫn chính xác bất chấp artifact thực sự, hoặc thất bại vì chúng, tùy vào việc artifact đó có vượt qua một ranh giới quyết định (decision boundary) hay không) — chỉ báo cáo SDR sẽ xóa mất đúng loại đánh đổi (trade-off) mà Section 7 cần mô tả.

2.3 Matrix factorization cho âm thanh: Non-negative Matrix Factorization (NMF)

Baseline 2 và 3 (Sections 6.5–6.6) không làm việc trên waveform thô mà trên *spectrogram* của nó: chia waveform thành các cửa sổ (windows) ngắn, chồng lấp nhau (a Short-Time Fourier Transform, STFT), lấy độ lớn (magnitude) của phổ tần số mỗi cửa sổ, và xếp các cột này cạnh nhau thành một ma trận V có kích thước (số bin tần số) $\times$ (số khung thời gian). Mọi phần tử của V là một giá trị năng lượng không âm — không tồn tại khái niệm năng lượng âm thanh âm (negative) tại một tần số và thời điểm cho trước.

Non-negative Matrix Factorization đặt câu hỏi: liệu V (kích thước $F \times T$) có thể được xấp xỉ như tích của hai ma trận không âm nhỏ hơn nhiều, $V \approx WH$, trong đó W (kích thước $F \times K$) chứa K “phổ cơ sở” (basis spectra) (các dạng tần số lặp lại) và H (kích thước $K \times T$) chứa, với mỗi khung thời gian, mức độ mỗi phổ cơ sở đang hoạt động mạnh đến đâu? K được chọn nhỏ hơn nhiều so với F hoặc T , do đó đây là một phép nén buộc thuật toán phải giải thích toàn bộ spectrogram chỉ bằng K khối xây dựng (building blocks) có thể tái sử dụng. Ràng buộc không âm chính là điều làm cho kết quả có thể diễn giải được như một biểu diễn *dựa trên các thành phần* (parts-based) (khác với, chẳng hạn, các phép phân rã PCA/SVD, vốn cho phép tổ hợp âm và tạo ra các thành phần chỉ có ý nghĩa khi cộng và trừ cùng nhau, không phải riêng lẻ): mỗi phổ cơ sở trong W là thứ mà một âm thanh thực sự có thể thực sự trông giống, và mỗi cột của H nói lên “có bao nhiêu của mỗi âm thanh thực đang hiện diện ngay lúc này,” theo cách không âm.

Việc fit W và H được thực hiện bằng *multiplicative updates* (Lee & Seung, 2001): bắt đầu từ một phỏng đoán không âm ngẫu nhiên, W và H được rescale lặp đi lặp lại bằng một tỉ số được đảm bảo không âm và được chứng minh làm giảm sai số tái tạo (reconstruction error) (cụ thể là KL-divergence giữa V và WH được dùng ở đây) ở mỗi bước. Đây là một dạng coordinate descent — đơn giản để triển khai và, khác với gradient descent tổng quát, không bao giờ cần một learning rate hay có nguy cơ tạo ra một giá trị âm phải bị clip.

Supervised NMF (Baseline 2, Section 6.5) khai thác điều này bằng cách học W *riêng biệt* cho tim và phổi, từ các bản ghi isolated, sạch của mỗi loại, trước khi từng nhìn vào một mixture — hoàn toàn tương tự việc học một bảng màu (palette) nhỏ gồm các “màu cơ sở” (basis colors) từ các mẫu sơn thuần khiết. Hai ma trận W sau đó được đóng băng (frozen) và xếp chồng lại với nhau, và chỉ H được giải lại (với cùng

quy tắc multiplicative-update, giữ W cố định) đối với spectrogram của một *mixture* cụ thể được giữ lại (held-out). Điều này trả lời câu hỏi “cần bao nhiêu của mỗi phổ cơ sở đã biết, tại mỗi thời điểm, để tái tạo lại mixture mới, chưa gán nhãn này?” — cùng ý tưởng như phân rã một màu trộn chưa biết thành tổ hợp của các màu cơ bản đã biết. Một khi H cho biết bao nhiêu phần của dictionary-tim và dictionary-phổi giải thích mixture tại mỗi điểm thời gian-tần số, tỉ lệ đó trở thành một *soft mask*: phần của spectrogram phức (complex spectrogram) của mixture tại mỗi điểm được gán cho tim là $W_{\text{heart}}H_{\text{heart}}/(W_{\text{heart}}H_{\text{heart}} + W_{\text{lung}}H_{\text{lung}})$, được áp dụng trực tiếp lên STFT (giá trị phức) của chính mixture trước khi biến đổi ngược lại thành waveform — đây là một mask kiểu *Wiener* (Wiener-style), cách chuẩn để biến một phép phân rã năng lượng thành một ước lượng âm thanh thực sự. **Standard NMF** (Baseline 3, Section 6.6) là cùng cơ chế đó nhưng bỏ đi một thành phần: hoàn toàn không có dictionary học trước, cả W lẫn H đều được factorize mới hoàn toàn từ spectrogram riêng của mỗi mixture (một cách dùng NMF thuần túy *blind*, hay *transductive*), điều này tách bạch xem bao nhiêu phần hiệu năng của Baseline 2 thực sự đến từ dictionary học được, đóng băng của nó, thay vì từ bản thân phép factorization NMF.

2.4 Một phép phân rã khác: Singular Spectrum Analysis (SSA)

Baseline 4 (Section 6.7) dùng một công cụ toán học hoàn toàn khác: Singular Spectrum Analysis. Cách xây dựng gồm ba bước. Đầu tiên, *embedding*: biến waveform 1 chiều có độ dài N thành một ma trận 2 chiều bằng cách trượt một cửa sổ độ dài L dọc theo nó từng mẫu (sample) một và xếp mỗi cửa sổ thành một cột, tạo ra một *trajectory matrix* kích thước $L \times (N - L + 1)$. Điều này biến “điều gì xảy ra theo thời gian” thành “cấu trúc của ma trận này trông như thế nào,” thứ mà đại số tuyến tính có các công cụ mạnh để xử lý. Thứ hai, *phân rã*: áp dụng Singular Value Decomposition (SVD), viết bất kỳ ma trận nào thành tổng của các thành phần hạng 1 (rank-1) (mỗi thành phần là một tích ngoài outer product của một vector “mẫu hình thời gian” và một vector “mẫu hình độ trễ”), được sắp xếp theo lượng năng lượng tổng của trajectory matrix mà mỗi thành phần giải thích được. Các thành phần hạng 1 này là cùng một đối tượng toán học đứng sau PCA, và mỗi thành phần tương ứng với một tần số dao động đặc trưng nào đó trong tín hiệu gốc — SSA không ép buộc cách diễn giải tần số này, nhưng nó tự nhiên xuất hiện vì một ma trận được xây từ một tín hiệu tuần hoàn đã qua windowing có đúng loại cấu trúc hạng thấp (low-rank) này. Thứ ba, *diagonal averaging* (Hankelization): mỗi thành phần hạng 1 được chọn, vẫn có dạng ma trận $L \times (N - L + 1)$, được chuyển ngược lại thành một *reconstructed component* (RC) 1 chiều bằng cách lấy trung bình dọc theo các đường chéo phụ (anti-diagonals) của nó, hoàn tác lại bước windowing từ bước một.

Với một tập các RC, mỗi RC có tần số trội (dominant frequency) riêng, việc phân tách tim khỏi phổi rút gọn lại thành việc sắp xếp chúng theo tần số đó và gộp các RC tần số thấp thành một đầu ra tim mạch (cardiac) và các RC tần số cao hơn thành một đầu ra hô hấp (respiratory) — đây chính xác là điều mà thiết kế hai giai đoạn của MSSA thực hiện (một lượt đầu với một ngưỡng tần số tim mạch/hô hấp cố định, sau đó một lượt SSA thứ hai trên phần dư còn lại để chọn lọc kỹ hơn nội dung hô hấp dựa trên lượng phương sai mỗi RC giải thích được và mức độ tương quan của nó với những gì đã được chọn). Không có dictionary nào được học từ các bản ghi khác ở bất kỳ đâu trong quá trình này — SSA hoạt động hoàn toàn trên một tín hiệu duy nhất

mà nó được cung cấp, đó là lý do Baseline 4 (giống Baseline 1) là một phương pháp zero-training không có phơi nhiễm rò rỉ (leakage exposure) nào cả (Section 4).

2.5 Phân rã dựa trên tối ưu hóa: Variational Mode Decomposition (VMD)

Baseline 5 (Section 6.8) dùng Variational Mode Decomposition (Dragomiretskiy & Zosso, 2014), phương pháp này đặt bài toán phân tách như một bài toán tối ưu hóa tường minh thay vì một matrix factorization hay một bộ lọc cố định: tìm K “mode,” mỗi mode là một tín hiệu càng hẹp băng (narrow-band) càng tốt (tập trung chặt xung quanh tần số trung tâm ω_k được phát hiện đồng thời của chính nó), với ràng buộc là toàn bộ K mode cộng lại đúng bằng tín hiệu gốc. “Càng hẹp băng càng tốt” được làm chính xác hóa bằng cách tối thiểu hóa tổng băng thông (bandwidth) của các mode (đo qua phép biến đổi Hilbert của tín hiệu, dịch phổ của một tín hiệu thực sao cho nội dung năng lượng của nó tại mỗi tần số có thể được tách riêng và bị phạt). Bài toán kết hợp này — K tín hiệu chưa biết và K tần số trung tâm chưa biết, tất cả được liên kết qua một ràng buộc chung — được giải bằng Alternating Direction Method of Multipliers (ADMM), một chiến lược tối ưu hóa tổng quát cho chính xác tình huống này: thay vì giải mọi thứ cùng lúc, luân phiên giữa việc cập nhật mỗi mode (giữ cố định tất cả các mode khác và các tần số) và cập nhật mỗi tần số trung tâm (giữ cố định các mode), lặp lại cho đến khi các mode ngừng thay đổi. Một khi mode có tần số trung tâm thấp nhất được xác định là thành phần tim mạch, trừ nó khỏi mixture sẽ cho ra ước lượng hô hấp. Bản thân K không được biết trước và được quét (sweep) từ 2 trở lên cho đến khi một kiểm tra chất lượng theo bài báo gốc được thỏa mãn (hoặc chạm một trần giới hạn) — một chi tiết thực hành khác được ghi lại trong Section 6.8.

2.6 Deep learning cho bài toán phân tách: convolution, dilation, và Conv-TasNet

Baseline 6 (Section 6.9) thay một phép biến đổi được thiết kế thủ công (STFT, trajectory matrix, mode decomposition) bằng các trọng số được học end-to-end từ dữ liệu. Phép toán cốt lõi của nó là convolution 1 chiều: một bộ lọc ngắn, được học (một vector nhỏ các trọng số) được trượt qua waveform, và tại mỗi vị trí nó tính một tổng có trọng số của các mẫu bên dưới nó — về mặt khái niệm cùng ý tưởng sliding-window như bộ lọc Butterworth của Baseline 1 (Section 6.4), ngoại trừ các hệ số của bộ lọc được học từ ví dụ thay vì được suy ra thủ công từ một khảo sát PSD. Xếp chồng nhiều lớp convolutional như vậy, mỗi lớp hoạt động trên đầu ra của lớp trước, cho phép mạng xây dựng các mẫu hình ngày càng trừu tượng, tầm xa hơn từ các mẫu hình ngắn, cục bộ.

Một chồng convolution thuần túy sẽ cần rất nhiều lớp để “nhìn thấy” một lượng ngữ cảnh (context) hữu ích (mỗi lớp chỉ mở rộng những gì lớp tiếp theo có thể nhìn thấy thêm khoảng bằng độ dài bộ lọc của chính nó), đây là lúc convolution *dilated* xuất hiện: một bộ lọc dilated bỏ qua các vị trí đầu vào theo một khoảng cách cố định, do đó cùng một số lượng nhỏ trọng số bao phủ một phạm vi đầu vào rộng hơn nhiều mà không thêm tham số. Xếp chồng các lớp với dilation tăng theo cấp số nhân (như Temporal Convolutional Network, TCN, của Conv-TasNet thực hiện: dilation $2^0, 2^1, 2^2, \dots$) cho phép *receptive field* hiệu dụng của mạng — lượng đầu vào mà bất kỳ giá trị đầu ra nào phụ thuộc vào — tăng theo cấp số nhân theo độ sâu thay vì tuyến

tính, đây là cách Baseline 6 bao phủ khoảng 1.27 s ngữ cảnh âm thanh chỉ với một số ít lớp (Section 6.9).

Kiến trúc tổng thể của Conv-TasNet có ba phần được huấn luyện cùng nhau: một *encoder* (một bank các bộ lọc convolution 1 chiều đã học, đóng vai trò tương tự STFT đối với các baseline NMF, ngoại trừ hình dạng bộ lọc được học thay vì là các sóng sin cố định), một *separator* (TCN dilated được mô tả ở trên, nhìn vào mixture đã được encode và dự đoán một soft mask cho mỗi nguồn), và một *decoder* (nghịch đảo toán học của encoder, biến một biểu diễn đã encode và đã mask trở lại thành waveform). Huấn luyện end-to-end như vậy có nghĩa là mạng không bao giờ cần được cho biết một “phổ cơ sở” hay một “dải tần số” là gì — nó tự khám phá bất kỳ biểu diễn và chiến lược masking nào tối thiểu hóa loss huấn luyện của nó một cách trực tiếp, đây vừa là lợi thế chính của Baseline 6 (về nguyên tắc nó có thể khám phá ra cấu trúc mà các phép biến đổi cố định của năm phương pháp còn lại không thể biểu diễn) và cũng là chi phí thực tế chính của nó (nó cần đủ ví dụ huấn luyện đã gán nhãn để tìm ra cấu trúc đó một cách đáng tin cậy, điều mà cả Section 6.9 và Section 7 đều đánh dấu là một hạn chế thực sự trên tập dữ liệu nhỏ về số bản ghi này).

2.7 Đánh giá machine learning một cách công bằng: cross-validation và data leakage

Bất cứ khi nào một phương pháp được *fit* vào dữ liệu (một dictionary đã học, một mạng đã huấn luyện, thậm chí một ngưỡng cắt bộ lọc được tinh chỉnh thủ công), việc đánh giá nó trên chính dữ liệu nó đã được fit sẽ phóng đại mức độ nó sẽ hoạt động tốt trên các bản ghi mới thực sự — phương pháp đó có thể chỉ đơn giản là ghi nhớ (memorized) các đặc điểm riêng của dữ liệu cụ thể đó thay vì học được điều gì đó tổng quát. Cách khắc phục tiêu chuẩn là giữ lại (hold out) một phần dữ liệu, chỉ fit trên phần còn lại, và chỉ đánh giá trên phần được giữ lại. *k-fold cross-validation* mở rộng điều này sao cho mọi điểm dữ liệu đều được nằm trong tập giữ lại đúng một lần (qua k “fold” khác nhau), do đó toàn bộ tập dữ liệu được dùng cho cả việc fit lẫn đánh giá mà không bao giờ đánh giá kết quả của một fold bằng dữ liệu nó đã được fit trên đó — quan trọng trên một tập dữ liệu nhỏ như thế này, nơi việc dành riêng một tập giữ lại lớn, cố định sẽ để lại quá ít dữ liệu để fit bất cứ điều gì có ý nghĩa.

Data leakage là tên gọi chung cho việc đánh giá một mô hình theo cách để cho thông tin từ tập giữ lại ảnh hưởng đến những gì nó được fit trên, thậm chí một cách gián tiếp, điều này âm thầm thổi phồng điểm số đo được mà mô hình thực ra không hề tổng quát hóa tốt hơn. Ví dụ kinh điển là fit và đánh giá trên dữ liệu hóa ra chứa các hàng trùng lặp: nếu một bản gán-sao-chép của một ví dụ test nằm trong tập huấn luyện, mô hình không cần phải tổng quát hóa để đạt điểm tốt trên nó. Leak-group split của dự án này (Section 4) tồn tại chính xác là vì HLS-CMDS có đúng vấn đề này ở một dạng không hiển nhiên: cùng một bản ghi vật lý xuất hiện nhiều lần dưới *các chuỗi ID khác nhau* (một lần độc lập trong HS.csv/LS.csv, và một lần nữa bên trong một hoặc nhiều hàng của Mix.csv), do đó một cách chia k -fold chuẩn, chung chung — chỉ biết về ID hàng, không biết về nội dung âm thanh nền tảng mà các ID đó trở tới — sẽ âm thầm làm rò rỉ chính ground truth của một mixture vào tập được dùng để fit dictionary phân tách của nó. Việc phát hiện điều này đòi hỏi hash trực tiếp nội dung âm thanh thô thay vì tin tưởng vào các cột ID (Section 4), và đơn vị chia kết quả không phải là hàng mà là một *leak group* (một thành phần liên thông của các hàng chia sẻ chung âm thanh) — thay đổi tối thiểu đối với cross-validation chuẩn thực sự

bị kín được rò rỉ cụ thể này.

2.8 Các metric phân loại: accuracy, macro-F1, và confusion matrix

Từ Section 8 trở đi, báo cáo đánh giá một *classifier* (một bản ghi âm tim thuộc về Normal, Murmur, Extra Sound, hay Rhythm Disorder?), việc này cần từ vựng riêng của nó. *Accuracy* đơn giản là tỉ lệ các ví dụ được phân loại đúng, nhưng nó có một điểm mù nổi tiếng trên dữ liệu mất cân bằng (imbalanced): nếu 24 trong số 50 bản ghi là Murmur và một classifier luôn dự đoán “Murmur” bất kể đầu vào, nó đạt 48% accuracy trong khi không hề học được điều gì hữu ích về ba lớp còn lại. *Macro-F1* là một cách khắc phục tiêu chuẩn: tính điểm F1 (trung bình điều hòa của precision và recall) riêng cho *mỗi* lớp, sau đó lấy trung bình các điểm theo từng lớp đó với trọng số bằng nhau — do đó một mô hình làm tốt lớp phổ biến nhưng thất bại ở mọi lớp hiếm không còn có thể ẩn mình sau kích thước của lớp phổ biến. Một *confusion matrix* là nguyên liệu thô mà cả hai metric trên được tính từ đó: một bảng mà các hàng là lớp thực (true class) và các cột là lớp dự đoán (predicted class), do đó ô (i, j) đếm có bao nhiêu ví dụ thuộc lớp thực i được dự đoán là lớp j . Đường chéo là những gì classifier đã đúng; mọi thứ ngoài đường chéo là một lỗi cụ thể, có thể gọi tên được (“các bản ghi Rhythm Disorder thường bị dự đoán nhầm nhất là Murmur”), điều này hữu ích hơn nhiều so với một con số tổng hợp đơn lẻ và chính là mức độ chi tiết mà Section 8.4 dùng để mô tả mẫu hình thất bại cụ thể của mỗi kiến trúc.

2.9 Chi phí tính toán: MAC, latency, và Pareto front

Section 9 đặt ra một loại câu hỏi khác: không phải “phương pháp này tốt đến mức nào,” mà là “chạy nó tốn kém đến mức nào.” Đơn vị chi phí cơ bản (atomic unit) được dùng xuyên suốt phần đó là *multiply-accumulate* (MAC): một phép nhân theo sau bởi một phép cộng vào một tổng chạy (running total), đây là phép toán đơn lẻ mà hầu như mọi phương pháp trong báo cáo này được xây dựng từ đó ở lõi của nó, cho dù đó là một phép nhân ma trận bên trong NMF, một convolution bên trong Conv-TasNet, hay một SVD bên trong SSA. Đếm MAC cho một đại lượng thay thế (proxy) độc lập với phần cứng cho “lượng công việc số học” mà một phương pháp cần — hữu ích vì nó có thể được tính toán một cách giải tích từ chính các phương trình của phương pháp đó, mà không cần thực sự chạy nó. Tuy nhiên, nó chỉ là một *proxy*: một số lượng MAC cho trước thực thi nhanh đến mức nào trên phần cứng thực phụ thuộc rất nhiều vào việc số học đó ánh xạ tốt đến đâu lên các đường dẫn mã đã được tối ưu hóa riêng của phần cứng (các lệnh vector hóa, truy cập bộ nhớ thân thiện với cache, một thư viện đã biên dịch trưởng thành so với một triển khai trực tiếp, chưa tối ưu của chính pseudocode trong bài báo). Section 9 đo cả MAC (proxy lý thuyết) lẫn latency thực tế theo đồng hồ treo tường (wall-clock) (kết quả đo được) *riêng biệt*, cụ thể là vì chúng hóa ra không đồng nhất với nhau đối với ít nhất một phương pháp ở đây — một sự khác biệt mà chỉ riêng số lượng MAC không bao giờ có thể tiết lộ được.

Việc so sánh nhiều phương pháp trên hai trục cùng lúc (chất lượng và chi phí) dùng ý tưởng về một *Pareto front*, mượn từ tối ưu hóa đa mục tiêu (multi-objective optimization): phương pháp A được nói là *thống trị* (dominate) phương pháp B nếu A tốt bằng hoặc hơn B trên mọi trục đang được so sánh và tốt hơn hẳn trên ít nhất một trục. Một phương pháp bị thống trị không bao giờ là lựa chọn đúng, bất kể hai trục được cân nhắc trọng số ra sao với nhau, vì một phương pháp khác đơn giản là tốt

hơn ở mọi khía cạnh quan trọng; một phương pháp *không* bị bất kỳ phương pháp nào thống trị đại diện cho một đánh đổi thực sự (chất lượng tốt hơn với chi phí cao hơn, hoặc chi phí thấp hơn với một mất mát chất lượng nào đó) mà một quyết định triển khai thực sự sẽ phải cân nhắc. Section 9.3 áp dụng điều này trực tiếp vào đánh đổi SDR-so-với-chi-phí-tính-toán của sáu baseline phân tách.

3 Tập Dữ Liệu

HLS-CMDS [1] ghi âm tim và phổi từ một bộ mô phỏng bệnh nhân mid-fidelity CAE Juno với các lớp da ngực (chest skins) nam/nữ có thể hoán đổi, được thu bằng một ống nghe kỹ thuật số trong một phòng yên tĩnh. Âm tim được ghi tại sáu mốc ngực (Apex, RUSB, LUSB, LLSB, RC, LC) qua mười lớp loại âm thanh (Normal, Late Diastolic Murmur, Mid Systolic Murmur, Late Systolic Murmur, Atrial Fibrillation, S4, Early Systolic Murmur, S3, Tachycardia, AV Block); âm phổi được ghi tại sáu mốc phía trước (LLA, LMA, LUA, RLA, RMA, RUA) qua sáu lớp (Normal, Wheezing, Fine Crackles, Rhonchi, Pleural Rub, Coarse Crackles). Tập dữ liệu đi kèm ba manifest CSV: `HS.csv` (50 bản ghi âm tim isolated), `LS.csv` (50 bản ghi âm phổi isolated), và `Mix.csv` (145 bản ghi mixed, mỗi hàng cũng nêu tên bản ghi tim và phổi được dùng để tạo ra mixture đó).

3.1 Nguồn gốc tập dữ liệu: được xác minh đối chiếu với bản phát hành

Bản sao tập dữ liệu mà mọi kết quả trong báo cáo này được tính toán trên là bản đã phát hành. Bản sao làm việc trong dự án này (HLS_CMDS/) được lấy từ github.com/Torabiy/HLS-CMDS thay vì bản phát hành Mendeley (data.mendeley.com/datasets/897) mà project charter quy định, và một đợt rà soát nội bộ (*TEEP2026_Sprint0_Review*, 2026-08-18) đã đánh dấu khả năng phát sinh rằng mọi thứ ở đây đã được đo trên một bản mirror tiện lợi đã bị hạ tần số lấy mẫu (downsampled). Khả năng đó đã được kiểm tra trực tiếp và không đúng: một lượt tải xuống Mendeley mới (v3, lấy ngày 2026-08-19), được checksum độc lập với công cụ của chính dự án này, giống hệt từng byte (byte-identical) với bản sao làm việc trên cả sáu artifact cấp cao nhất (Table 1).

Table 1: Kiểm tra nguồn gốc. Mendeley v3, lấy ngày 2026-08-19; MD5 trên mỗi artifact cấp cao nhất, giống hệt nhau giữa bản tải xuống mới và bản sao làm việc.

Artifact	MD5
HS.csv	b61a866d2e8d70ae2910e4c049327d1e
HS.zip	7d678aff168184c13d8893b3bef4f4ba
LS.csv	fa40494d1dcbcf173ee5e9583b3faf82
LS.zip	d56a0d8ce5941342f33bfa9b04ef2bd9
Mix.csv	377b55ea6361ff7c48994cddaf5c6e6f
Mix.zip	f28243aa5394324c58f90a466d60c2f4

Hai phát hiện vẫn đúng sau khi kiểm tra, như những thuộc tính của tập dữ liệu đã công bố, đó là lý do tiểu mục này được giữ lại thay vì bị xóa:

- **Tần số lấy mẫu.** Mọi file đều là 4000 Hz (Section 3.2), không phải 22,050 Hz mà bài báo mô tả HLS-CMDS nêu ra — một sự chênh lệch $5.5\times$ thuộc về chính bản phát hành. Việc bài báo mô tả sai tần số hay con số bị chép sai ở khâu sau đó không được giải quyết ở đây. Đây không phải là vấn đề đối với khoa học, vì năng lượng âm tim nằm ở 20–200 Hz và âm phổi ở 100–1000 Hz (Section 6.4), thoải mái dưới ngưỡng Nyquist 2000 Hz; nhưng nó là vấn đề đối với công cụ, và Section 6.9 ghi lại hệ quả của điều này.
- **Tính hợp lệ của mixture.** 109 trong số 145 hàng Mix.csv có bản ghi *mixed* không liên quan về mặt âm học đến các nguồn tim và phổi được gán tên của nó (Section 5); chỉ có 36 hàng dùng được làm ground truth phân tách theo cặp. Tải lại không thể khôi phục chúng, vì bản tải lại là cùng những byte đó.

Cả hai con số đều đến từ chính nội dung âm thanh của HS/LS/Mix.csv, không phải từ metadata, do đó chúng không phải là hiện tượng giả (artifacts) của một manifest bị gán nhãn sai. Theo đó, mọi con số chất lượng phân tách trong báo cáo này là một phép đo trên HLS-CMDS như đã công bố.

3.2 Audit chất lượng âm thanh

src/statistics/audio_quality.py quét mọi file .wav được tham chiếu bởi ba manifest (tổng cộng 535 file: $50 + 50 + 145 \times 3$, vì các thành phần tim, phổi, và mixed của mỗi hàng mix là các file riêng biệt) và báo cáo thời lượng, tần số lấy mẫu, số kênh, và clipping (các mẫu bị ghim ở mức rail full-scale 16-bit PCM, ± 32767), được tổng hợp theo từng lớp loại âm thanh và từng vị trí ghi âm.

Định dạng. Mọi file đều là 4000 Hz, mono, đúng 15.0 s (60,000 mẫu) — hoàn toàn đồng nhất, không cần resampling hay cắt xén trước khi trích xuất đặc trưng. Sự đồng nhất này tự nó là một phần của sự khác biệt về nguồn gốc: 4000 Hz không phải là tần số lấy mẫu mà bài báo mô tả HLS-CMDS ghi lại (Section 3.1).

Clipping. 36 trong số 535 file (6.7%) bị clip, nhưng chỉ đúng một mẫu mỗi file ($1/60,000 \approx 0.0017\%$ của file), và chỉ xảy ra ở thành phần *mixed* của các hàng Mix.csv (không bao giờ ở HS.csv, LS.csv, hay các thành phần tim/phổi riêng của tập mix) — cụ thể là hàng M0087 và dải liên tục M0111–M0145. Tập 36 hàng này chính xác là tập mà audit additive-triplet của Section 5 độc lập tìm thấy là hợp lệ về mặt âm học, điều này giải thích được cơ chế: đây là những hàng mà *mixed* thực sự được xây dựng bằng cách cộng các waveform tim và phổi rồi peak-normalize kết quả (một hệ số khuếch đại quan sát được từ $3.5\text{--}135\times$ trên 36 hàng), và peak normalization của một tổng đã ở full-scale sẽ ghim một mẫu ở mức rail 16-bit. Đây là một dấu vân tay của normalization (normalization fingerprint), không phải một lỗi ghi âm, và nó chưa từng được báo cáo ở nguồn gốc (upstream).

3.2.1 Phân tích theo lớp và theo vị trí

Ngoài bản tóm tắt toàn tập dữ liệu ở trên, audio_quality.py cũng tổng hợp thời lượng, tần số lấy mẫu, và clipping *theo từng lớp* và *theo từng vị trí ghi âm*, trên 100 bản ghi isolated (HS.csv + LS.csv). Thời lượng (15.0 s, std 0.0) và tần số lấy mẫu (4000 Hz) không đổi trên mọi lớp và mọi vị trí — được lược bỏ khỏi Tables 2–3 bên dưới vì không có sự biến thiên nào để báo cáo — do đó các cột duy nhất mang thông tin là số lượng mẫu của lớp/vị trí và số file bị clip của nó.

Table 2: Phân tích theo lớp, chỉ các bản ghi isolated (HS.csv/LS.csv).

Thành phần	Lớp	n	Bị clip
Heart	Normal	9	0
Heart	Mid Systolic Murmur	7	0
Heart	Late Diastolic Murmur	6	0
Heart	Early Systolic Murmur	6	0
Heart	Late Systolic Murmur	5	0
Heart	S3	5	0
Heart	Atrial Fibrillation	4	0
Heart	AV Block	3	0
Heart	Tachycardia	3	0
Heart	S4	2	0
Lung	Normal	12	0
Lung	Coarse Crackles	9	0
Lung	Pleural Rub	9	0
Lung	Rhonchi	8	0
Lung	Wheezing	7	0
Lung	Fine Crackles	5	0

Table 3: Phân tích theo vị trí, chỉ các bản ghi isolated (HS.csv/LS.csv).

Thành phần	Vị trí	n	Bị clip
Heart	LUSB	13	0
Heart	Apex	10	0
Heart	LLSB	10	0
Heart	RUSB	7	0
Heart	LC	6	0
Heart	RC	4	0
Lung	LUA	11	0
Lung	RLA	10	0
Lung	LMA	9	0
Lung	LLA	8	0
Lung	RUA	7	0
Lung	RMA	5	0

Tại sao phân tích này dùng được. Ba đặc tính khiến nó đáng tin cậy như một cơ sở cho các quyết định hạ nguồn, chứ không chỉ là một phần mô tả phụ. Thứ nhất, nó được đo trực tiếp từ chính các file âm thanh (header wave + quét mẫu), không đọc từ metadata trong manifest, do đó nó không thể kế thừa một lỗi gán nhãn CSV theo cách mà một bản tóm tắt chỉ dựa trên metadata có thể mắc phải. Thứ hai, thời lượng và tần số lấy mẫu *hoàn toàn* không đổi (std 0.0) trên cả 16 lớp và 12 vị trí có nghĩa là không lớp hay vị trí nào cần tiền xử lý khác biệt (resampling, padding, cắt xén) trước khi trích xuất đặc trưng — một yếu tố gây nhiễu (confound) theo từng lớp mà nếu không sẽ phải bị loại trừ riêng cho mỗi thí nghiệm hạ nguồn. Thứ ba, số file bị clip

là zero cho *mọi* lớp và *mọi* vị trí ở đây, điều này đối chiếu chéo với phát hiện clipping toàn tập dữ liệu (Section 3.2) ở độ chi tiết mịn hơn: clipping hoàn toàn giới hạn ở thành phần *mixed* của `Mix.csv` và không tập trung vào bất kỳ lớp tim/phổi hay vị trí ghi âm cụ thể nào. Điều này quan trọng vì clipping tương quan với một lớp cụ thể sẽ có nguy cơ khiến một classifier học artifact clipping đó như một tín hiệu lớp giả mạo (spurious); ở đây rủi ro đó bị loại trừ bằng phép đo trực tiếp thay vì chỉ được giả định.

Tại sao đây là tin tốt, kèm một lưu ý. Việc không có bất kỳ khiếm khuyết chất lượng nào theo lớp hay theo vị trí có nghĩa là giai đoạn phân loại đã lên kế hoạch (PROTOCOL.md §5.3) có thể coi mọi bản ghi isolated là đầu vào đáng tin cậy như nhau — không có lớp nào có các bản ghi bị suy giảm chất lượng một cách bất cân xứng và cần bị loại trừ hoặc gán trọng số lại chỉ vì lý do chất lượng. Lưu ý ở đây là về *kích thước* lớp, không phải chất lượng: một số lớp tim rất nhỏ ($S4_{n=2}$, AV Block và Tachycardia $n=3$ mỗi lớp), điều mà Table 2 làm rõ ràng thay vì chỉ để lại như một ẩn tượng. Dưới cách chia 5-fold tôn trọng leak-group của Section 4, một lớp chỉ có 2 bản ghi không thể xuất hiện trong tập huấn luyện của mọi fold, điều này sẽ thể hiện ra như accuracy theo từng lớp không ổn định hoặc không xác định được cho những lớp đó ở giai đoạn phân loại — đáng để nêu ra ngay bây giờ, trước khi giai đoạn đó được xây dựng, thay vì phát hiện ra nó như một kết quả gây bối rối sau này (Section 13).

4 Tái Sử Dụng Bản Ghi và Vấn Đề Rò Rỉ (Leakage)

Một giao thức đánh giá ngây thơ (naïve) sẽ chia `Mix.csv` theo hàng (hoặc theo Mixed Sound ID) thành các fold train/test, và fit bất kỳ dictionary phân tách nào trên toàn bộ `HS.csv/LS.csv`. Cả hai lựa chọn đều làm rò rỉ ground truth vào huấn luyện, và lý do là đặc thù của cách HLS-CMDS được lắp ráp, chứ không phải một lưu ý chung chung.

4.1 Audit content-hash

`HS.csv` và `Mix.csv` dùng *các không gian ID khác nhau* cho cùng những file nền tảng (ví dụ `F_N_RC` của `HS.csv` so với `H0001` của `Mix.csv`), do đó danh tính file không thể được kiểm tra bằng cách so sánh chuỗi ID. Thay vào đó, `src/split.py` hash các mẫu PCM thô của mỗi file (bỏ qua header WAV, qua đọc PCM 16-bit + MD5) để tìm ra:

- 145 hàng của `Mix.csv` chỉ lấy từ **89** bản ghi tim khác biệt và **74** bản ghi phổi khác biệt (theo nội dung), mỗi bản ghi được tái sử dụng qua nhiều hàng mix — lên đến $6\times$ đối với một bản ghi tim duy nhất.
- **32** trong số 89 bản ghi tim đó, và **30** trong số 74 bản ghi phổi đó, giống hệt từng byte (byte-identical, theo nội dung PCM) với một bản ghi cũng được liệt kê độc lập trong `HS.csv/LS.csv` — chính xác là tập mà một dictionary phân tách sẽ được fit trên.

4.2 Leak group

Vì các bản ghi được tái sử dụng *giữa* các hàng mix một cách độc lập cho thành phần tim và phổi, đơn vị an toàn rò rỉ (leakage-safe) không phải là một hàng mix đơn lẻ mà là một *thành phần liên thông* (connected component): hai hàng được liên kết bất

cứ khi nào chúng chia sẻ một bản ghi tìm hoặc phổi (trực tiếp hoặc bắc cầu qua một hàng thứ ba). Union-find trên toàn bộ 145 hàng dựa trên quan hệ này cho ra **29** thành phần liên thông (“leak group”), có kích thước từ 1 đến 32 hàng (Table 4).

Table 4: Phân bố kích thước leak-group trên 145 hàng của `Mix.csv` (tổng cộng 29 nhóm; các kích thước cộng lại bằng 145). Một bản trước đó ghi 13 nhóm kích thước một, cộng lại chỉ được 144 hàng và 28 nhóm — một lỗi chép, đã sửa theo đầu ra của `split.py`.

Kích thước nhóm	32	22	19	13	11	9	4	3	2	1
Số nhóm có kích thước này	1	1	1	1	1	1	2	3	4	14

Hàm `assign_folds()` của `split.py` phân chia 29 nhóm này (không bao giờ chia một nhóm qua nhiều fold) thành k fold thông qua một phép gán tham lam (greedy) nhóm-lớn-nhất-vào-fold-nhỏ-nhất, tạo ra các fold cân bằng khá tốt bất chấp giá trị ngoại lai kích thước 32 (Table 5). Hàm `dictionary_pool()` sau đó trả về, đối với một fold được giữ lại (held-out) cho trước, chính xác các hàng `HS.csv/LS.csv` mà nội dung của chúng *không* xuất hiện ở bất cứ đâu trong các mixture đánh giá của fold đó — tập mà bất kỳ bước học dictionary hay fit mô hình nào phải bị giới hạn trong đó khi fold đó được giữ lại.

Table 5: Cách chia 5-fold (seed 0): số lượng hàng và kích thước tập dictionary-fitting kết quả.

Fold	Hàng eval (trên 145)	Pool HS (trên 50)	Pool LS (trên 50)
0	32	44	46
1	29	43	46
2	28	44	47
3	28	40	43
4	28	47	38

Cách chia này chi phối cả việc fit dictionary/mô hình lẫn việc đánh giá — không chỉ đánh giá đơn thuần — đây là thuộc tính mà `src/test_split.py` xác thực trực tiếp (Section 6.1.1).

5 Tính Hợp Lệ của Mixture: Audit Additive-Triplet

Audit content-hash của Section 4 trả lời câu hỏi “ground truth của một hàng mix có rò rỉ vào tập được dùng để fit một mô hình phân tách hay không.” Nó không nói gì về một câu hỏi logic đến trước đó: liệu bản ghi *mixed* được nêu tên trong một hàng `Mix.csv` có thực sự liên quan đến các bản ghi tìm và phổi được nêu tên cùng với nó hay không? `verify_mix_triplets()` của `load_dataset.py` chỉ kiểm tra rằng ba file tồn tại, được ghép cặp duy nhất, và khớp về định dạng (Section 3) — nó sẽ vẫn pass ngay cả khi *mixed* là một bản ghi hoàn toàn không liên quan bị gộp chung dưới cùng một hàng do lỗi manifest.

`verify_additive_triplets()` kiểm tra giả định mà mọi baseline phân tách và điểm BSS Eval trong pipeline này thực sự dựa vào: $\text{mixed} \approx a \cdot (\text{heart} + \text{lung})$ với một hệ số khuếch đại vô hướng a nào đó. Đối với mỗi hàng, nó fit giá trị a theo bình phương tối thiểu (least-squares) tối thiểu hóa $\|\text{mixed} - a(\text{heart} + \text{lung})\|^2$, sau đó báo cáo phần dư tương đối (relative residual) $\|\text{mixed} - a \cdot \text{summed}\| / \|\text{mixed}\|$. Thuộc đo này về mặt thực nghiệm là song đỉnh (bimodal) với một khoảng cách rộng: các hàng thực sự additive rơi vào phần dư $\sim 10^{-8}$ – 10^{-4} (bị giới hạn bởi lượng tử hóa 16-bit), các hàng không liên quan về mặt âm học rơi vào phần dư ~ 1 (không có năng lượng chung nào cả), và không có hàng nào rơi vào khoảng giữa — do đó ngưỡng chính xác (0.001) không phải là một lựa chọn tinh chỉnh nhạy cảm.

Kết quả: chỉ 36 trong số 145 hàng (24.8%) đạt. File `mixed` của 109 hàng còn lại về cơ bản không chia sẻ năng lượng nào với chính các bản ghi tim và phổi được gán tên của nó. Điều này tái hiện độc lập con số của *TEEP2026_Sprint0_Review* trên cùng bản sao tập dữ liệu, và (như đã lưu ý ở Section 3.2) 36 hàng hợp lệ chính xác là 36 hàng thể hiện dấu vân tay clipping một-mẫu-đơn — hai phép đo độc lập (phần dư spectral/waveform, và một artifact clipping của quá trình mixing) hội tụ về cùng một tập hàng tự nó là bằng chứng cho thấy phát hiện này là một thuộc tính thực sự của dữ liệu, không phải một lỗi (bug) ở một trong hai phép kiểm tra.

Điều này có hai hệ quả cho phần còn lại của báo cáo. Thứ nhất, đây là một vấn đề thứ hai, độc lập với leakage (Section 4): một hàng có thể an toàn rò rỉ (leakage-safe) nhưng vẫn vô nghĩa về mặt âm học, hoặc ngược lại. Thứ hai, vì 75% `Mix.csv` bị ảnh hưởng, mọi kết quả trong Section 7 được báo cáo *hai lần* — trên toàn bộ tập 145 hàng (để nhất quán với công trình trước đó và cách chia của Table 5) và trên riêng tập con 36 hàng hợp lệ (Table 6) — vì lấy trung bình các hàng không hợp lệ vào một con số chất lượng phân tách chủ đạo sẽ trộn lẫn “phương pháp đã thất bại trong việc phân tách” với “ngay từ đầu không có gì mạch lạc để phân tách cả.”

Table 6: Kết quả audit additive-triplet trên HLS-CMDS như đã công bố (Mendeley v3, nguồn gốc đã được xác minh ở Section 3.1).

	Hàng	% trên 145
Additive ($\text{mixed} \approx a \cdot (\text{heart} + \text{lung})$)	36	24.8%
Không additive (không liên quan về mặt âm học)	109	75.2%

Độ nhạy của phán quyết với độ nghiêm ngặt của mô hình. Phản đối tự nhiên của reviewer là một phép fit một-hệ-số, không-trễ sẽ bác bỏ một mixture vì những lý do tầm thường — lệch DC, đảo cực tính, vài mẫu lệch thời gian, hệ số khác nhau cho mỗi nguồn. Mỗi thứ đã được thử trên cả 145 hàng (`additive_audit_sensitivity.py`): trừ trung bình trước khi fit; cho phép hệ số âm (hệ số bình phương tối thiểu vốn không bị ràng buộc, và 52 hàng thực sự chọn hệ số âm — không hàng nào trong số đó đạt ở bất kỳ ngưỡng nào); dịch tham chiếu tổng theo độ trễ nguyên tối ưu theo tương quan chéo trong ± 100 mẫu; hệ số riêng cho tim và phổi; và hệ số riêng kèm độ trễ riêng theo từng nguồn. Dưới cả bảy mô hình, số hàng đạt ở 10^{-3} đều là 36, không hàng nào đổi phán quyết. Residual của cụm đạt trải từ 0.9 đến 4.5×10^{-4} và của cụm không đạt từ 0.94 đến 1.0 ngay cả dưới mô hình lỏng nhất — khoảng cách hơn $2000\times$ — nên ngưỡng 10^{-2} và 10^{-3} cho cùng 36 hàng, còn 10^{-4} chỉ nhận 3, vì riêng lượng tử hóa 16-bit đã đặt một sàn $\sim 10^{-4}$ cho bất kỳ hàng additive thật nào. Trong 36 hàng, độ trễ tối ưu là 0 ở

mọi hàng; trong 109 hàng, độ trễ tối ưu trải đều tới biên ± 100 mẫu, đúng như tương quan chéo hai tín hiệu không liên quan tạo ra. Mọi file đều đọc ở 4000 Hz. Vì thế 109 hàng không phải là những trường hợp suýt đạt mô hình additive dưới bất kỳ nói lỏng nào đã thử; chúng là các hàng mà mixture không chia sẻ năng lượng nào với hai bản ghi được gán tên cạnh nó.

6 Phương Pháp Luận

6.1 Các metric BSS Eval

Chất lượng phân tách được chấm điểm bằng phép phân rã BSS Eval kinh điển [2]: một nguồn ước lượng $\hat{s}_j$ được phân rã thành một thành phần mục tiêu (phiên bản lọc bất biến theo thời gian tốt nhất của nguồn thực s_j , độ dài bộ lọc lên đến 512 tap), một thành phần nhiễu xen (leakage từ các nguồn thực khác), và một thành phần artifact (méo dạng còn sót lại không được giải thích bởi cả hai thành phần trên. Từ phép phân rã này:

$$\text{SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}} + e_{\text{artif}}\|^2} \quad (\text{Signal-to-Distortion Ratio}) \quad (6)$$

$$\text{SIR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}}\|^2} \quad (\text{Source-to-Interference Ratio}) \quad (7)$$

$$\text{SAR} = 10 \log_{10} \frac{\|s_{\text{target}} + e_{\text{interf}}\|^2}{\|e_{\text{artif}}\|^2} \quad (\text{Sources-to-Artifacts Ratio}) \quad (8)$$

src/metrics.py bọc quanh mir_eval.separation.bss_eval_sources [3], được chuyển biệt hóa cho trường hợp 2 nguồn (tim, phổi) của tập dữ liệu này:

```
def evaluate_heart_lung(heart_ref, lung_ref, heart_est, lung_est,
                       compute_permutation=True) -> dict:
    """Returns {"heart": {"sdr":..., "sir":..., "sar":...},
               "lung": {"sdr":..., "sir":..., "sar":...}}"""
```

cùng với evaluate_mix_row()/evaluate_dataset(), các hàm nạp ground truth qua load_dataset.py và chạy một separate_fn(mixed, sr) → (heart_est, lung_est) do người gọi cung cấp trên một hoặc toàn bộ các hàng Mix.csv, và summarize_by_class() để tổng hợp theo lớp/theo vị trí. mir_eval 0.8 đã deprecate bss_eval_sources để chuyển sang một API kiểu museval chưa được công bố; vì chưa có bản thay thế hoạt động được, lệnh gọi đã deprecated này vẫn được dùng với FutureWarning của nó bị chặn cục bộ.

6.1.1 Xác thực (Validation)

src/test_metrics.py (10 test) xác thực wrapper theo hai cách:

- **Đối chiếu chéo trực tiếp với mir_eval** — trên các mảng tổng hợp (synthetic) và trên âm thanh thực của tập dữ liệu, đầu ra của wrapper được so sánh từng bit (bit-for-bit) với một lệnh gọi mir_eval thô trên cùng các mảng đã stack, bắt được các lỗi wrapper (thứ tự stacking, cắt xén, gán nhãn permutation) độc lập với việc bản thân metric nền tảng có đúng hay không.

- **Các mixture tổng hợp với ground truth dạng đóng (closed-form).** Các nguồn white-noise bị làm nhiễu theo những cách mà điểm BSS Eval đúng của chúng được biết trước một cách giải tích, và điểm đo được phải nằm trong dung sai:

Table 7: Xác thực bằng ground-truth tổng hợp (test_metrics.py).

Cách xây dựng	Mục tiêu lý thuyết	Dung sai	Kết quả
$\hat{s} = s$ (identity)	SDR/SIR/SAR $\rightarrow \infty$	> 100 dB	> 300 dB
$\hat{s} = s + n$, SNR 20 dB	SDR $\approx$ SAR ≈ 20 dB	± 1.0 dB	trong dung sai
$\hat{s} = s + n$, SNR 6 dB	SDR $\approx$ SAR ≈ 6 dB	± 1.0 dB	trong dung sai
$\hat{s}_j = s_j + \alpha s_k$, $\alpha=0.1$	SIR $\approx -20 \log_{10} \alpha = 20$ dB	± 1.0 dB	trong dung sai
$\hat{s}_j = s_j + \alpha s_k$, $\alpha=0.3$	SIR ≈ 10.46 dB	± 1.0 dB	trong dung sai

src/test_split.py (15 test) xác thực cách chia (split), với bài test mang tính then chốt là test_dictionary_pool_excludes_all_held_out_recordings: với mỗi fold, nó hash mọi bản ghi trong pool dictionary của fold đó và khẳng định (assert) độ *chống lặp bằng không* với bất kỳ bản ghi nào được dùng trong các mixture đánh giá được giữ lại của fold đó. Nó cũng kiểm tra các leak group không bao giờ bị chia qua nhiều fold, độ phủ/cân bằng của fold, khả năng tái lập của cách chia dưới một seed cố định, và rằng cross_validate (Section 6.2) thực sự trao cho mỗi fold pool bị giới hạn của nó thay vì toàn bộ tập dữ liệu không giới hạn. Toàn bộ 25 test trên cả hai file đều pass (~2 phút tổng cộng, chủ yếu do việc chạy BSS Eval thực lặp đi lặp lại trên các bản ghi thực dài 15 giây).

6.2 Evaluation harness

src/eval_harness.py gắn kết Sections 4 và 6.1 lại với nhau:

```
def cross_validate(fit_and_separate_fn, n_folds=5, seed=0):
    """For each fold: fit_and_separate_fn(hs_allowed, ls_allowed) -> separate_fn,
    evaluate separate_fn on the held-out fold via metrics.evaluate_dataset.
    Returns (results_df, fold_summary, cv_summary)."""
```

fit_and_separate_fn là phương pháp hoặc mô hình phân tách của người gọi; harness cung cấp cho mỗi fold pool dictionary bị giới hạn leakage của nó và xử lý việc lặp qua fold, chấm điểm theo từng hàng, và tổng hợp. Một tham số tùy chọn mix_df giới hạn đánh giá vào một tập con của Mix.csv do người gọi cung cấp (các fold khi đó được tính chỉ trong tập con đó) — được dùng trong Section 7 để chạy mọi baseline trên tập con 36 hàng hợp lệ của Section 5 mà không phải nhân đôi logic gán fold. Kết quả được tổng hợp theo ba cách: SDR/SIR/SAR trung bình theo từng (fold, nguồn); trung bình $\pm$ std *giữa* các fold theo từng nguồn (aggregate_across_folds, độ phân tán ở cấp fold); và trung bình, trung vị (median), và std gộp trực tiếp trên mọi hàng đã đánh giá theo từng nguồn (summarize_pooled, độ phân tán ở cấp hàng). Cách cuối cùng này được thêm vào sau một quan sát trong *TEEP2026_Sprint0_Review*: lấy trung bình trong nội bộ fold trước khi tính độ lệch chuẩn có thể làm giảm đáng kể độ phân tán thực sự giữa các hàng (một con số across-fold 6.46 ± 3.71 dB cho cùng dữ liệu trở thành 6.30 ± 39.38 dB theo từng hàng trong một so sánh mà đột rà soát đó thực hiện), do đó

hai cách này không thể thay thế cho nhau và một báo cáo cần nói rõ nó đang dùng cách nào. Cả hai đều được báo cáo trong Section 7.

6.3 Tập mixture tổng hợp (synthetic) và cách chia an toàn rò rỉ riêng của nó

Chỉ 36 trong số 145 hàng của `Mix.csv` là additive (Section 5), quá ít để làm nguồn cho một phép sweep SDR-so-với-độ-khó liên tục. `synthetic_mix.py` thay vào đó xây dựng một tập tổng hợp mang tính tổ hợp (combinatorial): mỗi một trong 50 bản ghi tim được ghép với mỗi một trong 50 bản ghi phổi, được mix theo $\text{mixed} = a \cdot (\text{heart} + \text{lung}) + \text{noise}$ — cùng dạng hàm mà `verify_additive_triplets()` (Section 5) đã cho thấy các hàng additive gốc tuân theo, không phải một công thức tổng hợp tùy tiện. Hệ số khuếch đại a được lấy mẫu log-uniform theo từng cặp nguồn từ khoảng đã được fit theo thực nghiệm trên chính các hệ số khuếch đại của 36 hàng additive gốc; nhiều Gauss cộng được scale theo một SNR mục tiêu được sweep qua $\text{SNR_SWEEP_DB} = \{-5, 15, 35\}$ dB (được cắt giảm từ 5 mức sau khi đo được rằng một sweep 5 mức trên tất cả các baseline sẽ chạy nhiều giờ) để tạo ra một trục độ khó có thể kiểm soát được, thay vì mức nhiễu 2%-RMS cố định của chính Han & Quan. `validate_against_native` xác nhận cách xây dựng này trung thực, không chỉ được tuyên bố là như vậy: tái tạo lại mỗi trong 36 hàng additive gốc từ hệ số khuếch đại đã fit riêng của chúng và nhiễu bằng không, sau đó chạy lại `verify_additive_triplets()` trên kết quả tổng hợp, tái tạo được cả 36 hàng ở cùng mức phần dư $\sim 10^{-4}$ mà các hàng gốc thực sự có.

Cách chia phải chuyển từ hàng mixture sang file nguồn. Cách chia ở cấp triplet của Section 4 gán các *leak group* — các thành phần liên thông của các hàng mix chia sẻ một bản ghi tim hoặc phổi — vào các fold, đây là đơn vị đúng khi mỗi bản ghi chỉ xuất hiện trong một vài trong số 145 mixture gốc. Việc ghép cặp tổ hợp đầy đủ phá vỡ giả định đó: một file tim giờ xuất hiện trong 50 mixture, do đó chỉ sau vài bước, mọi file tim đều liên kết bắc cầu với mọi file phổi thông qua một mixture chung nào đó, khiến toàn bộ tập tổng hợp suy biến thành một thành phần liên thông duy nhất, không còn cách chia nào để thực hiện. `assign_source_folds` giải quyết điều này bằng cách gán mỗi trong số 50 file tim và 50 file phổi *độc lập* vào một trong 5 fold; một cặp tổng hợp (h, l) chỉ được dùng cho đánh giá held-out dưới fold k khi $\text{fold}(h) = \text{fold}(l) = k$, và `dictionary_pool_synthetic` loại trừ *cả hai* nguồn của mọi cặp held-out khỏi pool dictionary-fitting của fold đó — cùng hợp đồng không-chồng-lấp (no-overlap contract) như `split.dictionary_pool` (Section 4), được suy diễn lại ở cấp file thay vì cấp leak-group. Đây không phải là một thay đổi mang tính hình thức: làm sai điều này (ví dụ quay lại một cách chia cấp hàng hay cấp leak-group không còn phân tách được gì nữa) sẽ để cho dictionary của một fold nhìn thấy một file cũng xuất hiện trong các cặp đánh giá held-out của chính fold đó, thổi phồng mọi con số SDR/SIR/SAR hạ nguồn mà không có test nào bắt được lỗi này. `test_synthetic_mix.py` biến điều này thành một bất biến (invariant) được kiểm tra thay vì một tuyên bố một lần: với mỗi fold, `test_dictionary_pool_excludes_both_sources_of_held_out_pairs` khẳng định độ chồng lấp bằng không giữa pool dictionary của fold đó và bất kỳ bản ghi nguồn nào của một cặp bị giữ lại trong fold đó, cùng với `test_only_same_fold_pairs_are_included` xác nhận mọi hàng tổng hợp thực sự thỏa mãn $\text{fold}(h) = \text{fold}(l)$.

6.4 Baseline 1: lọc thông dải (bandpass filtering)

Một phương pháp phân tách cố định, không thích nghi (non-adaptive) đóng vai trò làm mức sàn kiểm tra tính hợp lý (sanity floor) mà bất kỳ phương pháp đã học nào cũng nên vượt qua một cách thoải mái — nếu không, điều đó báo hiệu một lỗi (bug) trong phương pháp đã học chứ không phải một bài toán thực sự khó. Các ngưỡng cắt (cutoffs) được suy ra từ chính nội dung spectral của tập dữ liệu này thay vì giả định từ tài liệu âm thanh tổng quát: phương pháp Welch (`scipy.signal.welch`, các đoạn 1024 mẫu) được chạy trên mọi bản ghi isolated trong `HS.csv` và `LS.csv` một cách riêng biệt và lấy trung bình theo từng nguồn, cho thấy năng lượng tim chiếm ưu thế dưới ~ 150 – 200 Hz, năng lượng phổi chiếm ưu thế từ ~ 200 Hz lên đến ~ 700 – 800 Hz, và cả hai nguồn đều rơi vào mức sàn nhiễu (noise floor) của bản ghi trên ~ 1 kHz (Table 8 cho các giá trị tiêu biểu; không có biểu đồ PSD nào được đưa vào bản nháp này).

Table 8: Các giá trị PSD tiêu biểu, đã lấy trung bình và peak-normalized (dB) từ audit thực nghiệm.

Tần số	Heart (dB)	Lung (dB)	Nguồn chiếm ưu thế
93.8 Hz	−2.3	−12.1	Heart
187.5 Hz	−15.7	−24.6	Heart (vùng crossover)
500 Hz	−26.7	−14.4	Lung
1000 Hz	−50.4	−50.2	Không nguồn nào (noise floor)

Dựa trên điều này, Baseline 1 dùng một dải tim 20–200 Hz và một dải phổi 150–1000 Hz, được áp dụng như một bộ lọc thông dải Butterworth bậc 4, zero-phase (`scipy.signal.butter` dạng second-order-section, lọc bằng `sosfiltfilt` để tránh group-delay mà một bộ lọc IIR một lượt (single-pass) sẽ gây ra). Sự chồng lấp 150–200 Hz giữa hai dải là có chủ đích: audit PSD cho thấy cả hai nguồn đều mang năng lượng thực sự ở đó, và ép buộc một cách chia không chồng lấp sẽ mô tả sai vật lý nền tảng cũng như xóa bỏ đúng chế độ thất bại (failure mode) do chồng lấp spectral mà baseline này tồn tại để phơi bày. Bậc và loại bộ lọc *đã không* được tinh chỉnh (tuned) dựa trên dữ liệu đánh giá — làm như vậy sẽ tái hiện, ở quy mô nhỏ, chính vấn đề leakage mà Section 4 tồn tại để ngăn chặn, vì nó sẽ fit baseline “zero-training” vào đúng các mixture mà nó được chấm điểm trên đó.

6.5 Baseline 2: supervised NMF

Baseline 1 trả lời câu hỏi “pipeline có hợp lý không.” Nó không nói gì về việc liệu một phương pháp phân tách đã công bố thực sự có hoạt động trên tập dữ liệu này hay không, do đó `src/baselines.py` cũng triển khai một bản tái hiện an toàn rõ ràng của một phương pháp non-negative matrix factorization (NMF) có giám sát (supervised) từ tài liệu được xác định trong quá trình rà soát công trình liên quan ([7]; Section 11), theo công thức phân tách supervised-NMF kinh điển của Smaragdis (2007): hai dictionary cố định, một cho mỗi nguồn, được học một lần từ các bản ghi isolated qua NMF multiplicative-update dựa trên KL-divergence (Lee & Seung 2001), sau đó được đóng băng và dùng để giải các activation theo từng mixture đối với một mixture được giữ lại (held-out). Các siêu tham số (hyperparameters) (hạng dictionary $K_i=10$ tim / $K_r=20$

phối, 100 vòng lặp MU để fit mỗi dictionary, 60 vòng lặp MU để giải các activation) được cung cấp trong quá trình bàn giao phương pháp của dự án này như các giá trị gốc của bài báo; chúng chưa được đối chiếu chéo độc lập với nguồn gốc chính (Section 13). Phân tách là một soft mask kiểu Wiener được áp dụng lên STFT phức của mixture ($N_{\text{mf}}=512$, hop 128), được xây dựng từ phổ độ lớn (magnitude spectra) đã tái tạo qua NMF của hai nguồn.

Bẫy leakage. Một bản tái hiện ngây thơ sẽ fit cả hai dictionary trên “toàn bộ HS.csv/LS.csv.” Section 4 cho thấy điều này để cho một phần ba số bản ghi đó — một số trong đó là chính ground truth của một mixture được giữ lại — lọt vào dictionary mà mixture đó sau này được chấm điểm dựa trên. Thay vào đó, `make_supervised_nmf_baseline()` chỉ nhận `pool_hs_allowed/ls_allowed` an toàn rõ ràng của mỗi fold từ `eval_harness.cross_validate` (Section 6.2) — cùng giới hạn mà Baseline 1 phải chịu, dù bandpass không có dictionary nào để rõ ràng vào. Do đó, một điểm số an toàn rõ ràng cho phương pháp này *được kỳ vọng* sẽ thấp hơn so với một bản tái hiện ngây thơ; con số thấp hơn đó, chứ không phải con số cao hơn, mới là kết quả đúng về mặt phương pháp luận (Section 7).

6.6 Baseline 3: standard NMF (dictionary ablation)

Baseline 2 gộp lẫn hai hiệu ứng: bản thân phép factorization NMF, và dictionary đã được huấn luyện trước/đóng băng cụ thể mà nó factorize dựa trên. Baseline 3 tách riêng hiệu ứng thứ hai bằng cách giữ cố định mọi giai đoạn khác của pipeline Baseline 2 (cùng tổng hạng $K_i+K_r=30$, cùng tham số STFT, cùng bộ lọc khử nhiễu Butterworth tiến xử lý) trong khi factorize W và H trực tiếp từ spectrogram riêng của mỗi mixture được giữ lại, hoàn toàn không có giai đoạn học dictionary từ các bản ghi isolated — do đó, giống Baseline 1, nó không bao giờ đụng đến `hs_allowed/ls_allowed` và không có gì để rõ ràng. Các thành phần thu được không có nhãn theo cấu trúc; chúng được gán cho tim hoặc phổi sau đó (post-hoc) bằng spectral centroid dựa trên khảo sát PSD của chính Baseline 1 (năng lượng tim tập trung dưới ~ 200 Hz), một tiên nghiệm (prior) vật lý cố định chứ không phải một nhãn nguồn, do đó phương pháp vẫn thực sự blind. Phép so sánh mà điều này mang lại là trực tiếp: bất kỳ khoảng cách nào giữa Baseline 2 và Baseline 3 đều có thể quy cho cụ thể dictionary đã học, đóng băng, chứ không phải cho phép factorization NMF hay tiền xử lý chung.

6.7 Baseline 4: multi-stage Singular Spectrum Analysis (MSSA)

Baseline 4 tái hiện phương pháp MSSA hai giai đoạn của Han & Quan [9], cùng bài báo mà Table I của chính nó được báo cáo cạnh kết quả của dự án này trong Section 7 để so sánh trực tiếp. Cả bốn siêu tham số được lấy đúng như đã nêu trong bài báo thay vì được tinh chỉnh: độ dài cửa sổ $L=50$, ngưỡng chia tim mạch/hô hấp ở 250 Hz (một tần số sinh lý theo Hz tuyệt đối, hợp lệ ở tần số lấy mẫu 4000 Hz của tập dữ liệu này vì Nyquist, 2000 Hz, vượt qua nó với biên độ dư), một ngưỡng đóng góp giá trị riêng (eigenvalue) là $100/L = 2\%$, và một ngưỡng tương quan 50%. Giai đoạn 1 phân rã SSA trên mixture thô thành 50 reconstructed component (RC), định tuyến mỗi RC theo tần số trội Welch-PSD của nó vào hoặc là đầu ra tim mạch trực tiếp (≤ 250 Hz) hoặc một pool phần dư đưa vào Giai đoạn 2; Giai đoạn 2 phân rã lại phần dư đó và chọn các thành phần hô hấp theo đóng góp eigenvalue, sau đó theo tương quan Pearson với tập đã được chọn. Giống Baseline 1, MSSA là zero-training — `hs_allowed/ls_allowed`

không được dùng đến và không có bất kỳ leakage nào áp dụng, vì hoàn toàn không có bước fit nào cả.

6.8 Baseline 5: EVMD (Enhanced Variational Mode Decomposition)

Baseline 5 tái hiện giai đoạn tách âm phổi (lung-isolation) dựa trên EVMD của Puneet et al. [8], bài báo công trình liên quan duy nhất trong Table 27 mà bước phân tách của nó không báo cáo bất kỳ metric chất lượng phân tách nào cả dù chạy trực tiếp trên các mixture của HLS-CMDS — Table 11 bên dưới, theo hiểu biết của dự án này, là lần đầu tiên SDR/SIR/SAR được tính cho phương pháp đó trên tập dữ liệu này. Bản thân Variational Mode Decomposition (Dragomiretzkiy & Zosso, 2014) được triển khai từ đầu (không có gói VMD nào được duy trì sẵn có), và phép sweep chọn K của chính bài báo ($K=2..10$ tại $\alpha=2000$, dừng lại một khi cả tiêu chí mất năng lượng lẫn một kiểm tra theo từng mode đều đạt) được tuân theo đúng như đã nêu. Kiểm tra bốn tiêu chí theo từng mode mà bài báo mô tả (chữ ký miễn tần số, tỉ lệ NPE, và một phương án dự phòng Kurtosis-Index) không được đặc tả đầy đủ trong nguồn gốc; cách đọc nỗ lực tốt nhất của dự án này về việc các tiêu chí kết hợp với nhau ra sao được ghi lại trực tiếp trong `code_description.md` thay vì được trình bày như một bản tái hiện nguyên văn. Giống Baseline 1 và 4, EVMD là zero-training. Toàn bộ sweep K tốn ~ 15 s/mixture, chủ yếu do các vòng lặp ADMM của VMD; do đó cột nền tổng hợp (synthetic-substrate) của Section 7 cho baseline này chạy trên một mẫu con phân tầng (stratified subsample) 75 hàng thay vì toàn bộ tập tổng hợp 1500 hàng, một sự thu hẹp phạm vi do compute được công khai chứ không phải âm thầm.

6.9 Baseline 6: Conv-TasNet-lite

Baseline 1-5 hoặc là zero-training hoặc là một fit dictionary cố định. Baseline 6 (`convtasnet.py`) là baseline phân tách neural, end-to-end được học đầu tiên của dự án này, và là nơi đầu tiên một vòng lặp huấn luyện với các epoch và một optimizer xuất hiện trong codebase này. Nó dùng chính thiết kế encoder/TCN-separator/decoder của Conv-TasNet [4], được thu nhỏ (“lite”, ~ 325 K tham số so với ~ 5 M của Conv-TasNet gốc) cho pool huấn luyện nhỏ hơn nhiều của tập dữ liệu này, với một sigmoid mask cho mỗi nguồn (không có ràng buộc tổng-bằng-một, theo đúng phát hiện ablation của chính Conv-TasNet rằng sigmoid hoạt động tốt ít nhất bằng một ràng buộc softmax) và một thứ tự nguồn cố định thay vì permutation-invariant training, vì tim và phổi ở đây là các lớp có thể phân biệt được chứ không phải các người nói có thể hoán đổi cho nhau.

Tần số lấy mẫu. Tập dữ liệu này vốn dĩ là 4000 Hz; các checkpoint Conv-TasNet/Sepformer công khai được huấn luyện ở 8-16 kHz và không áp dụng được nếu không resample, và việc resample lên không tạo ra thông tin thực nào. Thay vào đó, mô hình được huấn luyện từ đầu ở 4000 Hz gốc — không phải một hạn chế về độ phân giải đối với bài toán này, vì năng lượng tim (20-200 Hz) và phổi (100-1000 Hz) (theo chính khảo sát PSD của Section 6.4) nằm thoải mái dưới ngưỡng Nyquist 2000 Hz của 4000 Hz.

Vòng lặp huấn luyện. Một mạng mới được huấn luyện cho mỗi fold từ pool `hs_allowed/ls_allowed` an toàn rõ ràng của fold đó — cùng hợp đồng mà `fit_and_separate_fn` của mọi baseline khác nhận được (Section 6.2). Vì Conv-TasNet cần các ví dụ huấn luyện theo cặp (mixed, heart, lung) thay vì một dictionary spectral, các mixture huấn

luyện được tổng hợp ngay tại chỗ (on the fly) bằng chính công thức mixing của Section 6.3, do đó các mixture ở thời điểm huấn luyện được lấy từ cùng phân phối mà mô hình được đánh giá dựa trên. Một cách chia 80/20 ở cấp file bên trong pool được phép (không bao giờ đụng đến dữ liệu held-out của fold ngoài) cho ra một tập validation nội bộ để dừng sớm (early stopping) và lập lịch learning-rate (AdamW, LR giảm một nửa sau 4 epoch không cải thiện validation-SI-SDR, khôi phục checkpoint tốt nhất).

Phạm vi trung thực. Đây là baseline neural đầu tiên, thiết lập hạ tầng vòng lặp huấn luyện — không phải một mô hình đã được tinh chỉnh, hội tụ. Con số ~ 50 bản ghi/lớp của tập dữ liệu (ít hơn theo từng fold một khi các loại trừ leakage được áp dụng) là nhỏ đối với một bộ phân tách neural huấn luyện từ đầu ngay cả khi có augmentation mixing tại chỗ. Thay vì giả định rằng cấu hình được đóng gói sẵn (MAX_EPOCHS=25, STEPS_PER_EPOCH=40, BATCH_SIZE=8) đã huấn luyện đến hội tụ, baseline6_report.py báo cáo trực tiếp các chẩn đoán huấn luyện thực tế và wall-clock đo được của mỗi fold (Table 12).

7 Kết Quả

Theo Section 5, mỗi baseline được báo cáo trên hai tập hàng: toàn bộ 145 hàng của Mix.csv (Table 9, để nhất quán với các kích thước fold trong Table 5) và riêng tập con 36 hàng additive-valid (Table 10). Mọi con số đều là kết quả 5-fold cross-validated (eval_harness.cross_validate, seed 0), được báo cáo dưới dạng trung bình $\pm$ std giữa các fold (độ phân tán cấp fold, Section 6.2); Table 10 bổ sung thêm trung bình gộp cấp hàng $\pm$ CI 95% cho tập con hợp lệ cạnh trung bình fold, vì hai quy ước khác nhau và cả hai đều xuất hiện trong báo cáo này.

7.1 Toàn bộ tập 145 hàng

Table 9 báo cáo cả hai baseline cạnh một tham chiếu không-phân-tách (no-separation) (tín hiệu mixed thô được trả về nguyên vẹn như cả ước lượng tim lẫn phổi, tức là những gì một classifier sẽ thấy với hoàn toàn không có bước phân tách nào).

Table 9: Chất lượng phân tách 5-fold cross-validated trên toàn bộ tập 145 hàng: cả hai baseline so với không phân tách.

Phương pháp	Nguồn	SDR (dB)	SIR (dB)	SAR (dB)
Không phân tách	Heart	-13.68 ± 1.26	1.71 ± 0.93	6.46 ± 3.71
Không phân tách	Lung	-15.89 ± 0.84	-1.55 ± 0.91	6.46 ± 3.71
Baseline 1 (bandpass)	Heart	-12.65 ± 1.40	4.07 ± 1.16	-8.10 ± 1.44
Baseline 1 (bandpass)	Lung	-13.83 ± 1.12	1.73 ± 0.29	-9.27 ± 1.42
Baseline 2 (supervised NMF)	Heart	-12.82 ± 1.31	4.22 ± 1.63	-9.58 ± 0.90
Baseline 2 (supervised NMF)	Lung	-15.09 ± 0.98	0.32 ± 0.58	-9.23 ± 1.42

(SAR giống hệt nhau cho tim và phổi dưới điều kiện “không phân tách” vì khi không có xử lý nào được áp dụng, không có gì để phân biệt artifact của nguồn này với nguồn kia.)

Lọc bandpass cải thiện SIR cho cả hai nguồn (+2.4 dB tim, +3.3 dB phổi) — nó thực sự có phân tách trên cơ sở tần số — nhưng SAR sụp đổ khoảng 14–16 dB so với việc không làm gì cả, phù hợp với việc filter ringing và các họa âm bị phá hủy chiếm ưu thế trong sai số còn lại. SDR, vốn gộp nhiễu xen và artifact lại với nhau, do đó chỉ mang lại một mức cải thiện khiêm tốn $\sim 1\text{--}2$ dB so với tham chiếu không-phân-tách. Điều này khớp với đánh đổi lý thuyết giữa nhiễu-xen/artifact được thảo luận ở Section 11.2 và chính xác là kết quả mà một baseline sanity-floor bandpass được kỳ vọng tạo ra: phân tách thực sự nhưng có giới hạn, đánh đổi bằng một chi phí artifact thực sự. Baseline 2 (supervised NMF an toàn rò rỉ) về cơ bản hòa với Baseline 1 trên SDR tim (trong phạm vi một fold-std) và kém hơn ~ 1.3 dB trên SDR phổi — một dictionary đã học, đặc thù theo nguồn chỉ vừa đủ ngang bằng, và trên phổi còn thua, một bộ lọc không có tham số nào được học cả.

Nhưng 109 trong số 145 hàng này là không additive theo phép kiểm tra tham chiếu được gán tên (Section 5): đối với những hàng đó không có cấu trúc tim+phổi thực sự nào trong mixed để bất kỳ phương pháp nào khôi phục, do đó điểm số của một phương pháp trên chúng đo lường cách nó phản ứng với âm thanh về cơ bản tùy tiện, không phải chất lượng phân tách. Table 9 được giữ lại để nhất quán với cách chia được báo cáo trong Table 5, nhưng tập con 36 hàng bên dưới mới là con số mà báo cáo này coi là có ý nghĩa.

7.2 Tập con 36 hàng additive-valid

Mọi con số trong Table 10 được tính từ một file duy nhất, `results/canonical_rows.csv` (`canonical_rows.py`): điểm BSS Eval của các dạng sóng đã phân tách trong cache của lượt chạy S6-02 (`results/sdr_sweep_cache/`, PCM 16-bit, 2026-08-28), cũng chính là âm thanh mà Condition B và mọi đường knee-point tiêu thụ. Hai cách tóm tắt của cùng 36 hàng được đặt cạnh nhau vì cả hai đều được dùng trong báo cáo này và chúng khác nhau: trung bình giữa các fold của trung bình từng fold $\pm$ std trên năm fold (8/7/7/7/7 hàng), và trung bình gộp trên 36 hàng $\pm$ khoảng t 95% trên hàng. Một bản trước đó in SDR trung-bình-fold ở đây và SDR gộp trong bảng Condition B (5.61 so với 5.46 dB cho bandpass) mà không nói rõ; nó cũng mang một mục Baseline 2 (3.78/−0.44 dB) từ một lượt chạy trước các hiệu chỉnh theo nguồn chính của Section 6.5, và một mục Baseline 6 (5.15 dB) từ một lượt huấn luyện riêng cùng seed, vốn trên GPU không tái lập được từng bit (5.02 dB trong lượt được cache ở đây). Cả ba đều bị thay thế bởi bảng này.

Table 10: Chất lượng phân tách trên tập con 36 hàng additive-valid, 5-fold an toàn rò rỉ, từ `canonical_rows.csv`. SDR được cho theo cả hai quy ước; SIR và SAR là trung bình gộp. Baseline 0 chấm mixture thô như cả hai ước lượng.

Phương pháp	Nguồn	SDR (dB)		SIR (dB)	SAR (dB)
		TB fold $\pm$ std	gộp $\pm$ CI 95%		
Baseline 0 (mixture thô)	Tim	2.65 ± 4.34	2.48 ± 2.79	2.48	74.6
Baseline 0 (mixture thô)	Phối	-2.14 ± 4.19	-1.98 ± 2.66	-1.98	74.6
Baseline 1 (bandpass)	Tim	5.61 ± 3.68	5.46 ± 2.66	6.30	17.47
Baseline 1 (bandpass)	Phối	3.94 ± 3.57	4.05 ± 2.37	5.73	12.42
Baseline 2 (supervised NMF)	Tim	2.69 ± 2.96	2.58 ± 1.87	7.27	7.58
Baseline 2 (supervised NMF)	Phối	0.62 ± 3.14	0.71 ± 2.34	2.36	9.93
Baseline 3 (standard NMF)	Tim	3.44 ± 2.02	3.35 ± 1.88	6.97	9.37
Baseline 3 (standard NMF)	Phối	2.22 ± 3.91	2.31 ± 2.81	5.02	9.66
Baseline 4 (MSSA)	Tim	5.17 ± 4.05	5.02 ± 2.91	5.73	18.81
Baseline 4 (MSSA)	Phối	5.32 ± 3.20	5.39 ± 1.87	8.58	12.18
Baseline 5 (EVMD)	Tim	3.73 ± 3.43	3.61 ± 2.35	4.99	13.99
Baseline 5 (EVMD)	Phối	0.36 ± 3.39	0.50 ± 2.44	1.40	13.56
Baseline 6 (Conv-TasNet-lite)	Tim	5.19 ± 4.14	5.02 ± 2.77	6.28	16.45
Baseline 6 (Conv-TasNet-lite)	Phối	2.05 ± 3.20	2.13 ± 2.21	4.77	9.07

Hai điều nổi bật trên tập con hợp lệ. **Thứ nhất, chất lượng phân tách tuyệt đối cao hơn nhiều so với những gì Table 9 gợi ý:** SDR tăng vọt 15–18 dB cho cả hai phương pháp và cả hai nguồn một khi các hàng không có cấu trúc mixture thực sự bị loại trừ, xác nhận rằng các con số trên toàn bộ 145 hàng bị chi phối bởi các hàng mà không có gì mạch lạc để phân tách, chứ không phải bởi một bài toán phân tách khó một cách đồng đều. **Thứ hai, và có hệ quả lớn hơn, thứ hạng giữa hai baseline đảo ngược so với Table 9:** trên tập con hợp lệ, bandpass zero-training vượt qua supervised NMF an toàn rò rỉ khoảng ~ 2.9 dB SDR trên tim và ~ 3.3 dB SDR trên phối (trung bình fold), một khoảng cách lớn hơn đáng kể và nghiêng hẳn về một phía hơn so với thể gần-hòa quan sát được trên toàn bộ tập hàng (phần lớn không hợp lệ). Một phương pháp phân tách đã học, đặc thù theo nguồn không vượt qua được một bộ lọc cố định không có tham số nào được học, đúng trên những hàng mà việc phân tách thực sự khả thi, không phải là kết quả mà một cách đọc ngây thơ bài báo được tái hiện sẽ dự đoán, và được thảo luận thêm ở Section 12.

7.3 Sáu phương pháp, ba nền dữ liệu

Sections 6.6–6.9 thêm bốn baseline nữa, và Section 6.3 thêm tập tổng hợp 1500 hàng làm nền đánh giá chính của dự án này từ đây trở đi, với tập con 36 hàng native-additive được giữ lại song song như một cột riêng biệt thay vì gộp vào — sự đồng thuận hay bất đồng giữa hai nền dữ liệu tự nó là bằng chứng audit, không phải nhiều để lấy trung bình xóa đi. Mọi con số bên dưới là trung bình $\pm$ 95% CI; cột synthetic của Baseline 5 chạy trên một mẫu con phân tầng 75 hàng (Section 6.8), không phải toàn bộ 1500.

Table 11: Nền tổng hợp (synthetic, chính), 5-fold, $n=1500$ ngoại trừ Baseline 5 ($n=75$).

Phương pháp	Nguồn	SDR (dB)	SIR (dB)	SAR (dB)
Baseline 1 (bandpass)	Heart	2.10 ± 0.43	6.17 ± 0.47	11.01 ± 0.45
Baseline 1 (bandpass)	Lung	-1.24 ± 0.44	4.25 ± 0.40	5.92 ± 0.48
Baseline 2 (supervised NMF)	Heart	-0.70 ± 0.40	6.54 ± 0.44	3.47 ± 0.34
Baseline 2 (supervised NMF)	Lung	-3.73 ± 0.40	1.39 ± 0.39	3.59 ± 0.38
Baseline 3 (standard NMF)	Heart	-0.00 ± 0.43	6.93 ± 0.50	4.84 ± 0.37
Baseline 3 (standard NMF)	Lung	-3.53 ± 0.46	4.06 ± 0.39	1.39 ± 0.46
Baseline 4 (MSSA)	Heart	1.59 ± 0.47	5.58 ± 0.47	11.48 ± 0.52
Baseline 4 (MSSA)	Lung	-1.60 ± 0.50	6.24 ± 0.42	3.74 ± 0.55
Baseline 5 (EVMD)	Heart	2.42 ± 1.70	6.20 ± 2.02	10.15 ± 1.48
Baseline 5 (EVMD)	Lung	-2.99 ± 2.25	2.81 ± 1.90	5.63 ± 2.53
Baseline 6 (Conv-TasNet-lite)	Heart	3.16 ± 0.40	6.47 ± 0.51	12.14 ± 0.31
Baseline 6 (Conv-TasNet-lite)	Lung	0.37 ± 0.35	4.29 ± 0.41	6.96 ± 0.27

Cột native-additive của phép so sánh này là Table 10 ở trên (quy ước gộp của nó khớp với bảng tổng hợp).

Sự đồng thuận giữa các nền dữ liệu. Mọi baseline đều đạt điểm thấp hơn trên nền tổng hợp so với trên tập native 36 hàng (khoảng cách tim -1.2 đến -3.4 dB, khoảng cách phổi -1.8 đến -7.0 dB, tính bằng trung bình gộp synthetic trừ trung bình gộp native), nhất quán theo cùng một chiều trên cả sáu phương pháp. n lớn hơn nhiều của tập tổng hợp khiến CI của chính nó chặt bất kể khoảng cách này, do đó một CI cột synthetic chặt tự nó không phải là bằng chứng cho thấy hai nền dữ liệu đồng thuận với nhau — cột khoảng cách mới là bằng chứng, và nó cho thấy tập tổng hợp khó hơn, không phải rằng một trong hai nền dữ liệu không đáng tin cậy. Khoảng cách phía phổi nhất quán lớn hơn khoảng cách phía tim đối với mọi phương pháp, điều này khớp với chính khảo sát PSD của Baseline 1 (Section 6.4): năng lượng phổi trải rộng hơn và gần noise floor hơn, do đó nó nhạy cảm hơn với phép sweep nhiễu của tập tổng hợp so với năng lượng tim.

Baseline 6 vẫn chưa rõ ràng chiến thắng. Conv-TasNet-lite đạt điểm cao nhất trong sáu phương pháp trên SDR tim và phổi của tập synthetic, và cao nhất trên SDR tim của native-additive, nhưng trên SDR phổi của native-additive nó tụt lại sau Baseline 4 (MSSA, 5.39 so với 2.14 dB) nhiều hơn nửa độ rộng CI của cả hai phương pháp. Một mô hình đã học, end-to-end chỉ thỉnh thoảng vượt qua một phương pháp zero-training là kết quả dự kiến của một baseline neural đầu tiên được huấn luyện có chủ đích chưa đủ (undertrained) (Section 6.9), không phải bằng chứng rằng kiến trúc không phù hợp với bài toán này; Table 12 bên dưới là thứ quyết định liệu “chưa được huấn luyện đủ” có thực sự là lời giải thích đúng hay không.

Table 12: Chẩn đoán huấn luyện Baseline 6, các fold của tập synthetic (max_epochs=25, steps_per_epoch=40, batch_size=8).

Fold	Số epoch đã huấn luyện	SI-SDR val. tốt nhất (dB)	Thiết bị	Thời gian fit (s)
0	25	0.78	cuda	27.1
1	25	1.78	cuda	26.2
2	25	2.65	cuda	26.2
3	25	3.35	cuda	26.3
4	25	1.68	cuda	26.2

Mọi fold đều huấn luyện đủ ngân sách 25 epoch trọn vẹn mà tiêu chí giảm một nửa LR (4 epoch không cải thiện validation-SI-SDR) không bao giờ kích hoạt — best-validation-SI-SDR vẫn đang cải thiện, hoặc tệ nhất là chững lại (plateau) trong thời gian ngắn, khi huấn luyện dừng lại, điều này đọc như bị giới hạn bởi ngân sách compute chứ không phải đã hội tụ. Kết hợp với chỉ ~ 32 bản ghi huấn luyện mỗi lớp mỗi fold (kích thước pool ở quy mô Table 5, được chia tiếp 80/20 cho validation nội bộ), điều này ủng hộ khung “phạm vi trung thực” của chính Section 6.9: Baseline 6 xác lập rằng hạ tầng vòng lặp huấn luyện hoạt động end-to-end (Table 11 cho thấy đầu ra hữu hạn, không suy biến), chứ không phải rằng kiến trúc này đã được cho một cơ hội công bằng để chứng minh giới hạn trên (ceiling) của nó trên tập dữ liệu này.

8 Phân Loại Hạ Nguồn: Việc Phân Tách Có Làm Giảm Độ Chính Xác Không?

Mọi phần cho đến nay đều đo *chất lượng phân tách* (SDR/SIR/SAR). Chưa phần nào trong số đó trả lời câu hỏi mà toàn bộ dự án tồn tại để trả lời (Section 11.1): việc phân tách âm tim khỏi một mixture, trước khi phân loại nó, có thực sự tổn kém về độ chính xác hay không, và nếu có, tổn kém bao nhiêu, ở mức chất lượng phân tách nào? Trả lời điều đó cần một classifier, một cách để so sánh độ chính xác của nó trên âm thanh sạch so với âm thanh đã phân tách dưới một cách chia được kiểm soát, và — vì chất lượng phân tách là liên tục, không phải một con số đơn lẻ — một cách để sweep chất lượng phân tách và vẽ ra một đường cong thay vì so sánh hai điểm cố định.

8.1 Tại sao chỉ một kiến trúc classifier, một cách có chủ đích, lúc đầu

Classifier trong phần này được mô tả trong project charter như một *công cụ đo lường* (measuring instrument), không phải một đóng góp nghiên cứu tự thân: nhiệm vụ duy nhất của nó là tiết lộ liệu chất lượng phân tách và độ chính xác có liên quan với nhau hay không, chứ không phải để trở thành classifier âm tim tốt nhất có thể. Khung tư duy đó là lý do chính xác một kiến trúc được xây dựng trước (Section 8.9 xem lại điều này một khi kết quả của chính classifier hóa ra quan trọng hơn dự kiến ban đầu) — một kiến trúc thứ hai chỉ trở nên cần thiết một khi một độc giả có thể hợp lý đặt câu hỏi “kết quả này là về phân tách, hay về đúng cái classifier bạn tình cờ chọn,” đây là một câu hỏi về *công cụ đo lường*, không phải về nghiên cứu phân loại vì lợi ích của chính nó.

8.2 Quyết định gộp lớp (class-grouping)

HLS-CMDS gán nhãn mỗi bản ghi âm tim isolated với một trong mười loại cụ thể (Table 2). Phân loại trực tiếp cả mười loại là không khả thi dưới cách chia leak-group của dự án này: S4 chỉ có tổng cộng 2 bản ghi, và dưới một cách chia 5-fold được xây từ leak group (Section 4) không có gì đảm bảo 2 bản ghi đó rơi vào các fold khác nhau — nếu chúng rơi vào cùng một fold, ba trong số năm fold sẽ huấn luyện với *zero* ví dụ S4, và AV Block cùng Tachycardia ($n=3$ mỗi lớp) cũng chỉ khá hơn một chút. Đây chính xác là rủi ro lớp nhỏ mà Section 3.2.1 đã cảnh báo trước khi phần này được xây dựng.

Hai cách khắc phục đã được xem xét: gộp về phân loại nhị phân Normal/Abnormal, hoặc gộp mười loại thành một số nhỏ các danh mục có ý nghĩa lâm sàng. Các danh mục được chọn — Normal, Murmur, Extra Sound, Rhythm Disorder — không phải là một thỏa hiệp tùy tiện: chúng là *chính xác* sơ đồ bốn lớp mà Yaqub et al. (Section 11.1) đã dùng cho cùng tập dữ liệu này, được xác nhận bằng cách đọc trực tiếp nguồn chính của họ (Tables 6–9 của họ), và tái hiện đúng danh sách lớp mà sự sụp đổ 89%→41% của chính họ được đo trên đó. Việc ánh xạ mười loại của HLS-CMDS lên bốn loại đó là không mơ hồ theo thuật ngữ tim mạch học tiêu chuẩn: Normal vẫn là Normal ($n=9$); bốn phân nhóm murmur (Mid/Late Systolic, Late Diastolic, Early Systolic) trở thành Murmur ($n=24$); S3 và S4 — cả hai đều là các âm tim phụ xảy ra ngoài cặp S1/S2 bình thường — trở thành Extra Sound ($n=7$); và Atrial Fibrillation, Tachycardia, và AV Block — tất cả đều là các rối loạn về nhịp điện học hoặc dẫn truyền của tim, đối lập với van tim của nó — trở thành Rhythm Disorder ($n=10$). Điều này giải quyết vấn đề lớp nhỏ (nhóm nhỏ nhất, Extra Sound với $n=7$, đủ lớn một cách thoải mái để xuất hiện trong mọi fold) và, khác với việc gộp về nhị phân Normal/Abnormal, giữ cho các con số độ chính xác của dự án này có thể so sánh trực tiếp với các con số 86–89% mà Yaqub et al. báo cáo trên chính sơ đồ mà kết quả gọi mở của dự án này được neo vào.

8.3 Kiến trúc 1: thống kê tóm tắt MFCC + SVM kernel RBF

Với quyết định gộp lớp ở trên và khung “công cụ đo lường, không phải một đóng góp” của Section 8.1, bản thân classifier được chọn sao cho nhỏ, được hiểu rõ, và ít khả năng overfit trên một tập dữ liệu kích thước này (tổng cộng 50 bản ghi, ít nhất là ~35–40 trong tập huấn luyện của một fold): 13 Mel-Frequency Cepstral Coefficients (MFCC, một phép nén spectrogram tiêu chuẩn, có động lực về mặt cảm nhận thính giác thành một số nhỏ hệ số mỗi khung thời gian, được dùng rộng rãi trong phân loại giọng nói và âm thanh trước thời deep learning), được pool thành một vector đặc trưng 26 chiều duy nhất cho mỗi bản ghi bằng cách lấy trung bình và độ lệch chuẩn của mỗi hệ số theo thời gian, đưa vào một Support Vector Machine (SVM) với kernel Radial Basis Function (RBF) và trọng số loss cân bằng theo lớp (class-balanced) (để lớp Murmur đa số không đơn giản chiếm ưu thế trong tối ưu hóa chỉ vì số lượng). Một SVM với một số lượng support vector cố định, vừa phải, ít có xu hướng ghi nhớ (memorize) một tập huấn luyện nhỏ như thế này hơn nhiều so với một mô hình deep được huấn luyện bằng gradient, đây chính xác là đánh đổi mà Section 8.9 kiểm tra trực tiếp bằng một kiến trúc thứ hai thay vì giả định.

Việc đánh giá tuân theo cùng các leak-group fold mà mọi baseline phân tách sử dụng (`assign_hs_folds()`, mở rộng phép gán fold hiện có của `split.py` từ các hàng mixture sang chính các bản ghi `HS.csv` isolated), phân tầng (stratified) theo nhóm lớp

đối với bất kỳ bản ghi nào chưa từng được tái sử dụng bên trong một mixture, do đó không fold nào bị thiếu hụt một lớp thiểu số. Table 13 báo cáo kết quả 5-fold cross-validated thu được — con số accuracy đầu tiên trong báo cáo này; mọi kết quả trước đó là một con số SDR/SIR/SAR theo thang dB.

Table 13: Condition A (Kiến trúc 1, MFCC+SVM): phân loại âm tim isolated trên 50 bản ghi HS.csv, 5-fold cross-validated. Không cùng tập đánh giá với hàng “isolated” của Condition B (Table 14), hàng đó chấm cùng các classifier theo fold trên chính các tham chiếu tim của 36 mixture native-additive; hai accuracy (60.0% so với 52.8%) khác nhau là theo thiết kế.

Metric	Trung bình $\pm$ 95% CI ($n=5$ fold)
Accuracy	60.0% $\pm$ 10.7%
Macro-F1	0.43 $\pm$ 0.12

Với chỉ 5 fold, khoảng tin cậy này nên được đọc như mang tính chỉ báo (indicative), không phải quyết định (decisive) — cùng lưu ý mà Section 6.2 đã áp dụng cho độ phân tán giữa các fold của mọi baseline phân tách — nó cho biết đại khái độ chính xác nằm ở đâu, không phải rằng một lần chạy lặp lại sẽ rơi vào trong phạm vi ± 0.1 điểm phần trăm của nó.

8.4 Phân tích chế độ thất bại: confusion matrix

Một con số accuracy tổng hợp che giấu *lớp nào* thực sự đang bị nhầm lẫn với lớp nào. Confusion matrix của Section 2.8 làm điều này cụ thể hóa: gộp các dự đoán held-out của mọi fold (mỗi bản ghi được dự đoán đúng một lần, do đó việc gộp không bao giờ đếm trùng), recall theo từng lớp là Normal 44%, Murmur 92%, Extra Sound 29%, Rhythm Disorder 20%. Sự nhầm lẫn chiếm ưu thế mang tính định hướng (directional), không đối xứng: lỗi của các lớp thiểu số kéo về phía Murmur — Extra Sound bị dự đoán là Murmur *thường xuyên hơn* so với việc bị dự đoán đúng (43% so với 29% recall), và Rhythm Disorder bị dự đoán là Murmur hoặc Extra Sound (30% mỗi lớp) nhiều hơn là chính nó (20%) — trong khi Murmur gần như không bao giờ bị nhầm với thứ gì khác (0% bị nhầm là Normal, 4% mỗi lớp là Extra Sound và Rhythm Disorder). Normal là ngoại lệ duy nhất, với lỗi của nó chạy về phía Rhythm Disorder (33%) thay vào đó. Điều này phù hợp với việc lớp Murmur chiếm ưu thế trong tập huấn luyện ($n=24$ trên 50) kéo ranh giới quyết định của SVM về phía nó bất chấp việc gán trọng số loss cân bằng theo lớp — việc gán lại trọng số cho loss huấn luyện không đảm bảo các *dự đoán* cân bằng một khi bản thân không gian đặc trưng đã cho bốn lớp này khả năng phân tách hạn chế ngay từ đầu.

8.5 Một cách có kiểm soát để sweep chất lượng phân tách: sơ đồ suy giảm (degradation)

Việc so sánh accuracy tại đúng hai điểm — âm thanh isolated hoàn hảo, và đầu ra thực của một phương pháp phân tách — là điều Yaqub et al. làm (Section 11.1), và chính câu hỏi nghiên cứu C2 của charter yêu cầu nhiều hơn thế: không chỉ *liệu* phân tách có tổn accuracy hay không, mà là *knee point* — chất lượng phân tách (SDR) mà

dưới đó việc phân tách bắt đầu tích cực gây hại cho accuracy so với việc không phân tách gì cả. Một knee point không thể được xác định từ hai điểm đầu cuối; nó cần một đường cong liên tục trên một khoảng chất lượng phân tách.

Phép sweep nhiễu của `synthetic_mix.py` (Section 6.3) đã thay đổi độ khó *trước khi* phân tách chạy, bằng cách kiểm soát mức độ nhiễu của mixture đầu vào. Một núm điều khiển (knob) thứ hai, bổ sung, thay đổi độ khó *sau khi* phân tách: lấy đầu ra thực tế s của một phương pháp phân tách thực và ground truth g của nó, rồi trộn chúng,

$$\text{degraded}(\alpha) = (1 - \alpha)g + \alpha s = g + \alpha(s - g), \quad \alpha \in [0, 1], \quad (9)$$

do đó $\alpha=0$ tái tạo chính xác ground truth sạch và $\alpha=1$ tái tạo chính xác đầu ra phân tách thực của phương pháp đó. Điều này có cùng dạng “tín hiệu cộng với một số hạng lỗi được scale” như phép sweep nhiễu trong công thức mixing tổng hợp của Equation 1, ngoại trừ số hạng được thêm vào $s - g$ là *lỗi đặc trưng thực sự của chính một phương pháp phân tách* (hỗn hợp cụ thể riêng của nó giữa nhiễu xen và artifact, Section 2.2) thay vì nhiễu Gauss tổng quát — hai núm điều khiển được giữ song song một cách có chủ đích, vì chúng trả lời các câu hỏi khác nhau: phép sweep nhiễu hỏi “accuracy suy giảm như thế nào khi bài toán phân tách trở nên khó hơn,” trong khi phép sweep nội suy (interpolation) hỏi “tại một chất lượng phân tách cho trước, dù đạt được bằng cách nào, accuracy hành xử ra sao,” giữ cố định *hình dạng* lỗi của một phương pháp cụ thể và chỉ thay đổi độ lớn của nó.

Một lưu ý thực sự, được đo đạc về sơ đồ này, được phát hiện chứ không phải giả định. Việc xác thực Equation 9 trên âm thanh thực cho thấy một lưới giá trị α *đồng đều không* tạo ra một sự trải đều của SDR đo được: trên hàng được kiểm tra, SDR giảm từ ~ 262 dB tại $\alpha=0$ (gần như giống hệt bit-for-bit với tín hiệu sạch) xuống chỉ còn ~ 10.6 dB ngay tại $\alpha=0.1$, sau đó phẳng dần xuống ~ 4.8 dB tại $\alpha=1$ — vì decibel có tính logarit và nhạy cảm nhất với một lỗi *tuyệt đối* nhỏ đúng ngay tại nơi tín hiệu gần như sạch. Do đó một lưới α đồng đều lãng phí phần lớn các điểm của nó vào đuôi phẳng. Vì thế bản triển khai đầu tiên giải α bằng tìm kiếm nhị phân, dựa trên giả định — được kiểm tra trên một hàng, chưa được chứng minh — rằng SDR đơn điệu theo α . Giả định đó sau này được kiểm toán trên mọi đường cong và thấy sai ở bảy đường, và tìm kiếm nhị phân đã được thay thế; cả hai được mô tả ở đây vì tính hợp lệ của sweep dựa vào điều này.

SDR(α) có dạng đóng, nên kiểm toán là toàn diện. Equation 9 tuyến tính theo α , và phép phân rã của BSS Eval một ước lượng thành thành phần đích và thành phần lỗi là một phép chiếu tuyến tính cố định lên không gian sinh bởi các tham chiếu có trễ. Vì thế mọi thành phần của $d(\alpha)$ là cùng một tổ hợp lỗi của các thành phần của g và của s , và $\text{SDR}(\alpha) = 10 \log_{10} N(\alpha)/D(\alpha)$ với N và D là các hàm bậc hai theo α mà sáu hệ số là tích trong của hai phép phân rã. Hai phép phân rã BSS Eval cho mỗi (phương pháp, hàng, nguồn) cho đường cong chính xác ở bất kỳ độ phân giải nào (`sdr_alpha_audit.py`); dạng đóng được đối chiếu với `mir_eval` tại năm giá trị α bên trong trên 72 đường cong và khớp tới 10^{-5} dB ở mọi nơi mà phép tìm hoán vị của `mir_eval` giữ nguyên gán tim/phối. Trên lưới 1001 điểm, 425 trong 432 đường cong (sáu phương pháp, 36 hàng, hai nguồn) đơn điệu không tăng trong phạm vi 10^{-6} dB. Bảy đường không đơn điệu đều là đường cong tim của Conv-TasNet-lite: ước lượng của mạng trên các hàng đó gần với một bản sao đảo cực tính của tham chiếu, nên phép trộn đi qua vùng gần triệt tiêu và SDR sụt xuống -6 đến -20 dB tại $\alpha \approx 0.04$ – 0.17 trước khi tăng lên đầu ra thật của phương pháp (-7 đến $+19$ dB). Với các hàng đó, các target

dưới đầu ra thật có hai nghiệm, và tìm kiếm nhị phân, vốn chỉ dò $\alpha \geq 10^{-3}$ trên lưới log và giả định một giao cắt, đã kẹp chúng về đầu ra thật — một điểm sai âm thẳm. Trên các đường đơn điệu, tìm kiếm nhị phân là đủ: trung vị $|\alpha_{\text{bisection}} - \alpha_{\text{exact}}| = 2 \times 10^{-4}$, phân vị 90 là 3.5×10^{-3} , và SDR đạt được nằm trong 0.17 dB của target ở phân vị 90.

Sweep nay giải α chính xác, theo một quy tắc được nêu trước. Vì $\text{SDR}(\alpha) = t$ là phương trình bậc hai $N(\alpha) - 10^{t/10}D(\alpha) = 0$, mọi nghiệm trong $[0, 1]$ được tìm ở dạng đóng, không phải lo về độ phân giải lưới ngay cả khi đường cong rơi 230 dB trong 10^{-3} đầu tiên của α . Quy tắc, cố định trước khi chạy lại: lấy nghiệm *nhỏ nhất* — giao cắt trên nhánh bắt đầu từ ground truth — và ghi số nghiệm cùng việc đường cong của hàng có đơn điệu hay không vào provenance. Một target không có nghiệm trong $[0, 1]$ (trên $\text{SDR}(0)$, hoặc dưới cực tiểu của đường cong, với đường đơn điệu chính là đầu ra của phương pháp) được ghi là không đạt được và điểm đó đặt tại $\alpha=1$ với cờ kẹp, như trước; không bao giờ bị ép tìm. SDR của mỗi điểm sweep được chấm với gán tìm/phối cố định, vì bên trong vùng sụt triệt tiêu, ước lượng tìm gần như không còn năng lượng và phép tìm hoán vị của `mir_eval` sẽ chấm ước lượng phối như “tim”; giá trị hoán vị được lưu kèm và gán cờ ở nơi khác biệt. Sinh lại provenance từ âm thanh đã phân tách trong cache theo quy tắc này đã thay đổi α của 99 trong 1053 target tìm đạt được quá 0.01 — 22 trong số đó trên bảy hàng không đơn điệu, phần còn lại ở nơi dung sai 0.1 dB của tìm kiếm nhị phân hoặc một lần lật hoán vị trong SDR đo được đã dịch nó — và bỏ kẹp 55 điểm; mọi điểm đạt được nay nằm đúng target tới bốn chữ số thập phân; mọi đường cong, knee và bootstrap trong báo cáo này đều từ sweep đã sinh lại, và lưới điểm vẫn là $\{25, 20, 15, 10, 5, 0, -5\}$ dB.

8.6 Sinh ra phép sweep SDR

Với mỗi một trong sáu baseline phân tách, mỗi một trong 36 hàng additive gốc (Section 5), và cả hai nguồn, bộ sinh sweep (`sdr_sweep.py`) fit baseline đó một lần mỗi fold (cùng hợp đồng an toàn rõ rĩ mà mọi baseline trước đó dùng, Section 6.2), phân tách mixture được giữ lại, cache âm thanh đã phân tách thực vào đĩa, và sau đó áp dụng phép tìm nghiệm target-SDR của Section 8.5 để đạt mọi giá trị trong lưới mục tiêu — một lệnh gọi phân tách mỗi hàng, sau đó bảy điểm suy giảm rõ được tái tạo từ nó, thay vì bảy lần phân tách lại tốn kém. Điều này tạo ra tối đa $6 \times 36 \times 2 \times 7 = 3024$ hàng nguồn gốc (provenance), mỗi hàng ghi lại baseline nào, mixture nào, nguồn nào, SDR mục tiêu là gì, α nào thực sự đạt được nó, và SDR/SIR/SAR đo được kết quả.

Vì điều này liên quan đến việc chạy lại mọi baseline phân tách — bao gồm cả hai baseline tốn kém nhất, EVMD (Section 6.8, ~ 15 s mỗi mixture) và Conv-TasNet-lite (Section 6.9, cần một mạng mới được huấn luyện cho mỗi fold) — việc sinh dữ liệu được viết script để checkpoint dần dần và tiếp tục an toàn nếu bị gián đoạn, thay vì cần chạy từ đầu đến cuối trong một lần. Lướt sinh dữ liệu đã hoàn thành: toàn bộ 3024 hàng provenance đều tồn tại (sáu baseline $\times$ 36 hàng $\times$ hai nguồn $\times$ bảy target, mỗi baseline đủ năm fold), và đường cache-rời-tái-tạo được kiểm tra lại trên 20 hàng lấy mẫu so với achieved SDR đã ghi, với sai lệch tối đa 0.25 dB. Một tính chất của sweep quan trọng cho việc đọc mọi đường cong bên dưới: lưới target chạy từ -5 đến 25 dB, nhưng một điểm không thể bị suy giảm xuống dưới đầu ra thật của chính baseline, nên các target dưới sàn đó bị kẹp (clamp) về sàn (29-37% số hàng mỗi baseline). Vì thế đường cong của mỗi baseline bắt đầu từ SDR thật của chính nó — 2.6 dB với Baseline 2, 5.5 dB với Baseline 1 — và hai điểm target thấp nhất của nó trùng với điểm vận hành Condition B của nó.

8.7 Condition B: phân loại âm thanh đã phân tách, và quyết định weight-sharing

Một điểm tinh tế cần được giải quyết trước khi “phân loại âm thanh đã phân tách” được định nghĩa rõ ràng: classifier được dùng trên âm thanh đã phân tách là *cùng bộ trọng số đã huấn luyện* như Condition A, hay một classifier mới được huấn luyện lại trên âm thanh huấn luyện đã phân tách? Đây là các thí nghiệm khác nhau trả lời các câu hỏi khác nhau — cái trước tách riêng “phân tách có gây hại cho một classifier chưa bao giờ thấy artifact phân tách trong lúc huấn luyện hay không,” cái sau hỏi “một classifier có thể thích nghi với artifact phân tách nếu được phép hay không.” Dự án này dùng *cùng bộ trọng số đã huấn luyện* cho mỗi fold, khớp chính xác với phương pháp luận của chính Yaqub et al. (mô hình Experiment 3 của họ *chính là* mô hình Experiment 4 của họ, không có huấn luyện lại ở giữa, Section 11.1): tái hiện phát hiện của họ dưới các điều kiện được kiểm soát có nghĩa là khớp với tuyên bố nhân quả (causal claim) của họ, và cùng-trọng-số cũng là cách tách biệt sạch hơn để phương pháp phân tách là biến số duy nhất giữa hai điều kiện.

Một yêu cầu nhất quán thứ hai, dễ bị bỏ sót, tiếp theo sau: phép gán fold của classifier và phép gán fold của sweep phải đồng thuận về ý nghĩa của “fold k ,” nếu không classifier của một fold có thể vô tình bị đánh giá trên âm thanh đã phân tách bắt nguồn từ một bản ghi mà nó một phần đã được huấn luyện trên đó. Vì `sdr_sweep.py` tính các fold của nó trên riêng tập con 36 hàng native-additive (không phải toàn bộ 145 hàng `Mix.csv` mà phép gán fold độc lập mặc định của chính Condition A dùng), phép gán fold của classifier được xây dựng lại một cách tường minh từ cùng cơ sở 36 hàng đó cho Condition B, thay vì được giả định là đã khớp sẵn.

Một yếu tố gây nhiễu (confound) được phát hiện ở lần chạy đầu, và đã loại bỏ. Lần chạy Condition B hoàn chỉnh đầu tiên cho ra một kết quả trông như một phát hiện nhưng thực ra là một khiếm khuyết: Baseline 1, 4 và 6 — một bộ lọc cố định, một phân rã SVD và một mạng neural — trả về *cùng 36 dự đoán*. Âm thanh đã phân tách trong cache được kiểm tra trước và khác nhau theo từng phương pháp (hash nội dung khác, RMS khác), và đường nạp dữ liệu trở mỗi baseline về thư mục riêng của nó, nên classifier thực sự nhìn thấy âm thanh khác nhau mà trả về cùng một đáp án. Lý do là mức âm lượng. Các bản ghi `HS.csv` trải từ 0.0005 đến 0.013 RMS, trong khi ước lượng tim của mọi baseline thừa hưởng hệ số chuẩn hóa đỉnh của mixture và nằm ở 0.03–0.09 RMS — to hơn 20–25× so với bất cứ thứ gì classifier từng được huấn luyện. MFCC₀ là log-năng-lượng, nên các đầu vào đã phân tách rơi ở khoảng cách chuẩn hóa 11–15 tính từ gốc, so với một đám mây huấn luyện mà điểm xa nhất ở 7.4; một kernel RBF cho giá trị ≈ 0 ở xa mọi support vector như thế, và hàm quyết định của SVM sụp về cùng một vector ($[0.95, 3.13, -0.16, 2.10]$ cho bandpass lẫn Conv-TasNet) với mọi đầu vào ở xa. Dự đoán là một hằng số theo fold — Rhythm Disorder với classifier của fold 0, Murmur với bốn fold còn lại — và không phụ thuộc vào âm thanh. Tệ hơn, phép trộn của sweep $(1 - \alpha) \text{ref} + \alpha \text{est}$ đưa âm lượng trở lại vùng huấn luyện khi SDR tăng, nên đường cong accuracy-vs-SDR đầu tiên một phần là đường cong âm-lượng-vs-SDR.

Cách sửa là làm cả hai classifier bất biến với mức tuyệt đối: mọi dạng sóng được scale về RMS đơn vị trước khi trích đặc trưng (`heart_classifier.level_normalize`, áp dụng y hệt bởi cả hai backend lúc huấn luyện và lúc suy luận). Không gì khác thay đổi. Sau đó, các tham chiếu isolated nằm trong đám mây huấn luyện (độ lớn chuẩn hóa trung bình ≈ 1 mỗi đặc trưng) còn các ước lượng đã phân tách vẫn nằm ngoài — ở khoảng cách 8–13, nay trên các hệ số hình dạng phổ (MFCC₁–MFCC₄) thay vì trên năng

lượng — đó chính là méo mà Condition B tồn tại để đo: một bandpass 20–200 Hz loại bỏ phần lớn đường bao phổ mà một classifier tiếng tìm huấn luyện trên bản ghi toàn dải dựa vào. Condition A cũng thay đổi theo các classifier huấn luyện lại (Table 13 báo cáo giá trị sau sửa); mọi con số Condition B và knee-point trong báo cáo này đều từ lần chạy đã sửa, và lần chạy trước sửa chỉ được giữ lại trong đoạn này.

Table 14 báo cáo kết quả cho cả sáu baseline trên cùng 36 hàng, với cùng bộ trọng số theo fold xuyên suốt.

Table 14: Condition B, cả hai kiến trúc, 36 hàng native additive, cùng bộ trọng số theo fold như Condition A. Hàng isolated là chính các tham chiếu tìm của 36 hàng này được chấm bằng cùng các classifier theo fold — một tập khác, nhỏ hơn 50 bản ghi của Condition A (Table 13), vì thế ở đây là 52.8% còn ở đó là 60.0/62.0%. Accuracy gộp trên 36 hàng; mọi khoảng là khoảng phân vị 95% từ 5000 lần lặp cluster-bootstrap trên 24 leak group mà các hàng thuộc về (`condition_b_stats.py`). Δ là accuracy isolated trừ sau phân tách, ghép cặp theo hàng; Δ_{raw} là accuracy sau phân tách trừ accuracy mixture thô, chính câu hỏi knee tại một điểm. SDR trung bình là đầu ra tìm thật của mỗi phương pháp (Table 10, gộp).

KT	Đầu vào	Accuracy [CI 95%]	SDR	Δ so với isolated [CI 95%]	Δ_{raw} [CI 95%]
1	Ground truth isolated	52.8 [35.3, 70.0]	—	—	—
1	Mixture thô (không phân tách)	38.9 [24.1, 53.1]	2.48	+13.9 [0.0, 29.0]	—
1	Baseline 1 (bandpass)	30.6 [14.3, 45.2]	5.46	+22.2 [2.5, 45.2]	−8.3 [−28.6, 12.0]
1	Baseline 2 (supervised NMF)	41.7 [24.3, 57.9]	2.58	+11.1 [−8.6, 33.3]	+2.8 [−17.1, 22.7]
1	Baseline 3 (standard NMF)	27.8 [14.3, 39.5]	3.35	+25.0 [11.6, 41.9]	−11.1 [−24.1, 1.9]
1	Baseline 4 (MSSA)	36.1 [19.4, 51.2]	5.02	+16.7 [−2.4, 38.9]	−2.8 [−20.6, 15.0]
1	Baseline 5 (EVMD)	27.8 [10.7, 43.2]	3.61	+25.0 [3.2, 48.6]	−11.1 [−32.4, 10.2]
1	Baseline 6 (Conv-TasNet-lite)	36.1 [21.2, 51.4]	5.02	+16.7 [−2.6, 38.2]	−2.8 [−23.5, 17.9]
2	Ground truth isolated	52.8 [34.3, 69.2]	—	—	—
2	Mixture thô (không phân tách)	33.3 [17.9, 50.0]	2.48	+19.4 [−2.7, 38.9]	—
2	Baseline 1 (bandpass)	16.7 [3.1, 30.6]	5.46	+36.1 [19.4, 52.8]	−16.7 [−34.5, 1.1]
2	Baseline 2 (supervised NMF)	44.4 [29.0, 57.6]	2.58	+8.3 [−10.5, 25.0]	+11.1 [−6.9, 29.1]
2	Baseline 3 (standard NMF)	27.8 [11.8, 44.4]	3.35	+25.0 [7.4, 40.5]	−5.6 [−22.9, 11.7]
2	Baseline 4 (MSSA)	16.7 [3.1, 30.6]	5.02	+36.1 [19.4, 52.8]	−16.7 [−34.5, 1.1]
2	Baseline 5 (EVMD)	16.7 [3.1, 30.6]	3.61	+36.1 [19.4, 52.8]	−16.7 [−34.5, 1.1]
2	Baseline 6 (Conv-TasNet-lite)	36.1 [19.4, 52.9]	5.02	+16.7 [−3.4, 34.4]	+2.8 [−18.4, 24.0]

Chiều hướng khớp với sự sụp đổ của Yaqub et al. ở cả sáu phương pháp dưới cả hai classifier, và các khoảng tin cậy cho biết bao nhiêu phần của nó phân giải được ở $n=36$. Dưới Kiến trúc 1, mức giảm là 11–25 điểm, và khoảng bootstrap theo leak group của hiệu ghép cặp loại trừ số không với ba phương pháp (bandpass, standard NMF, EVMD); kiểm định hoán vị đảo dấu cấp nhóm cho $p = 0.075, 0.004$ và 0.062 cho ba phương pháp đó, và 0.18–0.43 cho phần còn lại. Dưới Kiến trúc 2, mức giảm là 8–36 điểm và loại trừ số không với bốn phương pháp (bandpass, standard NMF, MSSA, EVMD; hoán vị $p = 0.003$ – 0.031). Các paired t -test cấp fold trước đây ($n=5$) được giữ trong `condition_b_report.html` nhưng không là cơ sở của tuyên bố nào: năm trung bình fold kích thước không đều là đơn vị yếu hơn và kém trung thực hơn 36 hàng ghép cặp được lấy mẫu lại theo leak group. Cột Δ_{raw} hỏi câu hỏi knee tại một điểm — âm thanh đã phân tách có được phân loại *tốt hơn mixture chưa đựng tới không?* — và câu trả lời là không với mọi phương pháp dưới cả hai classifier: mọi khoảng đều chứa số không

hoặc nằm dưới nó, và với standard NMF (Kiến trúc 1) cùng bandpass, MSSA và EVMD (Kiến trúc 2) toàn bộ khoảng nằm dưới số không.

Hai điều trong bảng đi ngược một cách độc ngậy thơ cột SDR. Mức giảm *nhỏ nhất* thuộc về phương pháp có SDR *thấp nhất*: supervised NMF, ở 2.6 dB, mất 11 (Kiến trúc 1) và 8 điểm (Kiến trúc 2), trong khi bandpass, ở 5.5 dB, mất 22 và 36. Và ba phương pháp lược bỏ nội dung phổ — bandpass, MSSA và EVMD — cho ra dự đoán gần như giống nhau dưới SVM (31–34 trong 36 y hệt) và *chính xác* cùng 36 dự đoán dưới CNN, vốn chấm cả ba ở 16.7%. Con số cuối đó lại là một hằng số theo fold: logit của CNN trên các đầu vào đó khác nhau theo phương pháp nhưng có cùng arg-max ở mọi fold (Extra Sound ở fold 0 và 3, Normal ở các fold còn lại), nên một classifier huấn luyện trên bản ghi toàn dải, đối mặt với âm thanh đã bị lược bỏ phần lớn đường bao phổ, rơi về một lớp mặc định bất kể phương pháp. Chuẩn hóa mức âm lượng đã loại bỏ một nguyên nhân của sự sụp đổ này (Section 8.7); nguyên nhân phổ không phải khiếm khuyết mà là hiện tượng Condition B đo, và nó có nghĩa là tại đầu ra thật, ba phương pháp lược bỏ không phân biệt được với cả hai classifier — khác biệt accuracy giữa chúng trong bảng là khác biệt của không đến ba hàng. Rõ ràng classifier phản ứng với việc bao nhiêu phần đường bao phổ sống sót qua phân tách, điều mà SDR, một tỉ số năng lượng miền dạng sóng, không đo trực tiếp. Bảng này tái hiện chiều hướng của kết quả Yaqub ở khoảng một nửa độ lớn của nó; các quỹ đạo bên dưới mới là thứ câu hỏi C2 thực sự cần.

8.8 Đường cong accuracy-vs-SDR và knee point

Một khi mọi điểm target-SDR trong sweep (Section 8.6) đã được phân loại với thiết lập weight-sharing của Condition B, kết quả là một đường cong cho mỗi baseline: SDR đo được trên trục hoành, accuracy phân loại trên trục tung. Đây, theo những gì dự án này tìm thấy được, là lần đầu tiên accuracy phân loại HLS-CMDS và SDR chất lượng phân tách được đặt lên cùng một trục — Yaqub et al. báo cáo accuracy mà không gắn SDR vào bất kỳ điểm đầu cuối nào, và mọi bảng chất lượng phân tách trong báo cáo này (Section 7) báo cáo SDR mà không gắn accuracy hạ nguồn nào.

Định nghĩa của knee point. Thay vì một phương pháp heuristic “elbow” tùy tiện dựa trên độ cong, knee point được định nghĩa trực tiếp từ chính câu chữ của charter: SDR tại đó đường cong accuracy của một baseline cắt qua accuracy của việc *hoàn toàn không phân tách* (phân loại trực tiếp mixture thô, chưa phân tách bằng cùng bộ trọng số classifier), đi từ SDR cao xuống SDR thấp. Trên knee, việc phân tách có ích (hoặc ít nhất không gây hại); dưới nó, một classifier sẽ hoạt động tốt bằng hoặc tốt hơn nếu không phân tách gì cả. Điểm cắt được tìm bằng nội suy tuyến tính giữa hai điểm *đo được* liền kề bao quanh nó — không phải một đường cong được làm mượt (smoothed) fit qua cả bảy điểm, vì việc làm mượt qua chỉ bảy điểm (mỗi điểm là một trung bình trên một số ít hàng) sẽ tạo ra một độ chính xác giả tạo mà dữ liệu không hỗ trợ. Bốn kết quả có thể xảy ra và được báo cáo riêng biệt thay vì gộp thành một con số: một điểm cắt sạch; đường cong ở trên tham chiếu không-phân-tách trên toàn bộ phạm vi đo được (không tìm thấy knee, phân tách luôn có ích ở đây); ở dưới trên toàn bộ phạm vi (phân tách không bao giờ có ích); hoặc một điểm cắt “nhiều” theo chiều sai, được đánh dấu như vậy thay vì âm thầm được coi là một knee sạch.

Đường cong. Figure 1 là đường cong sáu-baseline đầy đủ cho Kiến trúc 1, toàn bộ 1512 điểm sweep nguồn tìm được phân loại bằng bộ trọng số theo fold của Condition B; Table 15 cho các knee point mà bộ phát hiện tìm thấy trên đó.

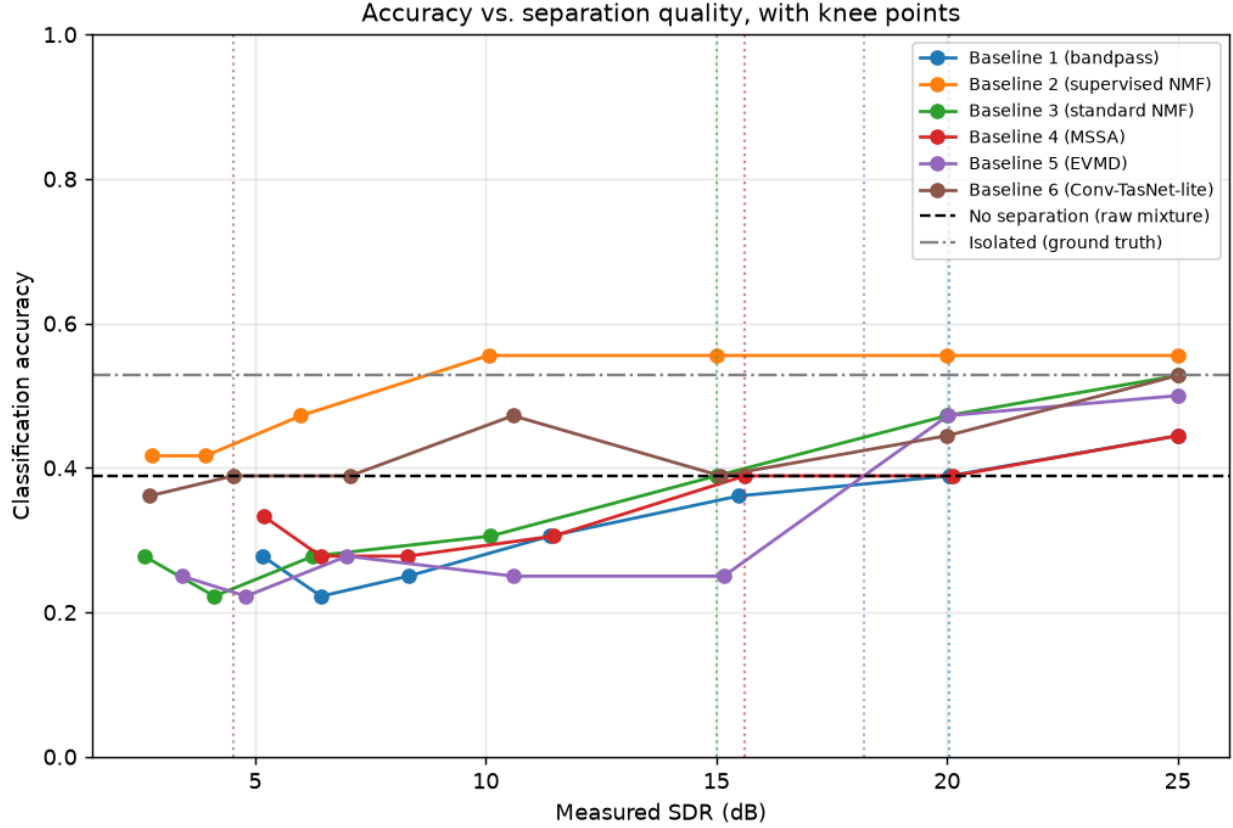


Figure 1: Accuracy phân loại theo SDR đo được, Kiến trúc 1 (MFCC+SVM), 36 hàng native additive mỗi điểm. Đường đứt nét là cùng classifier trên mixture thô (38.9%); đường chấm-gạch là ground truth isolated (52.8%). Các đường dọc chấm đánh dấu knee point của mỗi baseline. Các điểm ngoài cùng bên trái của mỗi đường nằm ở SDR đầu ra thật của chính baseline đó, vì sweep không thể suy giảm xuống dưới mức ấy.

Table 15: Giao cắt với mixture thô, Kiến trúc 1: SDR tại đó quỹ đạo của một baseline, đi từ SDR cao xuống thấp, cắt tham chiếu không-phân-tách (38.9%), bằng nội suy tuyến tính giữa hai điểm lấy mẫu kế nhau; “luôn ở trên” nghĩa là không điểm lấy mẫu nào rơi dưới tham chiếu. Các cột bootstrap từ Table 18 (lấy mẫu lại theo hàng); “k.x.đ.” = không xác định được trong lưới 25 dB. SDR đầu ra thật là đầu ra của chính phương pháp (Table 10, gộp).

Baseline	Giao cắt (dB)	$P(\text{cắt})$	Khoảng 95%	SDR đầu ra thật (dB)
Baseline 1 (bandpass)	20.1	0.71	[7.3, k.x.đ.]	5.46
Baseline 2 (supervised NMF)	luôn ở trên	0.43	—	2.58
Baseline 3 (standard NMF)	15.0	0.90	[5.0, k.x.đ.]	3.35
Baseline 4 (MSSA)	15.6	0.72	[7.3, k.x.đ.]	5.02
Baseline 5 (EVMD)	18.2	0.85	[6.1, k.x.đ.]	3.61
Baseline 6 (Conv-TasNet-lite)	4.5	0.63	[3.1, k.x.đ.]	5.02

Các quỹ đạo cho thấy gì. Sáu quỹ đạo tách thành hai nhóm mà riêng trục SDR không dự đoán được. Bandpass, standard NMF, MSSA và EVMD (Baseline 1, 3, 4, 5) đều bắt đầu ở 25–33% tại đầu ra thật của chính chúng — dưới tham chiếu không-phân-tách 38.9% — và leo lên 44–53% ở 25 dB, cắt tham chiếu trong khoảng 15 đến 20 dB. Supervised NMF và Conv-TasNet-lite (Baseline 2 và 6) bắt đầu *tại hoặc trên* tham chiếu dù có SDR thấp nhất và trung bình tương ứng: Baseline 2 không bao giờ rơi xuống dưới nó ở bất kỳ điểm lấy mẫu nào và nằm ở trần isolated từ 10 dB trở lên, còn lần duy nhất Baseline 6 rơi xuống dưới là hai điểm thấp nhất của nó (36.1% so với 38.9%, một hàng), mà bộ phát hiện báo là một giao cắt ở 4.5 dB. Cả hai đều là phương pháp có đầu ra là chính STFT của mixture dưới một mask biên độ (hoặc một tái tổng hợp học được của nó), nên đường bao phổ mà classifier được huấn luyện trên đó sống sót. Bandpass, MSSA và EVMD lược bỏ nội dung phổ theo cấu trúc; standard NMF cũng là phương pháp mask, nhưng mask của nó được fit chỉ trên mixture mà không có dictionary nguồn, và dưới classifier này nó hành xử như ba phương pháp lược bỏ. Với bốn phương pháp đó, hình vẽ nói điều nó đã nói trước khi sửa mức âm lượng, nay không còn yếu tố gây nhiễu: tại điểm vận hành mỗi phương pháp thực sự đạt được, đầu ra của nó phân loại tệ hơn mixture chưa đựng tới 6–14 điểm, và dọc theo giao thức nội suy này nó sẽ cần khoảng 15 đến 20 dB SDR nguồn tìm — nhiều hơn 10 đến 15 dB so với mức nó cho — trước khi phân tách thắng không-phân-tách. Với hai phương pháp dựa trên mask, cùng classifier đó thờ ơ với phân tách ở mọi SDR: đầu ra của chúng không bao giờ gây hại và, ở mức SDR thấp chúng đạt tới, cũng không bao giờ giúp ích.

Các quỹ đạo không cho thấy gì. Độ phân giải theo chiều dọc là một hàng trong 36 (2.8 điểm), và khoảng tin cậy nhị thức 95% của một accuracy trên 36 hàng quanh 40% là khoảng ± 16 điểm, rộng hơn khoảng cách giữa các quỹ đạo kề nhau ở phần lớn các SDR. Các cột bootstrap của Table 15 làm điều này cụ thể: với mọi baseline, đầu trên của khoảng giao cắt là đỉnh lưới đo, và với bandpass, MSSA và Conv-TasNet một phần tư đến một phần ba số lần lấy mẫu lại không có giao cắt nào. Vì thế các giá trị giao cắt riêng lẻ trong nhóm cắt-muộn không phân biệt được với nhau, và 4.5 dB của Baseline 6 là một lần trệch một hàng mà CNN bên dưới không tái hiện. Thứ sống sót qua nhiều là sự phân nhóm của Baseline 2 và 6 so với Baseline 1, 4 và 5 — nhất quán trên cả bảy mức target và, như Section 8.9 cho thấy, trên cả hai classifier — và thực tế là không điểm vận hành của phương pháp lược bỏ phổ nào nằm gần vùng giao cắt của nó. *Vùng* giao cắt của nhóm đó, khoảng 15–20 dB dưới classifier này, là con số trích dẫn được; các giá trị riêng lẻ thì không.

8.9 Kiểm tra tính vững: một classifier thứ hai, khác biệt, có thấy cùng một knee hay không?

Section 8.1 đã giải thích tại sao một kiến trúc classifier là chấp nhận được trong khi chất lượng phân tách (không phải accuracy) là kết quả chủ đạo của dự án này. Lý lẽ đó không còn áp dụng được một khi bản thân knee point *trở thành* kết quả chủ đạo: câu hỏi đầu tiên mà một độc giả hoài nghi sẽ đặt ra là liệu một knee point có phải là một thuộc tính của chất lượng phân tách, hay là một hiện tượng giả (artifact) của đúng một kiến trúc tình cờ đo được nó. Cách duy nhất để trả lời điều đó là lặp lại toàn bộ đường cong với một classifier thứ hai, khác biệt về kiến trúc, và xem liệu hai knee point có đồng thuận hay không.

“Khác biệt về kiến trúc” quan trọng ở đây, không chỉ đơn thuần là “một mô hình

thứ hai”: hai SVM khác nhau, hay một SVM với siêu tham số khác, sẽ không tách được đúng biến số, vì cả hai vẫn sẽ chia sẻ cùng một biểu diễn đặc trưng và cùng một loại ranh giới quyết định. Kiến trúc thứ hai được xây dựng cho kiểm tra này thay vào đó dùng một log-Mel spectrogram (một ảnh thời gian-tần số 2 chiều, cùng loại biểu diễn mà encoder của Baseline 6 học ngầm, Section 2.6) đưa vào một mạng neural convolutional nhỏ (hai khối convolutional, global average pooling, một lớp đầu ra tuyến tính duy nhất) — một mô hình được huấn luyện bằng gradient hoạt động trên một hình dạng đầu vào hoàn toàn khác với thống kê MFCC đã pool của Kiến trúc 1, được chọn cụ thể để tối đa hóa khả năng rằng một sự bất đồng, nếu được tìm thấy, phản ánh điều gì đó thực sự về ranh giới quyết định khác nhau của hai kiến trúc thay vì hai biến thể của cùng một ý tưởng.

Table 16: Kiến trúc 2 (log-Mel + CNN) so với Kiến trúc 1 (MFCC+SVM), Condition A, 5-fold cross-validated.

Metric	Kiến trúc 1 (MFCC+SVM)	Kiến trúc 2 (log-Mel+CNN)
Accuracy	60.0% $\pm$ 10.7%	62.0% $\pm$ 3.9%
Macro-F1	0.43 $\pm$ 0.12	0.54 $\pm$ 0.11

Cấu hình đầu tiên bị underfitting, và cách sửa là tối ưu hóa, không phải kiến trúc. Cấu hình đầu tiên của Kiến trúc 2 — đầu vào log-Mel thô, 40 bước Adam full-batch — đạt 30.0%, thấp hơn nhiều so với Kiến trúc 1, với confusion matrix sụp đổ về phía *Normal*. Một bản trước đó đọc điều đó như rủi ro overfitting dữ-liệu-nhỏ của Section 8.3 trở thành hiện thực. Thực tế ngược lại: accuracy *huấn luyện* của mô hình chỉ 23–40% mỗi fold, nên nó cũng chưa hề khớp được dữ liệu huấn luyện. Hai nguyên nhân, đều tầm thường. Đặc trưng log-Mel nằm quanh -12.4 ± 2.3 (một sàn 10^{-6} trên các bản ghi yên tĩnh), một độ lệch mà lớp convolution đầu tiên phải hấp thụ trước khi học được gì; và 40 bước full-batch là tổng cộng 40 lần cập nhật gradient. Chuẩn hóa từng Mel bin bằng mean và độ lệch chuẩn của chính fold huấn luyện (lưu trên classifier, áp dụng y hệt lúc test, nên không gì từ fold held-out lọt vào), huấn luyện theo mini-batch tám mẫu trong 300 epoch với weight decay 10^{-3} , và không thay đổi gì ở mạng, đưa accuracy huấn luyện lên 85–95%. Tổng cộng ba cấu hình đã được chạy và cả ba được ghi trong docstring của module; cấu hình được chọn là cấu hình đầu tiên có accuracy huấn luyện vượt 85% ở mọi fold — điểm mà underfitting rõ ràng đã hết — và người đọc nên biết rằng nó cũng có accuracy test cao nhất trong ba vào thời điểm đó, và cân nhắc điều đó tương ứng. Việc tinh chỉnh đó được làm trước khi phép chuẩn hóa mức âm lượng của Section 8.7 được thêm vào cả hai backend; với nó, và không thay đổi gì thêm, Kiến trúc 2 đạt $62.0\% \pm 3.9\%$ (accuracy các fold 60–70%, độ phân tán hẹp nhất trong mọi classifier của báo cáo này) với macro-F1 0.54. Confusion matrix của nó không còn sụp đổ: Normal recall 67%, Murmur 71%, Extra Sound 71%, Rhythm Disorder 30% — một hồ sơ cân bằng hơn Kiến trúc 1, vốn đối 92% Murmur recall lấy 20–29% ở các lớp thiểu số.

Đường cong thứ hai. Figure 2 lặp lại toàn bộ việc phân loại sweep với Kiến trúc 2 đã tinh chỉnh, trên cùng cơ sở fold 36 hàng, và Table 17 đặt knee point của hai kiến trúc cạnh nhau.

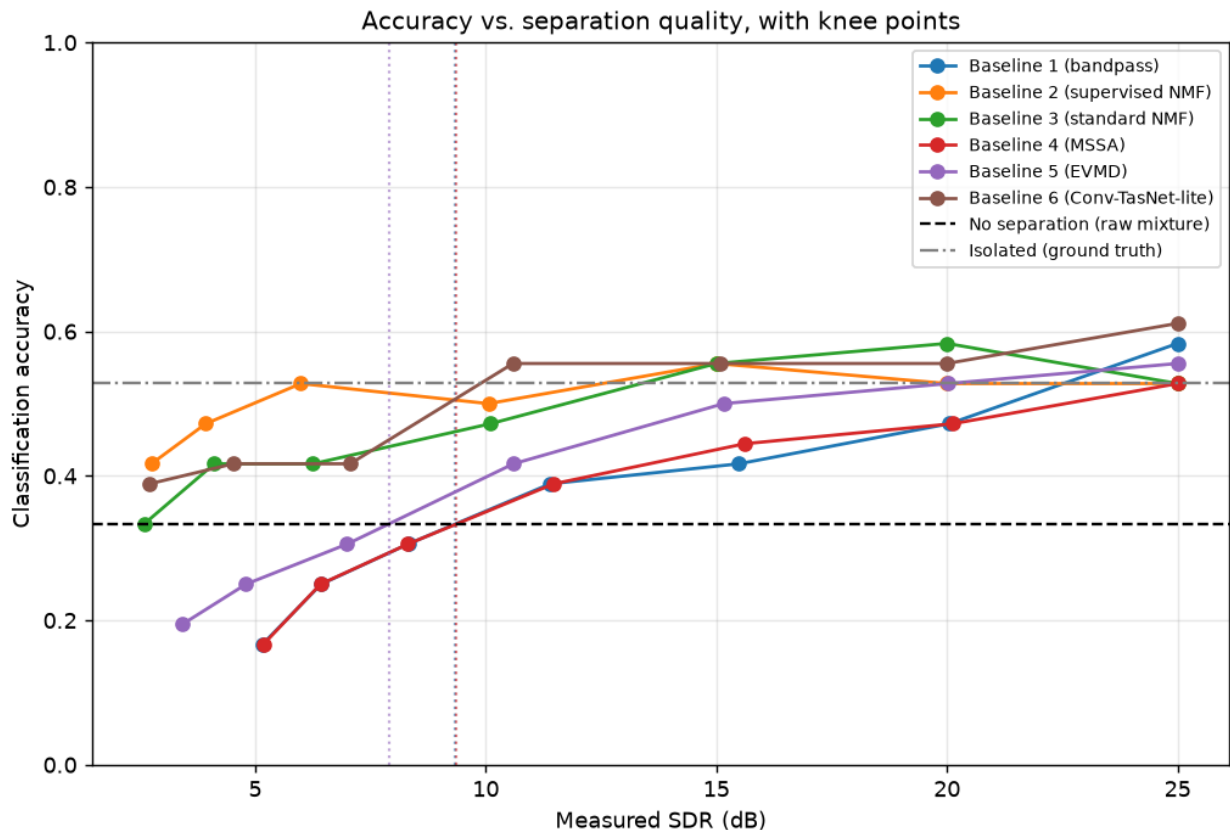


Figure 2: Cùng sweep đó được phân loại bằng Kiến trúc 2 (log-Mel + CNN). Tham chiếu không-phân-tách của nó là 33.3% và accuracy isolated của nó trên 36 hàng này là 52.8%.

Table 17: Knee point dưới cả hai kiến trúc. “luôn ở trên” nghĩa là đường cong không bao giờ rơi xuống dưới tham chiếu không-phân-tách của kiến trúc đó ở bất kỳ SDR nào đo được, nên không có knee trong phạm vi. Tiêu chí đồng thuận của bộ phát hiện là cả hai knee đều sạch và cách nhau trong 3 dB.

Baseline	Knee KT 1 (dB)	Knee KT 2 (dB)	Chênh lệch (dB)
Baseline 1 (bandpass)	20.1	9.3	10.7
Baseline 2 (supervised NMF)	luôn ở trên	luôn ở trên	—
Baseline 3 (standard NMF)	15.0	luôn ở trên	—
Baseline 4 (MSSA)	15.6	9.4	6.3
Baseline 5 (EVMD)	18.2	7.9	10.3
Baseline 6 (Conv-TasNet-lite)	4.5	luôn ở trên	—

Phán quyết: giá trị knee không vững trước kiến trúc; sự phân nhóm và hình dạng thì có. Theo tiêu chí đồng thuận vận hành 3 dB của nghiên cứu này — một ngưỡng làm việc chọn cho phân tích này, không phải một chuẩn — không baseline nào đồng thuận: ba giao cắt mà cả hai kiến trúc cùng tìm thấy cách nhau 6–11 dB, cái nào cũng thấp hơn dưới CNN, và với ba baseline còn lại CNN không tìm thấy giao cắt

ở nơi SVM tìm thấy, hoặc với supervised NMF, cả hai đều không. Hai điều giải thích phần lớn khác biệt đó. Ngưỡng tham chiếu của Kiến trúc 2 thấp hơn (33.3% so với 38.9%), nên cùng một quỹ đạo cắt nó sớm hơn; và các quỹ đạo của CNN dốc hơn và sạch hơn — cả sáu đường của nó đều đơn điệu hoặc gần như vậy trên bảy mức, tăng từ 17–42% lên 53–61%, trong khi các đường của SVM đi ngang ở ba mức đầu rồi mới leo. Vì thế một giao cắt trích dẫn tới từng decibel là thuộc tính của classifier đã đo nó, trên tập dữ liệu này ở cỡ mẫu này, và báo cáo này không trích dẫn con số nào như vậy.

Thứ hai bộ quỹ đạo chia sẻ là phần quan trọng với câu hỏi C2, và nó nhiều hơn hình dạng. Cả hai chia sáu phương pháp theo cùng một cách: supervised NMF “luôn ở trên” tham chiếu dưới cả hai, Conv-TasNet luôn ở trên dưới CNN và trệch xuống dưới một hàng dưới SVM, standard NMF luôn ở trên dưới CNN và cắt ở 15.0 dB dưới SVM, còn ba phương pháp lược bỏ phổ — bandpass, MSSA, EVMD — cắt muộn dưới cả hai, ở 15–20 dB với SVM và 8–9 dB với CNN. Dưới cả hai, điểm vận hành của ba phương pháp đó (3.6–5.5 dB) nằm dưới vùng giao cắt của chúng, nên đầu ra của chúng phân loại tệ hơn mixture thô. Kiểm tra tính vững vì thế làm yếu tuyên bố bằng số của Section 8.8 và làm sắc hơn tuyên bố định tính của nó: thứ một classifier hạ nguồn phản ứng với không phải là SDR như vậy mà là việc bộ phân tách có bảo toàn đường bao phổ hay không, và các phương pháp không bảo toàn sẽ cần cải thiện thêm vài decibel trước khi phân tách thẳng không-phân-tách.

Mỗi giao cắt bất định đến đâu, đo được. Table 18 bootstrap toàn bộ cấu trúc — quỹ đạo, trục SDR và tham chiếu cùng nhau, 2000 lần lặp, một lần trên hàng và một lần trên 24 leak group mà 36 hàng thuộc về — và tìm giao cắt trên mỗi lần lặp bằng cùng bộ phát hiện (`sdr_knee_bootstrap.py`). Hai điều mà ước lượng điểm không thể cho thấy. Thứ nhất, một giao cắt thậm chí không chắc tồn tại: với Conv-TasNet dưới Kiến trúc 1, một phần ba số lần lặp không tìm thấy, và với các phương pháp mask dưới Kiến trúc 2 phần lớn không tìm thấy. Thứ hai, ở nơi nó tồn tại, khoảng của nó trải gần hết phạm vi đo: mọi khoảng của Kiến trúc 1 đều chạm đỉnh lưới 25 dB và được ghi là không xác định được ở đó, và khoảng hẹp nhất trong bảng (EVMD dưới CNN, 3–15 dB) vẫn rộng 12 dB. Bootstrap theo cluster không dịch gì quá khoảng một decibel, nên sự phụ thuộc giữa các hàng không phải thứ giới hạn độ chính xác; $n=36$ mới là. Thứ ổn định qua các lần lặp là thứ tự mà đoạn văn trên dựa vào: dưới CNN, Baseline 1, 4 và 5 cắt trong 90–98% lần lặp còn Baseline 2 trong 15%, nên “các phương pháp lược bỏ có giao cắt còn supervised NMF thì không” sống sót qua lấy mẫu lại dù không giá trị giao cắt riêng lẻ nào sống sót.

Table 18: Độ bất định bootstrap của các giao cắt với mixture thô, 2000 lần lặp, lấy mẫu lại 36 hàng (i.i.d.) hoặc 24 leak group mà chúng thuộc về (cluster). Quỹ đạo, trục SDR của nó và tham chiếu không-phân-tách đều được tính lại trên mỗi lần lặp và giao cắt được tìm bằng cùng bộ phát hiện. $P(\text{cắt})$ là tỉ lệ lần lặp có giao cắt sạch; khoảng là phân vị 2.5–97.5 trên các lần lặp đó; “k.x.đ.” (không xác định được) nghĩa là phân vị nằm ở đỉnh lưới đo 25 dB.

KT	Baseline	Hàng ($n=36$)		Leak group ($n=24$)	
		$P(\text{cắt})$	Trung vị [CI 95%]	$P(\text{cắt})$	Trung vị [CI 95%]
1	B1 bandpass	0.71	15.4 [7.3, k.x.đ.]	0.75	15.6 [8.5, k.x.đ.]
1	B2 supervised NMF	0.43	6.2 [3.4, 22.5]	0.39	5.7 [3.1, 18.7]
1	B3 standard NMF	0.90	15.0 [5.0, k.x.đ.]	0.97	15.0 [6.0, 23.8]
1	B4 MSSA	0.72	15.4 [7.3, k.x.đ.]	0.75	15.6 [7.3, k.x.đ.]
1	B5 EVMD	0.85	18.0 [6.1, k.x.đ.]	0.92	18.1 [7.3, k.x.đ.]
1	B6 Conv-TasNet	0.63	18.4 [3.1, k.x.đ.]	0.65	19.0 [2.8, k.x.đ.]
2	B1 bandpass	0.95	9.5 [4.3, 21.7]	0.98	9.5 [3.6, 22.5]
2	B2 supervised NMF	0.15	3.9 [1.7, 22.5]	0.15	3.8 [1.7, k.x.đ.]
2	B3 standard NMF	0.45	4.2 [1.7, 13.7]	0.45	4.6 [1.7, 14.2]
2	B4 MSSA	0.94	9.4 [4.3, 21.7]	0.94	9.3 [3.5, 23.4]
2	B5 EVMD	0.90	8.3 [3.2, 15.2]	0.93	8.2 [2.5, 15.2]
2	B6 Conv-TasNet	0.27	8.2 [2.9, 22.5]	0.25	8.5 [2.0, 23.3]

9 Chi Phí Tính Toán

Mọi kết quả cho đến nay đều hỏi một phương pháp “tốt” đến mức nào. Một câu hỏi khác, riêng biệt nhưng cũng thực tiễn không kém — đặc biệt liên quan khi xét đến chính việc rà soát công trình liên quan của dự án này về một bộ phân loại phổ triển khai trên FPGA ([8], Table 27) — là mỗi phương pháp “tốn kém” đến mức nào để thực sự chạy. Phần này đo trực tiếp điều đó, dùng từ vựng của Section 2.9 (MAC, latency, Pareto front), cho cả sáu baseline phân tách cộng với classifier của Condition A.

9.1 MAC theo từng phương pháp

Table 19 báo cáo số lượng phép toán multiply-accumulate mà mỗi phương pháp thực hiện để xử lý một mixture thực dài 60,000 mẫu (15 s), cùng với số lượng tham số được lưu trữ (persisted) của mỗi phương pháp (bằng không đối với mọi phương pháp zero-training). Hai mức độ chính xác (precision) được báo cáo một cách tường minh thay vì được trình bày như có độ chắc chắn ngang nhau: các con số *exact* đến trực tiếp từ chính hình dạng phép nhân ma trận hoặc lớp convolution theo nghĩa đen của một phương pháp (hoặc, đối với Conv-TasNet-lite, từ các forward hook ghi lại hình dạng đầu ra thực trong một lượt forward pass thực, thay vì tự suy ra thủ công số học stride/padding/dilation); các con số *estimate* (SVD của MSSA, vòng lặp ADMM của EVMD) dùng các công thức độ phức tạp (complexity) tiêu chuẩn trong sách giáo khoa áp dụng vào các kích thước bài toán thực của dự án này, và là các con số theo bậc độ lớn (order-of-magnitude), không phải các con số đếm từng phép toán theo nghĩa đen.

Table 19: MAC và tham số theo từng phương pháp, một mixture/bản ghi thực 60,000 mẫu.

Phương pháp	Tham số	MAC/lượt suy luận	Độ chính xác
Baseline 1 (bandpass)	0	2.4 M	exact
Classifier Condition A (MFCC+SVM)	40 support vector	8.1 M	exact
Baseline 2 (supervised NMF)	7,710 (dictionary đóng băng)	219 M	exact
Baseline 3 (standard NMF)	0 (transductive)	1.16 G	exact
Baseline 4 (MSSA)	0	1.20 G	estimation
Baseline 6 (Conv-TasNet-lite)	325,465	1.96 G	exact
Baseline 5 (EVMD)	0	5.22 G	estimation

Hai phát hiện đáng được nêu bật cụ thể. Thứ nhất, số lượng tham số đo được của Conv-TasNet-lite (325,465) tái hiện độc lập con số $\sim 325K$ mà Section 6.9 đã báo cáo từ bản tóm tắt mô hình của chính PyTorch — một sự đối chiếu chéo thực sự giữa hai con số được tính toán độc lập, không phải một sự trùng hợp về làm tròn. Thứ hai, Baseline 3 tốn $\sim 5.3\times$ nhiều MAC hơn mỗi mixture so với Baseline 2 dù là ablation “đơn giản hơn” (không có dictionary học được, Section 6.6): Baseline 3 hoàn toàn *transductive* (mỗi trong số 160 vòng lặp multiplicative-update của nó cập nhật cả W lẫn H mới hoàn toàn mỗi mixture), trong khi Baseline 2 chỉ giải các activation (60 vòng lặp, chỉ H) đối chiếu với một dictionary đã đóng băng sẵn. Điều này cho sự phân biệt inductive-so-với-transductive đã được lưu ý ở Section 6.6 một con số chi phí tính toán cụ thể đi kèm với con số định tính.

9.2 Độ trễ (latency) trên desktop theo một giao thức thống nhất

MAC là một đại lượng thay thế lý thuyết (Section 2.9); con số thực sự mà một quyết định triển khai quan tâm là thời gian wall-clock. Latency được đo dưới một *giao thức thống nhất*, được giữ giống hệt nhau trên mọi phương pháp bất kể máy đang chạy gì khác: một lệnh gọi khởi động (warm-up) không tính thời gian (để bất kỳ chi phí cache hay JIT warm-up một lần nào bị loại trừ), sau đó nhiều lần lặp lại có tính thời gian trên cùng một đầu vào thực 60,000 mẫu, với *trung vị* (median, không phải trung bình, thứ mà một lần lặp chậm duy nhất có thể chi phối) và khoảng tứ phân vị (interquartile range) được báo cáo cho cả thời gian wall-clock lẫn thời gian CPU của tiến trình. Chi phí thiết lập một lần — fit dictionary NMF, huấn luyện Conv-TasNet-lite — được fit một lần, không tính thời gian, trước khi vùng có tính thời gian bắt đầu, vì phần này đo latency *suy luận* (inference), khớp với quy ước mà chính Yaqub et al. dùng cho phép so sánh mô hình của riêng họ (Table 5 của họ báo cáo thời gian suy luận, không phải thời gian huấn luyện).

Hai lượt đo, và vì sao lượt đầu bị rút lại. Phép đo đầu tiên (2026-08-27) được thực hiện trên một máy dùng chung với load average 1 phút là 74.6 trên 16 core — việc sinh sweep của chính dự án này cộng với job của người dùng khác — và được báo cáo kèm lưu ý đó cùng lập luận rằng một giao thức thống nhất giữ cho *thứ hạng* có ý nghĩa. Phép đo lại bên dưới, trên cùng máy khi rảnh (load average 0.6 lúc bắt đầu, 1.6 lúc kết thúc), cho thấy lập luận đó sai: lượt chạy dưới tải đã thổi phồng các phương pháp CPU-bound từ $10\times$ (bandpass, $36 \rightarrow 3.6$ ms) đến $340\times$ (standard NMF, $21,036 \rightarrow 61$ ms) trong khi gần như không đụng tới Conv-TasNet-lite chạy trên GPU

(23.9 $\rightarrow$ 16.7 ms), nên nó cũng không bảo toàn thứ hạng — nó đảo thứ tự hai phương pháp nhanh nhất. Mọi tuyên bố dựa trên latency trong báo cáo này đều từ lượt chạy máy rảnh. Table 20 cho lượt đó kèm độ phân tán mà người đọc cần để đánh giá; môi trường đo được chính script ghi lại (Table 21).

Table 20: Độ trễ suy luận trên desktop cho mỗi clip 15 s (60,000 mẫu), giao thức thống nhất: một lần warm-up không đo, rồi n lần lặp có đo; trung vị, IQR và p5–p95 của thời gian wall-clock, trung vị thời gian CPU của process, và hệ số thời gian thực (15 s/trung vị; trên 1 là nhanh hơn thời gian thực). Tiền xử lý theo từng lời gọi bên trong mỗi phương pháp (ví dụ bandpass khử nhiễu và STFT của Baseline 2) được tính; fit và huấn luyện một lần thì không. Máy rảnh: load average 0.6/16 core.

Phương pháp	n	Wall trung vị (ms)	IQR / p5–p95 (ms)	CPU trung vị (ms)	Hệ
Baseline 1 (bandpass)	5	3.63	0.01 / 3.62–3.70	3.6	
Classifier Condition A (MFCC+SVM)	5	3.69	0.02 / 3.67–3.74	63.6	
Baseline 6 (Conv-TasNet-lite), GPU	5	16.7	0.03 / 16.7–16.7	16.7	
Baseline 2 (supervised NMF)	5	17.2	2.2 / 17.1–22.2	107	
Baseline 3 (standard NMF)	5	61.4	0.7 / 60.6–61.6	367	
Baseline 4 (MSSA)	5	1,097	39 / 1,057–1,117	7,519	
Baseline 5 (EVMD)	3	16,812	49 / 16,794–16,882	16,812	

Table 21: Môi trường đo cho Table 20, ghi lại lúc chạy (results/latency_environment.csv).

CPU	Intel Core i9-9900K @ 3.60 GHz, 16 CPU logic
GPU	NVIDIA GeForce RTX 3060 (chỉ Conv-TasNet-lite; các phương pháp khác chạy CPU)
RAM	65.7 GB
OS / Python	Linux 6.8.0 (glibc 2.39) / 3.12.4
Thư viện	numpy 2.4.6, scipy 1.17.1, librosa 0.11.0, scikit-learn 1.9.0, torch 2.3.1+cu121, mir_eval 0.8.1
Thread	torch 8; OMP_NUM_THREADS không đặt (mặc định BLAS)
Batch	một clip mỗi lời gọi, không batch
Load average	0.64 (1 phút) lúc bắt đầu, 1.6 lúc kết thúc

Phép đo máy rảnh nói gì. Năm trong bảy phương pháp chạy nhanh hơn thời gian thực hàng trăm đến hàng nghìn lần trên workstation này; MSSA chạy ở $14\times$ thời gian thực và EVMD, ở $0.89\times$, là phương pháp duy nhất không theo kịp một clip 15 s trực tiếp. Thứ hạng wall-clock nay là bandpass < classifier < Conv-TasNet-lite $\approx$ supervised NMF < standard NMF < MSSA < EVMD, và nó ổn định: IQR dưới 1% trung vị với mọi phương pháp trừ supervised NMF (13%) và MSSA (4%).

MAC vẫn không dự đoán được latency, nhưng ví dụ đã đổi. Conv-TasNet-lite có $820\times$ số MAC của bộ lọc bandpass và chạy chậm hơn $4.6\times$, không phải $820\times$ — vì nó chạy trên GPU. EVMD có $2.7\times$ số MAC của Conv-TasNet-lite và chạy chậm hơn $1,000\times$ — vì nó là ADMM theo từng mode bằng NumPy trên CPU. Trong cùng một họ triển khai, hai thước đo đồng thuận: Baseline 2 và 3, đều là mã NMF của dự án này, có tỉ lệ thời gian CPU $367/107 \approx 3.4\times$ so với tỉ lệ MAC $5.3\times$. Sự phân kỳ giữa các triển khai là hiệu ứng hiệu quả triển khai và phần cứng, đó là lý do Section 2.9 khẳng định đo cả hai đại lượng; lượt chạy dưới tải đã phóng đại nó thành Conv-TasNet-lite là phương pháp nhanh nhất trong tất cả, điều không đúng.

Phạm vi của các con số này. Mọi con số trong tiểu mục này đều là phép đo trên desktop: chính workstation này, chạy các triển khai Python, khi rảnh. Chúng xếp hạng các phương pháp so với nhau trên một máy; chúng không phải là một tuyên bố về việc bất kỳ phương pháp nào trong số này tốn bao nhiêu trên phần cứng nhúng, nơi cả bộ xử lý lẫn triển khai đều khác. Với Baseline 1, tuyên bố đó giờ đã tồn tại riêng và được đo chứ không phải ngoại suy — xem Section 10.3, phần này thay thế con số của tiểu mục này cho riêng phương pháp đó.

9.3 Mặt phẳng SDR-so-với-compute

Việc kết hợp các con số chất lượng phân tách của Section 7 với các con số chi phí tính toán của phần này cho phép đặt mọi baseline lên một mặt phẳng duy nhất: chất lượng phân tách (SDR tim, tập con native additive) trên một trục, chi phí tính toán — MAC, hoặc latency đo được — trên trục kia. Hai panel được dùng thay vì gộp thành một trục “compute” duy nhất, chính xác là vì Section 9.2 đã tìm thấy MAC và latency bất đồng đáng kể đối với ít nhất một phương pháp; gộp thành một trục duy nhất sẽ che giấu chính xác loại phân kỳ vừa được chứng minh.

Định nghĩa Pareto-dominance của Section 2.9 được áp dụng trực tiếp: trên một trục cho trước, baseline A bị *thống trị* bởi baseline B nếu B có SDR bằng-hoặc-tốt-hơn và chi phí tính toán bằng-hoặc-thấp-hơn, với ít nhất một cải thiện chặt — nghĩa là A không bao giờ là lựa chọn đúng trên trục đó, bất kể chất lượng và chi phí được cân trọng số ra sao với nhau. Việc sinh Figure/Table cho mặt phẳng này (`sdr_compute_plane.py`) đọc SDR từ file cấp hàng canonical (Table 10, trung bình gộp và CI 95%) và latency từ lượt chạy máy rảnh của Table 20, nên mặt phẳng được suy ra từ cùng các artifact như mọi bảng khác chứ không từ các con số chép vào script.

Table 22: Trạng thái Pareto-dominance theo từng baseline, trên trục MAC và trục latency riêng biệt.

Phương pháp	SDR tim (dB)	Bị thống trị trên trục MAC?	Bị thống trị trên trục latency?
Baseline 1 (bandpass)	5.46 ± 2.66	Không	Không
Baseline 2 (supervised NMF)	2.58 ± 1.87	Có	Có
Baseline 3 (standard NMF)	3.35 ± 1.88	Có	Có
Baseline 4 (MSSA)	5.02 ± 2.91	Có	Có
Baseline 5 (EVMD)	3.61 ± 2.35	Có	Có
Baseline 6 (Conv-TasNet-lite)	5.02 ± 2.77	Có	Có

Trên mặt phẳng MAC, một phương pháp thống trị hoàn toàn. Lọc bandpass có cả SDR tim cao nhất trong sáu baseline lẫn, với một biên độ gần ba bậc độ lớn, số lượng MAC thấp nhất (Table 19) — mọi baseline khác đều tệ hơn hẳn trên cả hai trục cùng lúc, và do đó bị Pareto-thống trị. Nếu MAC là con số chi phí tính toán duy nhất có sẵn, kết luận sẽ rõ ràng không mơ hồ: không có lý do gì, dựa trên bằng chứng này, để ưu tiên bất kỳ phương pháp phân tách đã học hay lập nào hơn bộ lọc zero-training.

Trên mặt phẳng latency đo được, kết luận vẫn như vậy, và một bản trước của báo cáo này nói khác. Với các latency của máy dưới tải, Conv-TasNet-lite (23.9 ms) có vẻ nhanh hơn bandpass (36.0 ms) và cả hai được báo cáo là cùng không bị thống trị

— một đánh đổi thực sự. Trên máy rảnh, bandpass chạy trong 3.6 ms và Conv-TasNet-lite trong 16.7 ms trên GPU, nên bandpass có SDR cao hơn và latency thấp hơn, và mọi baseline khác bị thống trị trên cả hai mặt phẳng. Phát hiện bị rút lại là artifact của việc đo các phương pháp CPU-bound trên một CPU bão hòa trong khi phương pháp GPU-bound duy nhất không bị ảnh hưởng; nó được ghi lại ở đây vì đã được công bố trong một bản trước và vì nó là ví dụ sạch của một phép đo trông hợp lý đã đảo ngược một kết luận. Điều còn đúng là phát biểu yếu hơn: Conv-TasNet-lite là phương pháp duy nhất có latency trong vòng một bậc độ lớn so với bộ lọc dù có nhiều hơn ba bậc độ lớn phép toán, đó là thứ mà GPU mua được.

10 Triển Khai Nhúng: Baseline 1 trên Vi Điều Khiển

Mọi con số trong Section 9 đều là phép đo trên desktop hoặc một phép đếm giải tích. Không cái nào trả lời câu hỏi mà một quyết định triển khai thực sự đặt ra: phương pháp này có chạy được trên phần cứng mà nó được dự định chạy hay không? Bài báo Edge-Enabled [8] trong Section 11 nhắm tới một FPGA PYNQ-ZU ở khoảng 15 W; phần này báo cáo Baseline 1 được port sang, và đo trên, một mục tiêu nhỏ hơn đáng kể — một Arduino Nano 33 BLE Sense (nRF52840, Cortex-M4F 64 MHz, 256 kB RAM). Bản port nằm trong `firmware/`, với một mục tiêu ESP32-S3 được build từ cùng phần lõi nhưng chưa được chạy thử trên silicon.

Baseline 1 là ứng viên đầu tiên tự nhiên, và không chỉ vì nó đơn giản nhất: Section 9.3 đặt một mình nó trên Pareto front của cả hai mặt phẳng compute, nên nó là phương pháp mà một quyết định triển khai sẽ cân nhắc trước tiên.

10.1 Điểm khác biệt có chủ đích duy nhất: lọc nhân quả (causal filtering)

Baseline Python gọi `scipy.signal.sosfiltfilt`, hàm này chạy cascade theo chiều xuôi rồi chạy ngược lại trên tín hiệu đã đảo chiều. Cách đó triệt tiêu chính xác đáp ứng pha của bộ lọc, nhưng nó không thể bắt đầu cho đến khi *toàn bộ* bản ghi đã tồn tại. Firmware thay vào đó chạy tương đương với `sosfilt`: một lượt nhân quả duy nhất, từng block một, trạng thái bộ lọc được mang qua các lần gọi.

Đây là một lựa chọn bắt buộc, không phải sở thích. Một clip 15 s ở 4000 Hz là 60,000 mẫu; ở dạng `int16` thô là 120 kB, tức 46% tổng RAM của Nano 33 trước khi bất kỳ xử lý nào bắt đầu, và `filtfilt` còn cần thêm một bản sao làm việc `float32` của cùng tín hiệu — thêm 240 kB nữa, trên một linh kiện có tổng cộng 256 kB. Không vừa. Ngay cả ở nơi nó vừa, nó cũng đánh mất ý nghĩa của bài toán: không đầu ra nào có thể được tạo ra cho đến khi bản ghi kết thúc, tức là xử lý theo lô có đệm (buffered batch processing) khoắc áo vi điều khiển.

Hai cái giá phải trả theo sau, cả hai đều thật. Đầu ra mang theo đáp ứng pha của bộ lọc và một group delay khác không, nên các so sánh hình dạng sóng với baseline Python không còn tương đương (like-for-like). Và `sosfiltfilt` áp dụng cùng cascade hai lần, nên stopband của nó sâu gấp đôi tính theo dB — một lượt của thiết kế bậc 4 thực sự là một bộ lọc yếu hơn so với những gì Section 6.4 báo cáo.

Đo, không giả định. Chạy cả hai bộ lọc trên cùng 145 hàng `Mix.csv` với hệ số y hệt và metric BSS Eval y hệt — chỉ đổi `sosfiltfilt` thành `sosfilt` — cho ra các chênh lệch trung bình trong Table 23.

Table 23: Nhân quả trừ zero-phase, trung bình trên 145 hàng. Chỉ có chiều lọc thay đổi.

Nguồn	Δ SDR	Δ SIR	Δ SAR
Tim	+0.18 dB	-0.13 dB	+5.47 dB
Phổi	-0.04 dB	-0.50 dB	+7.30 dB

SDR không đổi trong phạm vi ± 0.2 dB — trên tập dữ liệu này bộ lọc nhân quả về cơ bản không tổn gì trên metric chính, tốt hơn những gì lý thuyết ở trên gợi ý. SIR giảm nhẹ, đúng như độ sâu stopband hiệu dụng bị giảm một nửa dự đoán, vì loại bỏ nguồn còn lại chính là thứ mà độ sâu stopband mua được. **Mức tăng SAR lớn là một artifact của metric và không được báo cáo như thể firmware phân tách sạch hơn:** phép chiếu của `mir_eval` khớp một bộ lọc méo *nhân quả* 512 tap, nên đáp ứng pha của một IIR nhân quả nằm trong không gian con méo-được-phép và được hấp thụ, trong khi đáp ứng phi nhân quả của một bộ lọc zero-phase không khớp và một phần bị tính là artifact.

10.2 Xác minh một bản port không có triển khai tham chiếu trên mục tiêu

Một bản build link được không phải là một bản build tính đúng, và board chạy một kernel khác với host: `arm_biquad_cascade_df2T_f32` của CMSIS-DSP trên Nano 33, `dsps_biquad_f32` của ESP-DSP trên ESP32-S3, so với C khả chuyển (portable C) trên desktop. Cả ba nằm sau một API duy nhất để cùng mã nguồn build được cho cả ba, nhưng các kernel của nhà cung cấp là mã bên thứ ba, và CMSIS viết phép đệ quy ước dấu cho a_1/a_2 ngược với `scipy` — một chi tiết mà bước khởi tạo phải sửa cho đúng và không trình biên dịch nào bắt được.

Vì thế bản port được xác minh qua bốn giai đoạn, ba trong số đó không cần phần cứng nào cả:

1. **Trên host, C khả chuyển so với scipy.** Lỗi C lọc cùng các vector vàng (golden vectors) mà `scipy` tạo ra và độ lệch được so với sàn lượng tử hóa (quantisation floor) `float32`. Cũng được kiểm tra ở đây: đầu ra y hết cho block size 1, 7, 64 và 333, đây là thứ chứng minh trạng thái streaming thực sự được mang qua các lần gọi chứ không bị reset âm thầm.
2. **Trên host, giao thức đường truyền (wire protocol).** Một board mô phỏng qua pseudo-terminal thử nghiệm khung serial theo cả hai chiều, xác nhận một stream trung thực được chấp nhận và một stream bị hỏng bị từ chối thay vì bị chấm điểm âm thầm.
3. **Trên silicon, kernel của nhà cung cấp.** Firmware `*-selftest` phát lại một tham chiếu 1024 mẫu giữ trong flash qua kernel của nhà cung cấp và in độ lệch ra serial. Bước này phải chạy *trước* bất kỳ phép đo nào: nếu kernel sai, mọi con số sau đó đều sai theo một cách trông giống như một vấn đề thiết kế bộ lọc.
4. **Trên silicon, clip đầy đủ.** Firmware `*-bench` phát lại các mixture HLS-CMDS thật được biên dịch vào flash và stream đầu ra đã lọc trở lại để chấm điểm.

Firmware bench cố ý *không có microphone*. Nó lọc cùng các mẫu `int16` mà baseline Python lọc — giống hệt từng bit, vì phép scale $1/32768$ tái tạo chính xác `librosa.load()` — nên đầu ra của board có thể diff với scipy từng mẫu một và chấm điểm bằng cùng các metric BSS Eval như phần còn lại của báo cáo này. Một microphone sẽ đưa vào một transducer, một bước resampling và một mức gain cần hiệu chuẩn loại bỏ, không cái nào trong số đó cần cho câu hỏi đang được đặt ra ở đây. Các firmware microphone (PDM trên Nano 33, I2S trên ESP32-S3) tồn tại cho một thiết bị thực trong tương lai nhưng không nằm trong đường đo.

Chỉ các mixture additive được nhúng. Section 5 tìm thấy 109 trong 145 hàng có mixture không phải là $a \cdot (\text{tim} + \text{phối})$ của các nguồn được gán tên, và một SDR chấm với các tham chiếu đó đo khiếm khuyết của tập dữ liệu chứ không phải bộ phân tách. Lần chạy phần cứng đầu tiên đã nhúng M0001–M0003, cả ba đều không additive, khiến cột SDR của nó không thể trích dẫn; việc chọn giờ lấy từ 36 hàng hợp lệ.

10.3 Kết quả trên silicon

Table 24 báo cáo lần chạy phần cứng đầu tiên: ba mixture additive được phát lại từ flash trên một Nano 33 BLE Sense.

Table 24: Nano 33 BLE Sense, ba mixture additive phát lại từ flash.

Clip	board – scipy , $\times$ sàn float32		SDR (dB), board = scipy		$\mu\text{s}/\text{mẫu}$
	tim	phối	tim	phối	
M0087	2.04	0.89	4.918	3.686	3.474
M0111	1.26	0.90	18.91	−12.43	3.488
M0112	0.82	0.71	4.556	1.197	3.431

Bản port là trung thực. Mọi SDR đều khớp với tham chiếu scipy đến chữ số cuối cùng được in, trên cả sáu phép đo clip/nguồn. Đây không phải hai ước lượng độc lập của cùng một đại lượng tình cờ trùng nhau — chúng là cùng một phép tính được thực hiện trên hai máy khác nhau, và sự trùng khớp chính là phép kiểm tra.

Thời gian: 3.47 μs mỗi mẫu, nhanh hơn thời gian thực 72 $\times$. Một clip 15 s xong trong 209 ms compute trên một Cortex-M4F 64 MHz, đo bao quanh phép scale `int16` và cả hai cascade dài, loại trừ I/O serial. Khoảng dư (headroom) đó là kết quả liên quan đến triển khai: phép tách dài này có thể chạy như một stream trực tiếp trên một linh kiện giá vài đô-la, với hai bậc độ lớn để dành. Với Baseline 1, con số này thay thế phép đo desktop của Section 9.2, vốn đo một máy khác làm một lượng việc khác. Không có gì trong mức dùng RAM của firmware tăng theo độ dài bản ghi — các bộ lọc là streaming và các clip nằm trong flash — nên một clip 15 giây và một stream dài một giờ tốn cùng vài trăm byte trạng thái bộ lọc.

10.4 Một phép đo trông hợp lý nhưng sai 18 $\times$

Con số thời gian đó ban đầu là 62.4 μs mỗi mẫu — 4.0 $\times$ thời gian thực. Không có gì ở nó trông sai. Đó là một con số hợp lý cho một linh kiện 64 MHz, nó nhất quán trên cả ba clip đến ba chữ số có nghĩa, và nó đến từ một firmware mà đầu ra số đã được

xác minh với scipy hai lần. Nó cũng chậm hơn $18\times$ so với thực tế, và lý do đáng được ghi lại vì không có gì trong bản thân phép đo có thể làm lộ điều đó.

Sự bất nhất chỉ nổi lên từ mô hình tài nguyên của Section 10.8, mô hình này dự đoán khoảng $400\times$ thời gian thực cho phương pháp này. Một mô hình lạc quan $5\times$ là chuyện thường; lạc quan $100\times$ là một tuyên bố rằng có gì đó sai ở đâu đó, và mô hình đủ cụ thể để đáng tin hơn trực giác rằng 62 μ s “có vẻ ổn”.

Đọc lệnh gọi trình biên dịch thực tế — thay vì cấu hình build, nơi chứa ý định nhưng không phải kết quả — cho thấy nguyên nhân. Builder Arduino-mbed nối thêm `-nostdlib -mfloat-abi=soft sau -mfloat-abi=softfp` có sẵn trước đó của framework, và GCC tôn trọng lần xuất hiện cuối cùng của cờ đó. Vì thế mọi phép toán dấu phẩy động trong firmware đều được biên dịch thành các lời gọi thư viện `software-float __aeabi_*`. Disassemble cả object bộ lọc của chính dự án lẫn kernel biquad của CMSIS-DSP đã xác nhận trực tiếp: không có lệnh VFP nào trong cả hai, trên một Cortex-M4F mà đơn vị dấu phẩy động chính là toàn bộ lý do cho hậu tố đó.

Thêm cờ đúng vào cấu hình build không sửa được, vì các cờ đó được phát ra *trước* nhóm cờ cuối của builder nên vẫn thua trong cuộc đua xuất-hiện-cuối — một chi tiết chỉ thấy được khi đọc lại dòng biên dịch sau lần sửa thử, và để bỏ sót nếu giả định rằng lệnh ghi đề đã có hiệu lực. `build_unflags` của PlatformIO gỡ hẳn cờ gây lỗi. Sau thay đổi: có lệnh VFP trong cả hai object, không còn lời gọi `software-float`, flash giảm 1,248 byte, và phép đo lại trong Table 24.

Hai điều khái quát được từ đây. Thứ nhất, một con số hiệu năng trông hợp lý không có nghĩa là đã được xác thực nhờ trông hợp lý; ở đây thứ duy nhất bắt được nó là một mô hình độc lập bất đồng đủ lớn để đáng theo đuổi. Thứ hai, self-test của Section 10.2 đã pass cả trước lẫn sau — một cách đúng đắn, vì software floating point tính ra đáp án đúng, chỉ là chậm. Xác minh số học và xác minh hiệu năng là hai nghĩa vụ tách biệt, và pass cái thứ nhất không nói lên gì về cái thứ hai.

10.5 Một ngưỡng dung sai được hiệu chuẩn trên sai tín hiệu

Lần chạy đầu tiên trên các clip additive đã *thất bại* phép kiểm tra độ chính xác, ở 1.045×10^{-5} so với dung sai tuyệt đối 10^{-5} — trên một board mà self-test trên silicon vừa pass ở 5.3×10^{-8} và SDR khớp với scipy đến bốn chữ số có nghĩa. Dung sai sai, không phải kernel, và lý do đáng được ghi lại vì đó là một cái bẫy tổng quát.

Board tính ở `float32` trong khi tham chiếu là `float64`, nên khoảng cách giữa chúng không phải hằng số: nó tỉ lệ với biên độ tín hiệu, vì một IIR tích lũy sai số làm tròn qua chính trạng thái đệ quy của nó. Con số 10^{-5} đã được hiệu chuẩn trên M0001-M0003, có đỉnh gần 0.05. 36 hàng additive được chuẩn hóa đỉnh về full scale — chính phép chuẩn hóa đó là dấu vân tay clipping nhận diện chúng (Section 5) — nên chúng to hơn khoảng $20\times$, và cùng một phép số học đúng rơi xa tham chiếu hơn khoảng $20\times$.

Phép kiểm tra quyết định là chạy *chính scipy* ở `float32` trên cùng clip: trên M0112 nó lệch 1.114×10^{-5} , tức *lớn hơn* mức 1.045×10^{-5} của board. Board không chỉ chấp nhận được; nó còn chính xác hơn cả triển khai tham chiếu nếu chạy ở cùng độ chính xác.

Tiêu chí giờ được đo thay vì giả định: sần `float32` được tính theo từng clip bằng cách lọc cùng các mẫu với scipy ở `float32`, và board bị giữ ở mức $4\times$ bất kể giá trị đó là bao nhiêu. Tỉ số quan sát trên phần cứng thực là 0.71-2.04 (Table 24). Để hiệu chuẩn, các lỗi được cố ý tiêm vào cho điểm cao hơn rất nhiều: sai số gain 0.01% cho 14.3, một mẫu bị rơi cho 15,691, và đảo dấu cho 296,503 — nên ngưỡng nằm trong một

khoảng trống rộng, với khoảng $7\times$ biên độ dư phía trên board thực và thêm hai bậc độ lớn phía dưới lỗi nhỏ nhất được thử.

10.6 Server so với vi điều khiển, trên khối lượng việc y hệt

Câu hỏi tự nhiên một khi một phương pháp chạy được ở cả hai nơi là việc di chuyển tốn gì. Trả lời nó cần cẩn trọng hơn là chia con số desktop của Section 9.2 cho con số của board, vì hai lý do mà cả hai đều tăng tốc board nếu bị bỏ mặc.

Hai nền tảng không làm cùng một việc. Section 9.2 đo `bandpass_separate`, hàm này gọi `sosfiltfilt`: xuôi rồi ngược, hai lượt cho mỗi dải. Firmware thực hiện một lượt nhân quả (Section 10.1). Vì thế con số desktop là cho khoảng gấp đôi lượng số học, và `server_vs_board.py` đo lại workstation làm chính xác những gì board làm, thay vì chia đôi con số hiện có và giả định các hằng số triệt tiêu nhau. Đo trực tiếp, phiên bản zero-phase tốn $4.8\times$ phiên bản nhân quả, không phải $2\times$ như riêng hai lượt của nó gợi ý — `sosfiltfilt` còn pad tín hiệu, đảo chiều nó hai lần, và giải điều kiện đầu cho từng section. Chia đôi con số hiện có do đó sẽ sai hơn một hệ số hai.

Một tỉ số thô trộn lẫn hai tuyên bố khác nhau. 3.6 GHz so với 64 MHz là bất lợi xung nhịp $56\times$ trước khi kiến trúc được xét đến. “Bộ xử lý này nhanh hơn” và “bộ xử lý này làm được nhiều hơn mỗi chu kỳ” là hai phát biểu tách biệt, và chỉ cái thứ hai nói lên điều gì đó về việc bản port có phải là ý hay hay không. Số chu kỳ mỗi mẫu tách hai điều đó ra.

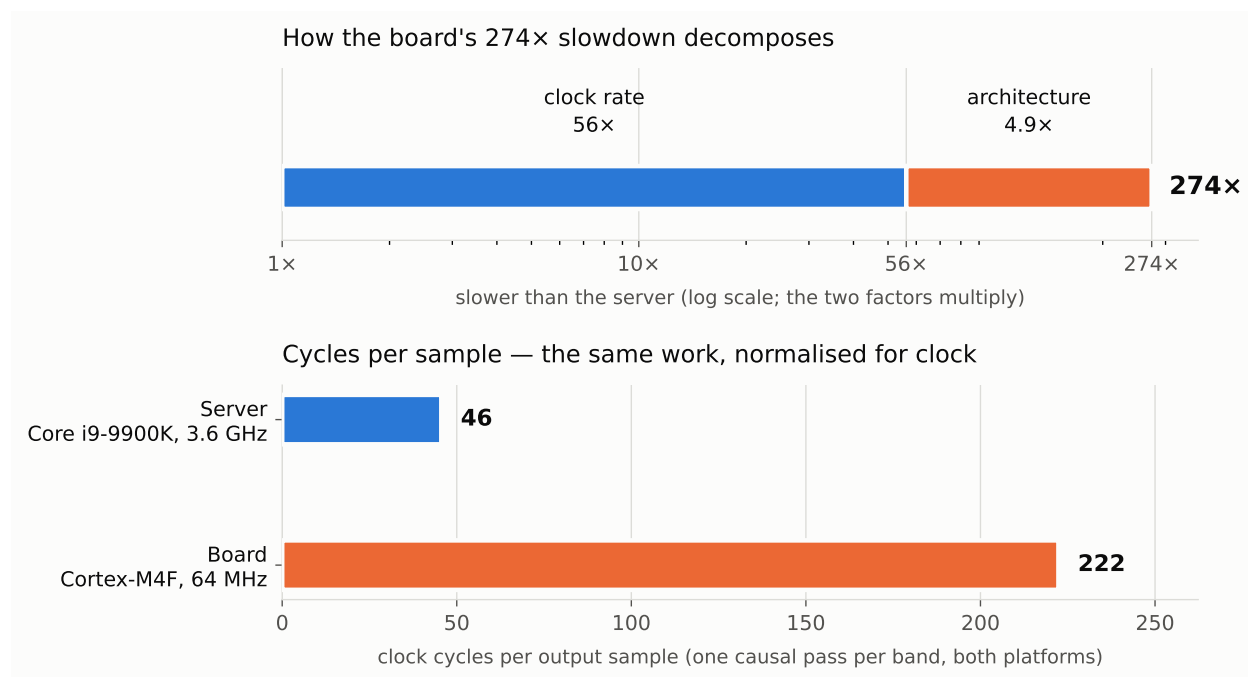


Figure 3: Khoảng cách throughput $276\times$ được phân rã. Vì hai thừa số nhân với nhau chứ không cộng, panel trên đặt chúng trên trục log, nơi độ dài các đoạn chính xác là $\log 56$ và $\log 4.9$. Panel dưới là phép đo bên dưới: cùng phép tách dải nhân quả, chuẩn hóa theo xung nhịp, nơi khoảng cách là hệ số năm thay vì hệ số 276.

Table 25: Khối lượng việc y hệt — một lượt nhân quả cho mỗi dải trên cùng mixture 60,000 mẫu. Con số server là trung vị của năm lần chạy độc lập (độ phân tán dưới 1%) ở load average 0.1; con số board đo trên silicon.

Nền tảng	ms/clip	μ s/mẫu	chu kỳ/mẫu	$\times$ thời gian thực
Server, i9-9900K @ 3.6 GHz	0.75	0.0126	45.2	19,900
Board, Cortex-M4F @ 64 MHz	208.5	3.474	222.3	72
Board, trước khi sửa FPU	3,747	62.45	3,997	4.0

Phần lớn khoảng cách là xung nhịp, nhưng không phải toàn bộ. Board chậm hơn $275\times$ mỗi mẫu so với bất lợi xung nhịp $56\times$, nên mỗi chu kỳ nó tiêu $4.9\times$ những gì workstation tiêu cho cùng phép lọc. Phân rã $275\times$ theo cách đó là bước hữu ích: khoảng một phần năm là kiến trúc và phần còn lại đơn giản là bị clock chậm hơn.

Cả hai phần đều giải thích được ở mức vòng lặp trong. Mỗi mẫu đầu ra cần tám section biquad (bốn cho mỗi dải, hai dải), nên workstation tiêu khoảng 5.7 chu kỳ mỗi section và board khoảng 28. Một biquad direct-form-II transposed là năm phép multiply-accumulate với phụ thuộc tuần tự giữa các mẫu liên tiếp: 5.7 gần với chi phí của một chuỗi bị giới hạn bởi phụ thuộc trên một core out-of-order, và 28 là chi phí kỳ vọng của cùng năm phép toán đó dưới dạng lệnh VFP trên một Cortex-M4F in-order, cộng overhead vòng lặp. Không con số nào bất thường, và đó là điểm mấu chốt — phép so sánh đang đo hai kiến trúc chứ không phải một artefact.

Đây là luận điểm thực chất cho bản port, phát biểu ở đúng sức mạnh thật của nó: một vi điều khiển từ bỏ khoảng hệ số năm về hiệu quả mỗi chu kỳ ở đây, không phải hệ số 275 mà thời gian thô gợi ý, và khoảng dư $72\times$ thời gian thực khiến điều đó chấp nhận được. Cấu trúc tuần tự của một cascade IIR là thứ giữ cho hình phạt kiến trúc nhỏ như vậy — độ rộng issue và thực thi out-of-order phải vật lộn với một chuỗi phụ thuộc. Một phương pháp có instruction-level parallelism khai thác được, hoặc có working set vừa cache desktop nhưng không vừa 256 kB SRAM, sẽ cho tỉ số tệ hơn nhiều; Section 10.8 nói về điều gì xảy ra với các phương pháp đó ở đây vì một lý do cơ bản hơn.

Hàng cuối là cùng silicon đó trước khi sửa build ở Section 10.4, ở 3,997 chu kỳ mỗi mẫu — hình phạt mỗi chu kỳ $88\times$ so với workstation thay vì $4.9\times$. Nó được giữ lại vì đó là điều phép so sánh này sẽ kết luận từ một phép đo trông hoàn toàn hợp lý vào thời điểm đó: không phải “toolchain bị cấu hình sai” mà là “vi điều khiển kém hiệu quả mỗi chu kỳ gần hai bậc độ lớn so với desktop, đó đơn giản là cái giá của DSP nhúng”. Kết luận đó sẽ sai, dễ trích dẫn, và rất khó gỡ bỏ về sau.

Phía server của bảng này cần cùng một sự hiệu chỉnh vì cùng một lý do. Phiên bản đầu của nó đo một hàm thiết kế hệ số bộ lọc ngay bên trong lời gọi được đo — việc mà firmware không bao giờ làm, vì hệ số của nó là hằng số thời-gian-biên-dịch trong flash — và do đó tính cho workstation $2.8\times$ chi phí thật, điều này sẽ báo cáo hình phạt kiến trúc là $1.7\times$ thay vì $4.9\times$. Con số đó cũng ổn định qua các lần lặp và hợp lý về bề ngoài. Hai trường hợp độc lập trong một phép so sánh là lý lẽ hợp lý để coi “con số trông hợp lý” là không mang chút trọng lượng chứng cứ nào.

10.7 Năng lượng, nơi câu trả lời chưa được biết

Throughput là trục sai cho một thiết bị chạy pin, thứ được đánh giá bằng joule. Hai cái không nhất thiết đồng thuận: một bộ xử lý chậm hơn $276\times$ nhưng tiêu thụ ít hơn $1000\times$ công suất vẫn thắng về năng lượng. Một nửa phép so sánh đó đo được ở đây và một nửa không, và nửa không đo được hóa ra là nửa quyết định.

Phía server được đo, không cần thiết bị đo. CPU Intel để lộ một bộ đếm năng lượng package qua RAPL dưới dạng một file thông thường trong `/sys`, tính bằng microjoule, nên lấy mẫu nó ở hai đầu của một khối lượng việc đã biết cho ra năng lượng thật thay vì một ước lượng TDP. Công suất package khi rảnh là 28.0 W ; dưới đúng khối lượng việc của board nó là 46.7 W , nên bản thân phép tính tiêu 18.7 W trên mức rảnh. Ở 0.75 ms mỗi clip đó là **14.3 mJ** biên (marginal), hoặc 35.8 mJ nếu tính toàn bộ package — trung vị của năm lần chạy, độ phân tán 2%. Cả hai đều được báo cáo vì con số đúng phụ thuộc vào câu hỏi: biên nếu server dù sao cũng đã tồn tại và câu hỏi là thêm một clip nữa, toàn package nếu một thiết bị chuyên dụng đang được so với một máy chuyên dụng.

Phía board không được đo, và không thể đo từ đây. Nano 33 BLE Sense không có cảm biến dòng tích hợp, nên firmware không thể tự báo cáo mức tiêu thụ của mình và không gì trên server có thể suy ra nó. Thứ được báo cáo thay vào đó là *điểm hòa vốn* (break-even): vì thời gian mỗi clip của board đã biết chính xác (208.5 ms), năng lượng là thời gian đó nhân với con số còn thiếu duy nhất, và điểm cân bằng đến ở **69 mW** so với chi phí biên của server, hoặc 172 mW so với toàn package của nó.

Con số đó là lý do không thay thế bằng một số liệu từ datasheet. Một board phát triển với USB, bộ điều áp, cảm biến tích hợp và một LED đang bật có thể nằm ở bất kỳ phía nào của 69 mW một cách hợp lý, nên kết luận lật ngược ngay trong phạm vi mà một giả định hợp lý sẽ bao phủ. Nơi một phép so sánh không nhạy với một giả định, giả định là được; phép so sánh này thì đảo chiều, và một kết luận bị đảo chiều khẳng định từ datasheet sẽ tệ hơn một khoảng trống được thừa nhận. Một đồng hồ đo công suất USB inline trong một lần chạy `*-bench` sẽ giải quyết nó đến bậc độ lớn mà lập luận cần.

Hai điều đáng lưu ý về những gì một phép đo như vậy sẽ cho thấy. Nếu board thực sự dưới 69 mW , thì nền tảng chậm hơn $276\times$ đồng thời rẻ hơn mỗi clip, và lập luận triển khai đảo ngược so với Table 25 — đó là kết quả mà phần này tồn tại để đánh dấu là khả dĩ chứ không phải để tuyên bố. Và cùng phép đo đó sẽ định giá cơ build của Section 10.4: binary trước khi sửa chạy lâu hơn $18\times$ cho cùng đầu ra, nên ở mức tiêu thụ công suất tương đương nó cũng tốn $18\times$ năng lượng, điều mà trên pin là khác biệt giữa một ngày và hai tuần.

10.8 Tại sao năm baseline còn lại không chạy được ở đây

Port Baseline 1 đặt ra câu hỏi hiển nhiên: có gì *tốt hơn* mà cũng vừa không? Thử port thêm năm cái nữa để tìm hiểu sẽ tốn kém, nên `src/mcu_feasibility.py` mô hình hóa bộ nhớ làm việc đỉnh, lưu trữ tham số và thời gian tính toán của từng phương pháp so với cả hai mục tiêu, suy ra mọi con số từ chính các hằng số được import của phương pháp.

Yếu tố phân biệt không phải là số học. Một phương pháp là *streaming* nếu nó tạo đầu ra từ một cửa sổ đầu vào bị chặn trong khi chỉ mang trạng thái cố định giữa các lần gọi, và là *batch* nếu toàn bộ bản ghi phải thường trú trước khi bắt kỳ đầu

ra nào tồn tại. Baseline 1 là streaming: 16 float trạng thái biquad, y hết dù bản ghi dài 15 giây hay một giờ. Tính chất đó, chứ không phải số MAC, quyết định phần lớn Table 26.

Table 26: RAM làm việc đỉnh so với 211 kB còn lại trên một Nano 33 BLE Sense sau footprint của chính firmware Baseline 1. Mọi con số ở float32.

Phương pháp	Streaming	RAM đỉnh	Kết luận
Baseline 1 (bandpass)	có	832 B	vừa ($0.004\times$)
Baseline 2 (supervised NMF)	có	9.1 kB	vừa ($0.04\times$)
Baseline 3 (standard NMF)	không	1.0 MB	vượt $4.7\times$
Baseline 4 (MSSA)	không	36 MB	vượt $167\times$
Baseline 5 (EVMD)	không	15.4 MB	vượt $71\times$
Baseline 6 (Conv-TasNet-lite)	không	9.2 MB	vượt $43\times$

Baseline 4 và 5 là batch theo cấu trúc, không phải theo triển khai. SSA nhúng tín hiệu vào một ma trận quỹ đạo $L \times K$ ($L=50$, $K=59,951$), lấy SVD của nó, và tái tạo từng thành phần bằng phép lấy trung bình đường chéo ngược lại trên toàn bộ chiều dài; mọi cấu trúc đó đều là $O(LN)$, và phép tái tạo tham chiếu toàn bộ tín hiệu. ADMM của VMD lặp trong miền tần số trên tín hiệu đã mirror-pad, giữ sống tất cả K mode và phổ của chúng qua mọi vòng lặp. Không cái nào có một công thức streaming để có thể viết cẩn thận hơn — bản thân các thuật toán được định nghĩa trên toàn bộ bản ghi.

Baseline 6 là batch chỉ theo cách viết hiện tại. Nó mã hóa toàn bộ clip và chạy TCN trên toàn bộ 6,000 frame thu được cùng lúc, nên 9.2 MB của nó là activation chứ không phải trọng số — tham số là 318 kB ở int8 và vừa flash thoải mái. Một Conv-TasNet theo chunk, nhân quả là một biến thể đã biết và sẽ làm activation $O(\text{chunk})$; đó là một thiết kế lại mô hình, không phải một bước triển khai, và đó là mô tả trung thực về điều đứng giữa phương pháp này và board.

Lượng tử hóa không cứu được cái nào trong số đó. Chuyển từ float32 sang int8 chia bộ nhớ cho bốn, để lại Baseline 3 vượt $1.2\times$, Baseline 6 vượt $10.7\times$, Baseline 5 vượt $17.8\times$ và Baseline 4 vượt $41.6\times$. Không cái nào vừa. Căn bản hơn, với phần lớn chúng int8 hoàn toàn không phải một biểu diễn áp dụng được: dung sai hội tụ ADMM của Baseline 5 là 10^{-6} , thấp hơn bốn bậc độ lớn so với độ phân giải khoảng 4×10^{-3} của int8, nên solver không thể phát hiện chính sự hội tụ của mình; Baseline 4 chọn thành phần bằng ngưỡng đóng góp trị riêng 2% trên các giá trị kỳ dị trải nhiều bậc độ lớn, điều 256 mức không thể biểu diễn; và các multiplicative update của NMF mang một sàns epsilon 10^{-12} . Chỉ Baseline 6 là mục tiêu lượng tử hóa tự nhiên — post-training quantisation cho mạng neural đã chín muồi và có công cụ tốt — và với nó lượng tử hóa là bước cuối cùng sau khi thiết kế lại theo streaming, không phải một lựa chọn thay thế.

Điều này có nghĩa gì cho câu hỏi triển khai. Trên tập con additive gốc (Table 10), phương pháp duy nhất thắng Baseline 1 trên bất kỳ nguồn nào là Baseline 4 trên phổ, với 1.34 dB — và Baseline 4 là mục kém khả thi nhất trong bảng. Mọi phương pháp có thể chạy ở đây đều điểm thấp hơn Baseline 1 trên cả hai nguồn. Phát hiện vì thế là một kết quả âm, và đáng nói thẳng: trên tập dữ liệu này, với sáu phương pháp này, phương pháp rẻ nhất cũng là phương pháp hoạt động tốt nhất, đó là lý do

Section 9.3 đã đặt nó trên Pareto front của cả hai mặt phẳng compute. Baseline 2 vẫn là bản port không tầm thường duy nhất sẽ vừa, và giá trị của nó sẽ là chứng minh chuỗi xác minh của firmware khái quát được sang một thuật toán khác về căn bản — FFT và cập nhật ma trận lặp thay vì một cascade biquad — chứ không phải cải thiện chất lượng phân tách.

Một lưu ý về cột thời gian. Các ước lượng compute của mô hình giả định một MAC mỗi chu kỳ với toán hạng trong thanh ghi. Hiệu chuẩn với phương pháp duy nhất hiện đã được đo trên silicon, nó lạc quan $5.6\times$ ($400\times$ dự đoán, $72\times$ đo được). Áp dụng hệ số đó cho Baseline 2 sẽ đưa nó về $0.8\times$ thời gian thực thay vì $4.4\times$ — nên biên độ thời gian thực của nó thực sự không chắc chắn, và sẽ cần đo thay vì khẳng định. Các cột bộ nhớ, thứ quyết định mọi hàng không khả thi, không bị ảnh hưởng.

11 Công Trình Liên Quan

Table 27 tóm tắt sáu bài báo được rà soát đối chiếu với công trình này, được tổ chức theo những gì mỗi bài đo. Chi tiết trích dẫn đầy đủ cho cả sáu bài giờ đã được điền vào bên dưới, lấy nguồn từ Notion Reading List; năm trong số sáu bài đã được đối chiếu chéo độc lập với các PDF nguồn chính của chúng thay vì dựa vào các bản tóm tắt gián tiếp ([5], một luận án tiến sĩ, vẫn là ngoại lệ duy nhất — được đánh dấu trong bibliography).

Table 27: Công trình liên quan, được tổ chức theo những gì được đo.

Bài báo	Dataset	Phương pháp phân tách	Metric phân tách	Tác vụ hạ nguồn	Cách chia
HLS-CMDS de-scriptor [1]	— (nguồn)	—	—	—	—
AI-Driven Cardiorespiratory Signal Processing [5]	HLS-CMDS	PL-NMF + tần số cơ bản dẫn dắt bởi LLM (“LingoNMF”)	không báo cáo (paired t -test trên một proxy không xác định)	clustering + phát hiện bất thường, không phải accuracy	không nêu
Spectrotemporal DL for Heart Sound Classification [6]	PhysioNet 2016 (huấn luyện) + HLS-CMDS (external validation, Sec. 5 của bài báo)	bandpass filter (bài kiểm tra stress Experiment 4 của họ: âm tim được phân tách bằng tính toán từ các bản ghi tim-phổi mixed của HLS-CMDS)	không có (không có SDR/SIR/SAR — chỉ accuracy hạ nguồn được báo cáo)	phân loại PCG 4 lớp (Normal/-Murmur/Extra Sound/Rhythm Disorder); accuracy sup đố 89.0%→41.0% (Exp. 3 sạch so với Exp. 4 đã phân tách) — xem Sec. 11.1 bên dưới, đây là kết quả gọi mở cho toàn bộ câu hỏi nghiên cứu của dự án này	không nêu
NMF-Enhanced Log-Mel + CRNN [7]	ICBHI 2017 + Fraiwan CWLS	tăng cường dựa trên NMF	không có (không có SNR/SDR/-correlation)	phân loại bệnh hô hấp đa lớp	cấp bản ghi (leakage được các tác giả tự nêu)
Edge-Enabled Portable Lung-Sound Classifier [8]	HLS-CMDS	tách âm phổi được đề xuất từ các mixture	không có	phân loại âm phổi	không nêu
Performance Measurement in Blind ASS [2]	— (phương pháp)	—	định nghĩa SDR/SIR/SAR	n/a	n/a
Cardiorespiratory Separation via SSA [9]	không nêu	Singular Spectrum Analysis hai giai đoạn	SDR, STOI, correlation so với 5 baseline	không có	n/a

11.1 Yaqub et al.: kết quả gọi mở, không phải một bài báo phân loại không liên quan

Một bản nháp trước đó của bảng này xếp [6] vào mục “PhysioNet 2016, không có phân tách, phân loại nhị phân normal/abnormal.” Điều đó sai ở mọi khía cạnh quan trọng, giờ đã được xác nhận bằng cách đọc trực tiếp nguồn chính thay vì một bản tóm tắt gián tiếp. Yaqub et al. huấn luyện một classifier PCG ResNet-18 trên PhysioNet 2016, sau đó dùng HLS-CMDS thuần túy cho *external validation* qua bốn thí nghiệm: (1) fine-tuning nhị phân, 88.0% accuracy; (2) fine-tuning 4 lớp, 86.0%; (3) huấn luyện lại hoàn toàn từ trọng số ImageNet trên chính các nhãn 4 lớp của HLS-CMDS, 89.0% (kết quả tốt nhất, cân bằng nhất của họ); và (4) một bài kiểm tra stress đánh giá lại mô hình Experiment-3 trên âm tim *được phân tách bằng tính toán từ các bản ghi tim-phổi mixed của HLS-CMDS bằng một bộ lọc bandpass tiêu chuẩn* — nơi accuracy sụp đổ xuống 41.0% (Table 9 của [6]: $n=5365$, weighted F1 giảm từ 0.89 xuống 0.39, precision và recall của mọi lớp đều suy giảm, không chỉ một lớp).

Đây không phải là một kết quả bên lề. Đây là phát hiện thực nghiệm duy nhất mà toàn bộ câu hỏi nghiên cứu của dự án này tồn tại để giải thích: việc phân tách một bản ghi cardiorespiratory mixed trước khi phân loại nó tốn kém điều gì đó, và Yaqub et al. là những người đầu tiên đo chi phí đó trực tiếp, end to end, trên chính tập dữ liệu của dự án này. Chính phần Discussion của họ (Sec. 6.2) quy sự sụp đổ này cụ thể cho “các méo dạng tinh vi và khả năng mất thông tin do bộ lọc bandpass tiêu chuẩn dùng cho source separation gây ra” và “hạn chế cố hữu... để phân tách hoàn hảo các tín hiệu chồng lấp phổ” — đây chính xác là cùng loại phương pháp như Baseline 1 của chính báo cáo này (Section 6.4), và chính xác là đánh đổi SIR-so-với-SAR mà các kết quả bandpass của chính dự án này chứng minh trực tiếp về phía chất lượng phân tách (mức tăng SIR thực, chi phí SAR thực). Yaqub et al. cho thấy đánh đổi đó tốn kém gì đối với một classifier hạ nguồn; các metric chất lượng phân tách của dự án này là cơ chế dự đoán khi nào điều đó sẽ xảy ra. Con số 89%→41% của họ là một điểm trên đường cong mà phép so sánh Condition A/B của dự án này (Section 13) được xây dựng để vẽ ra liên tục theo SDR, không phải một giai thoại hai điểm để tái hiện một lần.

11.2 Các lưu ý về metric từ tài liệu BSS Eval

Hai điểm từ [2] định hình trực tiếp thiết kế trong Section 6.1–6.4: (1) SAR nhạy cảm với méo dạng mà accuracy của một classifier hạ nguồn có thể hoàn toàn không ghi nhận (một classifier có thể vẫn đúng bất chấp artifact tín hiệu thực sự, nếu méo dạng đó không vượt qua một ranh giới quyết định), điều này thúc đẩy việc báo cáo SDR/SIR/SAR thay vì chỉ accuracy; và (2) SDR không phải là một thước đo cảm nhận thính giác (perceptual) và có thể bị đánh lừa (gameable) — một ước lượng được lọc thông thấp nhẹ có thể đẩy SDR về phía $+\infty$ dưới một bộ lọc biến đổi theo thời gian mà ước lượng đó thực sự không hề cải thiện — đây là lý do tại sao Tables 9 và 10 được báo cáo với cả ba metric thay vì chỉ SDR đơn lẻ. Cùng nguồn đó lưu ý rằng các mixture của HLS-CMDS chỉ có tim+phổi, không có tham chiếu nhiều cảm biến độc lập, do đó số hạng nhiễu trong một phép phân rã BSS Eval đầy đủ dựa trên images về cơ bản bằng không ở đây; SAR trong báo cáo này phản ánh artifact thuật toán, không phải khả năng chống chịu với nhiễu thu âm thực.

Một sự đối chiếu chéo khác xuất hiện trong lúc rà soát [8]: số lượng lớp phổi HLS-CMDS mà bài báo đó nêu ra (Normal 28, Wheezing 28, Pleural Rub 25, Rhonchi 23,

Fine Crackles 22, Coarse Crackles 19) khớp với số lượng *tổng-trong-Mix.zip* đã được mã hóa sẵn trong `load_dataset.py` của repository này (cột `mix_zip` của `PAPER_LUNG_COUNTS`, đã được xác thực đối chiếu với các CSV trực tiếp), không phải số lượng nhỏ hơn của `LS.csv isolated` (5–12 mỗi lớp). Hai con số này dễ bị nhầm lẫn với nhau và bất kỳ báo cáo nào dựa trên tập dữ liệu này nên nói rõ nó đang trích dẫn con số nào.

12 Thảo Luận

Trong sáu bài báo được rà soát, đúng một bài — [8] — đánh giá trên HLS-CMDS và phân loại âm thanh đã phân tách từ mixture, nhưng báo cáo không một metric chất lượng phân tách nào và, theo bản tóm tắt được rà soát, không so sánh với việc phân loại trực tiếp ground truth isolated. Bài báo SSA [9] báo cáo các metric phân tách đúng dẫn nhưng hoàn toàn không có tác vụ phân loại; bài báo NMF+CRNN [7] báo cáo accuracy phân loại với các con số thực nhưng trên một tập dữ liệu khác, không có metric phân tách, và chính các tác giả của nó nêu ra một cách chia cấp bản ghi (dễ bị leakage) như một hạn chế. Không có bài báo nào được rà soát báo cáo cùng lúc chất lượng phân tách, accuracy phân loại âm thanh đã phân tách, và accuracy phân loại ground truth isolated, dưới một cách chia đã được xác minh không rò rỉ bản ghi giữa pool được dùng để fit bất cứ thứ gì và pool được dùng để đánh giá nó.

Điều kiện thứ tư đó không phải là một mối lo mang tính giả thuyết đối với riêng tập dữ liệu này: audit của Section 4 phát hiện rằng một phần ba số bản ghi isolated là bản sao y hệt của các bản ghi được nhúng trong các mixture, và rằng các hàng mix nối chuỗi với nhau thành các leak group lớn đến 32 trong số 145 hàng. Một cách chia cấp bản ghi — chính hạn chế mà [7] nêu ra trong kết quả của chính nó — sẽ làm rò rỉ ground truth vào huấn luyện trên tập dữ liệu này theo một cách rất cụ thể.

Kết quả Baseline 1 (Section 7) chứng minh đánh đổi SIR/artifact mà Section 11.2 dự đoán trên dữ liệu thực thay vì chỉ khẳng định nó từ lý thuyết: mức tăng SDR của một phương pháp phân tách thực, dù thô sơ, so với việc không làm gì cả nhỏ hơn nhiều so với những gì chỉ riêng mức tăng SIR sẽ gợi ý, vì SAR di chuyển mạnh theo chiều ngược lại. Đây là lập luận cụ thể cho việc báo cáo accuracy phân loại cùng với SDR/SIR/SAR trong giai đoạn tiếp theo đã lên kế hoạch (xem `PROTOCOL.md`) thay vì chỉ riêng một họ metric nào: một phương pháp phân tách đánh đổi SIR lấy SAR theo cách này có thể hoặc không thể gây hại cho một classifier hạ nguồn, và đó là một câu hỏi thực nghiệm mà pipeline này được xây dựng để trả lời, không phải một câu hỏi mà báo cáo này tự trả lời được.

Phép so sánh của Baseline 2 với Baseline 1 nêu ra một mối lo thứ hai, cụ thể hơn. Trên toàn bộ tập hàng, hai phương pháp gần như hòa nhau; trên tập con 36 hàng nơi việc phân tách thực sự khả thi (Section 5), supervised NMF an toàn rò rỉ rõ ràng *kém hơn* bộ lọc zero-training, chứ không phải tương đương với nó. Hai lời giải thích không loại trừ lẫn nhau xuất phát từ cách pipeline này được xây dựng. Thứ nhất, việc bịt kín khoảng hở leakage (Section 4) loại bỏ chính xác loại thổi phồng mà một bản tái hiện ngẫu nhiên sẽ giữ lại — theo giả định làm việc của chính dự án này, một con số thấp hơn, an toàn rò rỉ mới là con số *đúng*, không phải bằng chứng rằng bản thân phương pháp bị hỏng. Thứ hai, khoảng cách lớn hơn nhiều và nhất quán hơn trên tập con hợp lệ so với trên toàn bộ tập, đây là điều ngược lại với những gì chỉ riêng sự thổi phồng do leakage sẽ dự đoán (leakage lẽ ra sẽ thổi phồng trường hợp dictionary đã fit của NMF bất kể tính hợp lệ của hàng); điều này thay vào đó chỉ ra rằng hạng dictionary

cố định ($K_i=10$, $K_r=20$) hoặc số lượng vòng lặp cố định (Section 6.5) không chuyển giao tốt sang các mixture âm học thực của tập dữ liệu này, khác với các mixture được cộng tổng hợp bằng số hay đã được tuyển chọn mà một phương pháp đã công bố có thể đã được tinh chỉnh dựa trên đó. Việc phân biệt các lời giải thích này chưa được giải quyết bởi báo cáo này. Một lời giải thích thứ ba, lạnh lùng hơn đã được nêu ra một cách không chính thức khi Baseline 2 đang được xây dựng: một mask *oracle* được xây từ chính các spectrogram tim/phổi thực của một hàng mixture — ăn gian có chủ đích, chỉ để giới hạn trần của phương pháp — có vẻ chỉ vượt qua tham chiếu không-phân-tách một cách khiêm tốn, điều này nếu đúng sẽ có nghĩa là các bản ghi mixture thực của tập dữ liệu áp đặt một trần cứng lên phân tách dựa trên mask mà không mức độ tinh vi nào của NMF khắc phục được. Phép thăm dò đó nay là một script đã commit (`oracle_mask_ceiling.py`) chạy trên toàn bộ 36 hàng hợp lệ trên chính lưới STFT của Baseline 2, và nó không sống sót qua lần kiểm tra lại. Ideal ratio mask — đúng dạng mask của Baseline 2 với biên độ thực thay vào — đạt 11.5 ± 1.9 dB SDR tim và 9.2 ± 1.7 dB phổi (CI 95% trên các hàng), so với 2.5 và -2.0 dB của mixture thô và 5.5 và 4.1 dB của bandpass; ideal Wiener filter đạt 12.4 và 10.6 dB. Vì thế Baseline 2 nằm dưới trần của chính họ phương pháp của nó 7.8 dB (tim) và 9.7 dB (phổi), và sự thiếu hụt là của dictionary và activation, không phải của tập dữ liệu. Ghi chú một-hàng gần như chắc chắn được thực hiện trên một hàng không additive, nơi Section 5 cho thấy mọi phương pháp — kể cả oracle — thừa hưởng năng lượng của mixture không liên quan như méo không loại bỏ được. Phép kiểm tra có xác lập một trần thuộc loại khác: một mask biên độ trên STFT 512 điểm đạt tối đa gần 12 dB trên các bản ghi này, thấp hơn vùng 15–20 dB nơi classifier thứ nhất của Section 8.8 bắt đầu ưa thích đầu ra của một bộ phân tách lược bỏ phổ. Liệu một lưới thời gian-tần số mịn hơn hay một bộ phân tách miền thời gian có vượt qua được mức trần đó hay không chưa được kiểm tra ở đây.

13 Hạn Chế và Công Việc Tương Lai

- **Tần số lấy mẫu được nêu trong descriptor vẫn chưa được đối chiếu thống nhất.** Tập dữ liệu đã phát hành đo được 4000 Hz xuyên suốt (Section 3.1) trái ngược với 22,050 Hz mà bài báo descriptor của nó nêu ra. Bản thân nguồn gốc (provenance) đã được giải quyết — bản sao được audit ở đây giống hệt từng byte với bản phát hành Mendeley — do đó câu hỏi còn mở hẹp hơn so với lúc đầu: liệu descriptor có mô tả sai tần số, hay con số bị chép sai ở khâu sau đó. Việc giải quyết điều này đối chiếu với chính văn bản của descriptor, kèm tham chiếu trang, vẫn còn tồn đọng, và nó quyết định mức độ mạnh mẽ mà bình luận audit tập dữ liệu đi kèm có thể diễn đạt khuyến nghị.
- **Các quỹ đạo đã hoàn thành nhưng độ phân giải theo chiều dọc của chúng là 36 hàng, và một giao cắt là ước lượng điểm từ bảy mức lấy mẫu.** Đường cong của Section 8.8 và lần lặp lại của nó ở Section 8.9 nay là kết quả sáu-baseline đầy đủ, và cả hai nói cùng một điều về mặt định tính. Nhưng mọi điểm trên chúng là một accuracy trên 36 bản ghi — một hàng là 2.8 điểm, và khoảng tin cậy nhị thức 95% quanh 40% là khoảng ± 16 điểm — đó là lý do các giá trị knee dịch 6–11 dB giữa hai classifier, là lý do khoảng bootstrap của chúng chạm đỉnh lưới đo (Table 18), và là lý do báo cáo này trích dẫn một vùng giao cắt

theo một giao thức nội suy được nêu rõ, không phải một ngưỡng. 36 hàng cũng không phải 36 đơn vị độc lập: chúng rơi vào 24 leak group (một nhóm 5, một nhóm 3, sáu nhóm 2, mười sáu nhóm đơn), nên cỡ mẫu hiệu dụng nằm giữa 36 (không tương quan nội nhóm) và 17.5 (Kish, tương quan hoàn toàn); tương quan nội nhóm thực nghiệm của độ đúng theo hàng là 0.24 trên âm thanh isolated và gần không trên âm thanh đã phân tách, đặt nó gần 29. Bootstrap theo leak group của Table 18 đã tính đến điều này và không nói rộng gì quá một decibel. Thu hẹp điều đó cần nhiều hàng hợp lệ hơn HLS-CMDS có, và đó là việc của nền tổng hợp (Section 6.3); chạy sweep và cả hai classifier trên 1500 hàng của nó là bước tiếp theo trực tiếp.

- **Kiến trúc 2 được tinh chỉnh một lần, bằng tay.** CNN của Section 8.9 đi từ 30.0% lên 62.0% nhờ chuẩn hóa đặc trưng và một lịch huấn luyện mini-batch dài hơn, với ba cấu hình được thử và việc lựa chọn dựa trên độ khớp tập huấn luyện; đó là một cuộc tìm kiếm nhỏ, được công khai, nhưng vẫn là một cuộc tìm kiếm, và cấu hình được chọn cũng là cấu hình có accuracy test tốt nhất vào thời điểm đó. Phép chuẩn hóa mức âm lượng thêm vào sau giữ nguyên accuracy của nó và thu hẹp CI xuống ± 3.9 điểm. Một phép chọn lồng nhau (nested), hoặc đơn giản là một seed thứ hai, sẽ cho biết bao nhiêu phần của 62.0% là do cấu hình và bao nhiêu là do cách rút fold — chưa cái nào được chạy.
- **Yếu tố gây nhiễu mức âm lượng được bắt nhờ triệu chứng, không phải nhờ thiết kế.** Lần chạy đầu của Section 8.7 sai theo cách tạo ra một bảng tổng hợp lý, và chỉ được phát hiện vì ba phương pháp không liên quan trả về dự đoán y hệt nhau. Các classifier nay bất biến với mức âm lượng, nhưng chưa có gì kiểm tra rằng đầu vào đã phân tách rơi vào vùng mà kernel của SVM còn mang thông tin; một báo cáo khoảng-cách-chuẩn-hóa cho từng điều kiện sẽ làm khiếm khuyết tiếp theo kiểu này lộ ra trước khi được công bố.
- **Sáu phương pháp phân tách được đánh giá; không phương pháp nào thống trị hoàn toàn.** Baseline 1, 4, và 5 là các sản phẩm kiểm tra tính hợp lý zero-training theo thiết kế; Baseline 2, 3, và 6 liên quan đến một dạng fit nào đó. Section 7.3 cho thấy Baseline 6 (Conv-TasNet-lite) đạt điểm tốt nhất trên SDR nền synthetic cho cả hai nguồn và trên SDR tim native, nhưng thua Baseline 4 (MSSA) trên SDR phổi native nhiều hơn nửa độ rộng CI của cả hai phương pháp — do đó pipeline này vẫn chưa có một phương pháp duy nhất chứng minh được sự cải thiện rõ ràng và nhất quán so với baseline zero-training mạnh nhất trên cả hai nguồn và cả hai nền dữ liệu. NMF dẫn dắt bởi LLM (“LingoNMF”), phương pháp được rà soát duy nhất chưa được tái hiện (Table 27), vẫn là một ứng viên còn để ngỏ.
- **Baseline 6 là một baseline neural đầu tiên chưa được huấn luyện đủ, không phải một mô hình đã tinh chỉnh.** Table 12 cho thấy mọi fold đều tiêu tốn hết ngân sách 25-epoch mà tiêu chí dừng sớm không bao giờ kích hoạt, trên một pool ~ 32 bản ghi huấn luyện mỗi lớp mỗi fold — phù hợp với một kết quả bị giới hạn bởi ngân sách compute chứ không phải đã hội tụ (Section 6.9). Kết quả của nó nên được đọc như xác thực hạ tầng, không phải kết quả neural tốt nhất có thể đạt được của dự án này; một ngân sách huấn luyện dài hơn và/hoặc augmentation tại chỗ mạnh tay hơn là các bước tiếp theo tự nhiên trước khi coi

thứ hạng của Baseline 6 so với Baseline 1-5 là cuối cùng.

- **Cột nền synthetic của Baseline 5 là một mẫu con, không phải toàn bộ tập.** Sweep VMD $K=2..10$ đầy đủ trên toàn bộ synthetic $n \approx 1500$ được đo là ~ 15 s/mixture (Section 6.8), ngoại suy ra thời gian chạy nhiều giờ; do đó cột synthetic của nó trong Table 11 chạy trên một mẫu con phân tầng 75 hàng, cho nó một CI rộng hơn rõ rệt so với các cột 1500-hàng của năm phương pháp còn lại. Đây là một sự thu hẹp phạm vi được công khai, không phải một phép so sánh ngang hàng (like-for-like), và không nên được đọc như việc Baseline 5 nhiều hơn theo nghĩa tuyệt đối.
- **Khoảng hệ số khuếch đại của tập synthetic được fit từ một mẫu nhỏ.** Phân phối hệ số khuếch đại của `synthetic_mix.py` (Section 6.3) được fit từ 36 hàng additive gốc thay vì được chọn tùy tiện, đây là một điểm mạnh của cách xây dựng và đồng thời cũng là một sự phụ thuộc của nó: nền synthetic thừa hưởng bất cứ điều gì đặc thù về các hệ số khuếch đại mixing riêng của 36 hàng đó ($3.5-135\times$), và $n=36$ là một mẫu nhỏ để fit một khoảng từ đó. Một kiểm tra độ nhạy (sensitivity check) — xây dựng lại tập synthetic dưới một khoảng hệ số khuếch đại cố ý rộng hơn và một khoảng cố ý hẹp hơn, sau đó chạy lại một baseline — sẽ giới hạn mức độ Table 11 phụ thuộc vào phép fit đó.
- **Siêu tham số của Baseline 2: đã xác minh với nguồn chính, kèm hai hiệu chỉnh.** Một bản trước đó liệt kê các hạng dictionary NMF và số vòng lặp (Section 6.5) là được truyền đạt gián tiếp và chưa xác minh. Chúng đã được đối chiếu trực tiếp với [7] (Sections 3.2.3-3.2.4 của bài): $K_t=10$, $K_r=20$, 100 và 60 vòng lặp multiplicative-update cùng của sổ STFT được xác nhận đúng, và việc đối chiếu tìm thấy hai sai khác đã được sửa — hop STFT trước đó là 128 chưa kiểm chứng (của bài là 256), và bài báo khử nhiễu mọi đoạn bằng bandpass 50-1800 Hz trước STFT, một giai đoạn mà bản tái hiện đầu tiên thiếu. Không thay đổi nào dịch SDR toàn tập của Baseline 2 quá độ phân tán giữa các fold, nên sự thiếu hụt của phương pháp so với Baseline 1 không do hai điều đó gây ra. Mô tả tập huấn luyện của chính bài báo vẫn mơ hồ giữa ICBHI2017+Fraivan và các bản ghi isolated của HLS-CMDS (Table 27); dự án này fit trên các bản ghi isolated của HLS-CMDS dưới pool an toàn rõ ràng và nói rõ điều đó.
- **Trần oracle-mask nay đã được đo, và nó bác bỏ ghi chú không chính thức đã khơi mào nó.** `oracle_mask_ceiling.py` chạy ba oracle mask trên toàn bộ 36 hàng hợp lệ (Section 12); ideal ratio mask đạt 11.5 dB SDR tìm so với 2.5 dB của mixture thô, nên không có trần nào do tập dữ liệu áp đặt gần sàn không-phân-tách. Điều phép kiểm tra xác lập là một trần khác: một mask biên độ trên STFT 512 điểm không thể vượt quá khoảng 12 dB ở đây, thấp hơn vùng knee của Kiến trúc 1. Liệu một lưới mịn hơn hay một phương pháp miền thời gian có nâng được mức trần đó hay không chưa được kiểm tra.
- **Không có tuyên bố về tính vững trước nhiễu cảm biến.** Như đã lưu ý ở Section 11.2, các mixture của HLS-CMDS không chứa nguồn nhiễu độc lập nào, do đó SAR của pipeline này không nói lên được gì về tính vững trước nhiễu thu âm thực (liên quan đến các classifier triển khai trên phần cứng như của [8]).

- **Tính đầy đủ của trích dẫn.** Chi tiết trích dẫn thư mục đầy đủ (tác giả, venue, năm) cho bốn trong sáu bài báo công trình liên quan chưa có sẵn khi báo cáo này được soạn thảo ([7] đã được giải quyết từ đó, Section 11); các mục còn lại nên được hoàn thiện trước bất kỳ lần nộp bên ngoài nào.
- **Kiểu chữ Table 6.** Bảng công trình liên quan được scale theo chiều rộng trang qua `resizebox` để vừa với sáu cột, điều này tạo ra một vài chỗ xuống dòng khó coi trong các ô hẹp (ví dụ “HLS-CMDS”, “SNR/SDR/correlation”). Chỉ mang tính thẩm mỹ, nhưng đáng để mở rộng thủ công (ví dụ chia thành hai bảng, hoặc xoay sang landscape) trước bất kỳ lần nộp hoàn thiện nào.
- **Chưa có CI wiring.** Bộ 31 test (Section 6.1.1) chỉ chạy khi được gọi thủ công (`make test`).
- **Các lớp âm tim nhỏ đe dọa độ phủ theo từng fold của classifier đã lên kế hoạch.** Table 2 (Section 3.2.1) cho thấy S4 ($n=2$), AV Block, và Tachycardia ($n=3$ mỗi lớp) nhỏ hơn nhiều so với các lớp tim khác. Dưới cách chia 5-fold tôn trọng leak-group, một lớp 2-bản-ghi không thể được đại diện trong tập huấn luyện của mọi fold; giai đoạn phân loại (PROTOCOL.md §5.3) nên quyết định trước liệu có gộp, loại trừ, hay phân tầng đặc biệt các lớp này thay vì phát hiện ra vấn đề như một dị thường accuracy theo từng lớp không giải thích được sau này.

References

- [1] Y. Torabi, S. Shirani, and J. P. Reilly, “Descriptor: Heart and Lung Sounds Dataset Recorded from a Clinical Manikin using Digital Stethoscope (HLS-CMDS),” *IEEE Data Descriptions*, 2025. doi:10.1109/IEEEDATA.2025.3566012.
- [2] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 4, pp. 1462–1469, 2006.
- [3] C. Raffel et al., “mir_eval: A Transparent Implementation of Common MIR Metrics,” in *Proc. ISMIR*, 2014.
- [4] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing Ideal Time-Frequency Magnitude Masking for Speech Separation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, no. 8, pp. 1256–1266, 2019.
- [5] Y. Torabi, “AI-Driven Cardiorespiratory Signal Processing: Separation, Clustering, and Anomaly Detection,” Ph.D. dissertation, McMaster University, 2025. (Cited via the Notion Reading List; not yet independently read from the primary source — see Section 13.)
- [6] A. Yaqub, M. S. Orakzai, M. F. Qureshi, Z. Mushtaq, I. Siddique, and T. Radwan, “Spectrotemporal Deep Learning for Heart Sound Classification under Clinical Noise Conditions,” *Computer Modeling in Engineering & Sciences*, vol. 145, no. 2, pp. 2503–2529, 2025. doi:10.32604/cmescs.2025.071571. (Independently read from the primary source — see Section 11.)

- [7] Y. Han, Z. Quan, K. Matuszewski, and B. Corbett, "Respiratory Disease Classification Using NMF-Enhanced Log-Mel Spectrograms and Convolutional Recurrent Neural Networks," *Sensors*, vol. 26, no. 13, art. 4268, 2026. doi:10.3390/s26134268. (Independently read from the primary source.)
- [8] A. Puneet, P. Shankar, S. R. R. Koluguri, and A. Srivastava, "Edge-Enabled Portable Classifier for Lung Sounds Using Convolutional Neural Networks," in *Proc. 2025 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, pp. 21–25, 2025. doi:10.1109/BioCAS67066.2025.00016. (Independently read from the primary source.)
- [9] B. Han and W. Quan, "Cardiorespiratory Sound Separation Using Singular Spectrum Analysis," in *Proc. 2025 17th Int'l Conf. on Signal Processing Systems (ICSPS)*, 2025. doi:10.1109/ICSPS66615.2025.11347745. (Independently read from the primary source.)